Printout Buddy

Architecture and Android API handoff

Backend: `origin/main` @ `080c6bd` · Production `https://printoutbuddy.tech` (API `/api/v1`)

32 database models · 113 registered routes · 26 diagrams · OpenAPI 3.0.3 spec in `docs/openapi/api-v1.yaml`

Generated 2026-10-27. Documentation only: no application code, schema or deployment was changed. No secrets are included.

Contents

1. Overview, verification and findings
2. Android integration guide (Kotlin) with examples
3. Sequence diagrams A-G
4. System architecture
5. Full database ER diagram
6. ER: authentication and identity
7. ER: shop and pricing
8. ER: orders and documents
9. ER: devices and printers
10. ER: notifications
11. ER: admin and security
12. ER: legacy billing
13. API catalog (all 113 routes)
14. API to database mapping

Printout Buddy: architecture and Android API handoff

Documentation-only deliverable for the Android developer. It describes the **existing, production backend on** `origin/main` (commit `080c6bd`, 2026-10) so the app can be integrated without reading the repository. No application code, schema, migration, deployment or production setting was changed to produce it.

- Production: `https://printoutbuddy.tech` (API prefix `/api/v1`)
- Repository: `singhdaksh7/printout-`
- Combined handoff PDF: `printout-buddy-android-handoff.pdf` (same content as the files below, in one document)
- OpenAPI 3.0.3 spec: `../openapi/api-v1.yaml` (113 operations)

Start here (Android)

1. android-integration-guide.md: auth, token handling, Kotlin examples, FCM, errors, offline.
2. sequence-diagrams.md: flows A-G (onboarding, sign-in, refresh, upload/order, printing, retention, pricing).
3. api-inventory.md: every route with auth, schemas, errors, idempotency, rate limits.
4. api-entity-map.md: endpoint ↔ tables read/written, and whether Android should use it.

Document index

File	Contents
full-er-diagram.md	All 32 models, enums, non-FK references
er-auth.md	User, identities, sessions, refresh tokens, onboarding grants
er-shop-pricing.md	Shop, settings, hours, pricing rules
er-orders-documents.md	Orders, documents, lifecycles, retention fields
er-devices-printers.md	Devices, pairing, printers, print-job state machine
er-notifications.md	Inbox/outbox, FCM registrations, SSE
er-admin-security.md	Admin 2FA, login challenges, audit log
er-legacy-billing.md	Plans, subscriptions, invoices, payments (platform is free)
system-architecture.md	Component and flow diagrams
sequence-diagrams.md	Sequence diagrams A-G
api-inventory.md / api-entity-map.md	API catalog and DB mapping
android-integration-guide.md	Kotlin integration guide

Headline numbers

Prisma models	**32** (24 in active use, 8 schema-only target tables)
Prisma enums	14
Registered routes	**113** (111 under `/api/v1` + `/health` + `/ready`)
Deprecated (retired, answers 410)	1 (`POST /shop/orders/:id/print-confirmation`)
Routes by area	Auth/onboarding 19, Profile 5, Settings 10, Pricing/quotes 5, Orders 11, Printing/devices 24, Notifications 6, Analytics 3, Customer public 6, Admin 21, Ops 3 (see inventory for the exact list)
Mermaid diagrams	**26** (9 ER, 3 state, 7 flowcharts, 7 sequence), all rendered without errors (see validation)

What is on `main` vs not

Item	Status
Web app, API, worker, Windows agent, mobile auth/profile/FCM APIs	**On** `main`
Phase A: live queue filters (`print=`, `sort`, `/shop/orders/counts`), customer tracking (`printIssue`, `serverTime`), privacy hardening	**On** `main` (PR #41)
Google sign-up + new-shop onboarding (web PR #39), free platform with immediate `ACTIVE` shop (PR #40)	**On** `main`
Phase B (`feature/daksh-phase-b-smart-print-printers`)	**Not on** `main`**.** The branch has **zero commits** beyond `main` at the time of writing, so nothing Phase-B-specific exists to document.
Android app source (`feature/daksh-android-companion`, `phase3/android*`, 21 commits ahead, `apps/android`)	**Not on** `main`
Other unmerged branches (`development`, `feature/daksh-firebase-profile-api`, `feature/daksh-release-coordinator`, `feature/resend-email-system`)	**No route exists on any of them that is absent from** `main` (checked by extracting the registered routes of each branch and diffing: `main` is a superset).

How this was verified

1. **Routes (two independent checks):** (a) a static scan of every `*.ts` under `apps/api/src` for `app|scope.<verb>('<path>')` registrations (multi-line aware) found **113**; (b) the real application was instantiated from the TypeScript source (`createApp` in `src/app.ts` via `tsx`, dummy config, **no database or network**) and every route was collected through a Fastify `onRoute` hook: **111** routes under `/api/v1` plus `/health` and `/ready` (registered directly on the root) = **113**, with identical method+path sets. The documentation table, the API inventory, the entity map and the OpenAPI spec are all generated from **one** route table, and generation aborts if that table's method+path set differs from the scan (113 = 113, no missing, no extra, no duplicates). The throwaway scripts are not committed (this change is documentation only). Note: the checked-in `apps/api/dist` build is **stale** (110 routes, built before Phase A) and was deliberately not used.
2. **Schemas and errors:** transcribed from the route handlers, their zod schemas and the domain functions they call (request bodies, query strings, response objects, `AppError` codes). The spec was **not** inferred from tests or older docs.
3. **Prisma:** the ER diagrams were written from `schema.prisma`: 32 `model` blocks, 14 `enum` blocks, all relations, uniques and `onDelete` rules. 8 models were confirmed unused by searching the application source for any Prisma access to them.
4. **Mermaid:** every ```` ```mermaid ```` block was parsed and rendered with `mermaid@11.4.1` in headless Chromium (the same pipeline produced the PDF); the check fails on any syntax error.
5. **OpenAPI:** validated with `@apidevtools/swagger-parser` (OpenAPI 3.0.3: 100 paths, 113 operations, 21 component schemas, all `$ref`s resolved).
6. **Secrets:** no tokens, keys, passwords or production values appear in these files; examples use placeholders. Only `deploy/env.production.example` *key names* were consulted, never values.

Findings: missing, ambiguous or stale contracts

#	Finding	Impact / recommendation
1	**Docs vs code on new-shop status.** `docs/MOBILE_AUTH_API.md`, `docs/API_CONTRACT.md` and `docs/FIREBASE_PHONE_LOGIN.md` say self-onboarded shops are `PILOT`, `acceptsOrders=false`, awaiting admin activation. The code (since PR #40) creates them `ACTIVE`**,** `acceptsOrders=true`**, no subscription**.	Android should trust the response (`shopStatus`, `nextStep`), not those docs. Recommend fixing the old docs (not done here: docs-only scope of *new* files).
2	**Production feature flags unknown.** `GOOGLE_LOGIN_ENABLED`, `SHOP_SELF_ONBOARDING_ENABLED`, `FCM_PUSH_ENABLED` default to `false` in code; production values, `FIREBASE_PROJECT_ID`, `STORAGE_DRIVER`, `RATE_LIMIT_*` overrides and token TTL overrides are not in the repo.	**Requires confirmation** with whoever operates the server. The app must handle `503` `GOOGLE_LOGIN_UNAVAILABLE`, `403` `SELF_ONBOARDING_DISABLED` and absent push. Read TTLs from `expiresIn` / `refreshTokenExpiresIn`.

#	Finding	Impact / recommendation
3	**Lost refresh response is unrecoverable.** Refresh rotates the token server-side before the reply arrives; a lost reply leaves the app holding a spent token (reuse → family revoked).	Documented in the guide: generous timeout, treat as signed-out. A server-side idempotent refresh would be a backend change (out of scope).
4	**Firebase token freshness.** `POST /auth` needs a Firebase ID token whose `auth_time` is ≤ 300 s old and strictly newer than the last accepted one. `getIdToken(true)` does not help.	The app must perform a real Google sign-in right before each exchange (documented).
5	Response shapes not pinned by a schema in code: `GET /shop/analytics/revenue` (`totals`, `daily`, `breakdown`, `comparison`), `GET /shop/hours` (`availability`), `GET /public/shops/:slug`, all admin responses.	Marked `any`/partial in the spec and **Requires confirmation** before hard-typing.
6	Whether the owner app should also pair as a **device** (`/device/*`) is a product decision. Owner-session Bearer is the documented path; device mode exists for a companion Android device.	**Requires confirmation.**
7	Several updates use `PUT` with **partial** bodies (`/shop/settings`, `/shop/hours`, `/shop/pricing-rules/:id`) and have no version/ETag (last write wins).	Documented; do not send the whole object expecting replace semantics.
8	`POST /public/shops/:slug/orders` returns **200** (not 201) both for a new order and for an idempotent replay.	Do not use the status code to tell them apart.
9	Realtime is best-effort: SSE bus is in memory on one API process; worker events never reach it. Customers have no push.	Use FCM + `GET /shop/notifications?after=` + polling as recovery.
10	CORS allows only `content-type`, `x-csrf-token`, `last-event-id` request headers (browser-only concern; native apps ignore CORS).	Irrelevant to OkHttp; matters only if a WebView calls the API.
11	`docs/openapi/current.yaml` is a path-only subset (86 paths in the older file) with no schemas.	Superseded for integration by `docs/openapi/api-v1.yaml`; the old file was left untouched.
12	An existing owner who never linked Google and signs in with Google gets the **restricted** pair even with a matching email; onboarding then fails with `409 EMAIL_UNAVAILABLE`.	Documented flow: sign in by the existing method, then link Google.
13	`ShopStatus` values `PILOT`, `PAST_DUE`, `CANCELLED` are never set by application code (an admin can set them); `Membership` is unused for authorization.	Treat unknown/other statuses as "not live" (`isActive=false`).

Conventions

- **Status labels** on each endpoint: *Verified from code* (all 112 active routes), *Deprecated* (1), *Not implemented* (listed in the inventory's own section, **not** as routes), *Requires confirmation* (called out inline where production config or an inner shape decides the behaviour).
- Paths are shown with the `/api/v1` prefix except `/health` and `/ready`.
- Never commit production secrets into these documents.

Android integration guide (Kotlin)

Audience: the Android developer integrating the **shop-owner app** with the existing Printout Buddy backend. Every endpoint, field and error code below is verified against the route code on `origin/main` (`080c6bd`); details per endpoint are in api-inventory.md and the machine-readable OpenAPI spec. Flows are in sequence-diagrams.md.

Ground rules (read first)

1. **Mobile uses its own token contract, not cookies.** Send `Authorization: Bearer <accessToken>` on every owner call. There is **no CSRF header** and no cookie handling for Bearer requests.
2. **The Firebase ID token is used exactly once**, at `POST /auth`, to prove identity. It is **never** accepted by `/shop/*`, `/users/*` or `/shops/*`.
3. **Two secrets, rotating:** a short-lived opaque *access token* and a single-use *refresh token*. Refresh is single-flight; if a refresh result is lost the user must sign in again (see §6).
4. **Request bodies are strict.** Unknown keys → `400 VALIDATION_ERROR`. Omit optional fields instead of sending `null` (configure `explicitNulls = false`).
5. **Money is integer paise.** Dates are ISO-8601 UTC. Business time is `Asia/Kolkata`.
6. **Admin accounts cannot sign in on mobile** (`403 FORBIDDEN`). The app is for shop owners.
7. **Feature flags are server-side and default OFF in code:** `GOOGLE_LOGIN_ENABLED`, `SHOP_SELF_ONBOARDING_ENABLED`, `FCM_PUSH_ENABLED`. Their production values are **Requires confirmation** (production env is not in the repository). The app must tolerate `503 GOOGLE_LOGIN_UNAVAILABLE`, `403 SELF_ONBOARDING_DISABLED`, and missing push.

1. Production base URL

```
object Api {
    const val BASE_URL = "https://printoutbuddy.tech/api/v1/"   // trailing slash for Retrofit; paths below are relative
    const val ROOT = "https://printoutbuddy.tech"                // /health lives here, not under /api/v1
}
```

Responses are `{ "data": ... }` on success and `{ "error": { code, message, requestId, details? } }` on failure. `204` has no body.

Common models and JSON config

```
val json = Json {
    ignoreUnknownKeys = true   // the server adds fields over time (additive compatibility)
    explicitNulls = false      // omit nulls: request bodies are strict
    encodeDefaults = false
}

@Serializable data class Envelope<T>(val data: T)

@Serializable data class ApiErrorBody(val error: ApiErrorDto)
@Serializable data class ApiErrorDto(
    val code: String, val message: String, val requestId: String? = null, val details: JsonElement? = null
)

class ApiException(
    val status: Int, val code: String, override val message: String,
    val requestId: String?, val details: JsonElement?, val retryAfterSeconds: Long? = null
) : Exception(message)

@Serializable data class UserProfile(
    val id: String?, val role: String?, val name: String?, val email: String?, val phoneNumber: String?,
```

```
    val loginMethods: List<String>, val emailVerified: Boolean, val phoneVerified: Boolean,
    val createdAt: String?, val updatedAt: String?, val lastLoginAt: String?
)
@Serializable data class RateInfo(val paperSize: String, val colourMode: String, val sides: String, val
pricePerSheetPaise: Int)
@Serializable data class ShopProfile(
    val id: String, val shopName: String, val slug: String, val address: String?, val pincode: String?,
    val phoneNumber: String?, val shopStatus: String, val isActive: Boolean, val acceptsOrders: Boolean,
    val showQr: Boolean, val qrCodeUrl: String?, val rateInfo: List<RateInfo>, val onboardingStatus: String
)
@Serializable data class MobileAuth(
    val accessToken: String, val refreshToken: String, val tokenType: String,
    val expiresIn: Long, val refreshTokenExpiresIn: Long,
    val user: UserProfile, val shops: List<ShopProfile>, val shop: ShopProfile?,
    val nextStep: String, val isShopRegisteredAtLeastOne: Boolean, val showQr: Boolean, val onboardingAvailable: Boolean
)
```

HTTP client skeleton (OkHttp + Retrofit)

```
// build.gradle: okhttp, okhttp-sse, retrofit, retrofit2-kotlinx-serialization-converter, kotlinx-serialization-json
class AuthHeaderInterceptor(private val tokens: TokenStore) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val req = chain.request()
        val t = runBlocking { tokens.read() }?.access
        return chain.proceed(if (t != null && req.header("Authorization") == null)
            req.newBuilder().header("Authorization", "Bearer $t").build() else req)
    }
}

fun okHttp(tokens: TokenStore, installationId: String): OkHttpClient {
    val refreshClient = OkHttpClient.Builder().callTimeout(30, TimeUnit.SECONDS).build()   // no authenticator on this
one
    return OkHttpClient.Builder()
        .addInterceptor(AuthHeaderInterceptor(tokens))
        .authenticator(TokenAuthenticator(tokens, installationId, refreshClient))
        .connectTimeout(10, TimeUnit.SECONDS).readTimeout(30, TimeUnit.SECONDS)
        .build()                                                  // add a logging interceptor ONLY in debug and redact
Authorization
}

val retrofit: Retrofit = Retrofit.Builder().baseUrl(Api.BASE_URL).client(okHttp(tokens, installation.id()))
    .addConverterFactory(json.asConverterFactory("application/json".toMediaType())).build()

/** Maps any non-2xx into ApiException so callers branch on `code`, never on message text. */
suspend fun <T> safe(block: suspend () -> T): T = try { block() } catch (e: HttpException) {
    val body = e.response()?.errorBody()?.string()
    val err = runCatching { json.decodeFromString<ApiErrorBody>(body!!).error }.getOrNull()
    throw ApiException(e.code(), err?.code ?: "HTTP_${e.code()}", err?.message ?: e.message(), err?.requestId,
err?.details,
                       e.response()?.headers()?.get("Retry-After")?.toLongOrNull())
}

interface AuthApi {
    @POST("auth")             suspend fun signIn(@Body b: AuthRequest): Envelope<MobileAuth>
    @POST("auth/refresh")     suspend fun refresh(@Body b: RefreshRequest): Envelope<MobileAuth>
    @POST("auth/logout")      suspend fun logout(@Body b: LogoutRequest): Response<Unit>        // 204
```

```
    @POST("auth/logout-all")  suspend fun logoutAll(@Body b: LogoutAllRequest): Response<Unit>  // 204
    @POST("shops/onboarding") suspend fun onboard(@Body b: OnboardingRequest): Envelope<MobileAuth>
    @PUT("users/me/fcm-token") suspend fun putFcm(@Body b: FcmReg): Envelope<JsonElement>
    @POST("users/me/identities/google") suspend fun linkGoogle(@Body b: LinkGoogle): Envelope<JsonElement>
}
```

2. Firebase Google sign-in → token exchange

The token must be fresh. The server rejects a Firebase ID token whose `auth_time` (the instant the user *actually signed in with Google*) is older than `FIREBASE_AUTH_MAX_AGE_SECONDS` (300 s), and each token can be exchanged **once** (a later `auth_time` is required every time). `getIdToken(true)` refreshes the token but does **not** change `auth_time`. So: **do a real Google sign-in immediately before every** `POST /auth`, send the token right away, then discard the Firebase session.

```
// Credential Manager + Firebase Auth. webClientId = the OAuth "Web client" id of the Firebase project.
suspend fun googleIdToken(activity: Activity, webClientId: String): String {
    val request = GetCredentialRequest.Builder()
        .addCredentialOption(GetGoogleIdOption.Builder()
            .setServerClientId(webClientId)
            .setFilterByAuthorizedAccounts(false)
            .build())
        .build()
    val cred = CredentialManager.create(activity).getCredential(activity, request).credential
    val googleToken = GoogleIdTokenCredential.createFrom(cred.data).idToken
    val auth = FirebaseAuth.getInstance()
    val user = auth.signInWithCredential(GoogleAuthProvider.getCredential(googleToken, null)).await().user!!
    return user.getIdToken(false).await().token!!      // fresh: auth_time == now
}
```

```
@Serializable data class AuthRequest(
    val loginMethod: String,                // "google" | "otp" | "email"
    val firebaseIdToken: String? = null,    // required for google / otp
    val email: String? = null, val password: String? = null,   // loginMethod = "email" only
    val fcmToken: String? = null,           // optional; registered only after auth succeeds
    val installationId: String,             // 16-64 chars [A-Za-z0-9_-], random, stable per install
    val platform: String = "android"
)

suspend fun signInWithGoogle(activity: Activity): MobileAuth {
    val idToken = googleIdToken(activity, BuildConfig.GOOGLE_WEB_CLIENT_ID)
    val fcm = runCatching { FirebaseMessaging.getInstance().token.await() }.getOrNull()
    val result = safe {
        authApi.signIn(AuthRequest("google", firebaseIdToken = idToken, fcmToken = fcm, installationId =
installation.id()))
    }.data
    FirebaseAuth.getInstance().signOut()          // the Firebase session is not needed afterwards
    tokens.save(result)                           // §5
    return result
}
```

Installation id: generate once (`SecureRandom`, 24 bytes → Base64 URL-safe, no padding → 32 chars), persist, and reuse for sign-in, refresh (`deviceId`), onboarding (`deviceId`) and FCM (`installationId`). It binds the refresh token to this install: a refresh from another id is treated as theft.

`loginMethod` alternatives (same endpoint, same response): `"otp"` (Firebase phone ID token + optional `phoneNumber` that must equal the verified number) and `"email"` (`email` + `password`). Email sign-in works only for shop owners.

Handling the response: `nextStep`

`MobileAuth.nextStep` (also returned by `GET /users/me`) drives navigation:

nextStep	Meaning	App action
`create_shop`	verified identity, no account (restricted token, `user.id == null`)	onboarding screen → `POST /shops/onboarding`
`complete_profile`	shop exists but owner name or address is empty	profile form → `PATCH /users/me`, `PATCH /shops/me`
`generate_qr`	shop exists but is not live (`shopStatus != ACTIVE`, or bad slug)	QR screen using `GET /shops/me/qr` (shows `reason`)
`dashboard`	ready	main app
`account_blocked`	shop `SUSPENDED`	blocked screen. Sign-in itself returns `403 ACCOUNT_BLOCKED` with `details.nextStep = "account_blocked"` and **no tokens**.

3. First-time Google user onboarding

An unlinked Google identity receives the **restricted pair**: `accessToken` (scoped grant) + `refreshToken` (kind `onboarding`), `user.id = null`, `user.email` = the Google-verified email, `nextStep = "create_shop"`. It is valid only for `POST /shops/onboarding`, `GET /users/me` and `POST /auth/logout`, and only for the access-token lifetime (refresh it like a normal token).

```
@Serializable data class OnboardingRequest(
    val ownerName: String,            // 1-100
    val shopName: String,             // 2-100
    val address: String,              // 5-300
    val pincode: String,              // ^[1-9][0-9]{5}$
    val client: String = "mobile",
    val deviceId: String,             // = installationId
    val email: String? = null         // Google: omit (or equal to the verified email). Phone principals: required.
)

suspend fun createShop(req: OnboardingRequest): MobileAuth {
    val auth = safe { authApi.onboard(req) }.data                    // Bearer = restricted token (sent by the
interceptor)
    tokens.save(auth)            // REPLACE the stored pair with the account's own pair
    return auth                  // 201 created, or 200 when the identical request is repeated
}
```

Errors: `403 SELF_ONBOARDING_DISABLED` (registration not open: show "contact support"), `409 ALREADY_ONBOARDED` (go to sign-in), `409 EMAIL_UNAVAILABLE` (that Google email already belongs to an account: sign in with the existing method, then link Google in Settings), `400 VALIDATION_ERROR` (show `details.fieldErrors`), `401` (grant expired: sign in with Google again).

Result: user + shop + Google identity exist; the shop is created `ACTIVE`, `acceptsOrders = true` (code since PR #40; some older docs still say `PILOT`, which is stale). The new owner has **no password** and **no pricing rules**: next screen should prompt them to set prices (§8), otherwise customers get `422 NO_PRICING_RULE`.

4. Existing shop-owner sign-in

Same endpoint, `loginMethod: "google"`. If the Google identity is **already linked** to an owner account the response is the full pair with `nextStep = "dashboard"` (or `generate_qr` / `complete_profile`).

Matching is **only** by the Firebase uid of a *linked* identity. An owner who has never linked Google, even with the same email, gets the restricted pair, not a session. They sign in with their existing method (`"email"` / `"otp"`) and then link Google:

```
@Serializable data class Reauth(val currentPassword: String? = null, val firebaseIdToken: String? = null)
@Serializable data class LinkGoogle(val firebaseIdToken: String, val reauth: Reauth)

// freshGoogleToken = googleIdToken(...) obtained NOW; reauth = exactly ONE of password or another linked identity's
fresh token
suspend fun linkGoogle(freshGoogleToken: String, password: String) =
    safe { authApi.linkGoogle(LinkGoogle(freshGoogleToken, Reauth(currentPassword = password))) }
```

Linking ends **all other sessions and refresh families** of the user (the current one stays). `GET /users/me/identities` returns masked link state; `POST /users/me/identities/google/unlink` refuses the last sign-in method (`409 LAST_SIGNIN_METHOD`).

5. Secure credential storage

Store: `accessToken`, `refreshToken`, absolute `accessExpiresAt` (= `now + expiresIn`), `refreshExpiresAt`, `installationId`. Never log them; never put them in `SharedPreferences` plain text, intents, crash reports or backups.

```
class TokenStore(context: Context) {
    private val aead: Aead = AndroidKeysetManager.Builder()
        .withSharedPref(context, "pb_keyset", "pb_keyset_prefs")
        .withKeyTemplate(KeyTemplates.get("AES256_GCM"))
        .withMasterKeyUri("android-keystore://pb_master_key")   // key material never leaves the Keystore
        .build().keysetHandle.getPrimitive(Aead::class.java)
    private val ds = context.createDataStore("pb_tokens")       // or any file you encrypt with `aead`

    data class Tokens(val access: String, val refresh: String, val accessExpiresAtMs: Long, val refreshExpiresAtMs:
Long)
    suspend fun save(a: MobileAuth) = write(Tokens(
        a.accessToken, a.refreshToken,
        System.currentTimeMillis() + a.expiresIn * 1000, System.currentTimeMillis() + a.refreshTokenExpiresIn * 1000))
    suspend fun read(): Tokens? = /* decrypt with aead, associated data = "pb_tokens" */ TODO()
    suspend fun clear() { /* delete file */ }
    // write()/read(): aead.encrypt(plain, "pb_tokens".toByteArray()) / decrypt(...)
}
```

Add `android:allowBackup="false"` (or exclude the token file in `dataExtractionRules`/`fullBackupContent`). Treat decrypt failure (e.g. restored-from-backup device) as "signed out".

6. Refresh-token rotation, logout and session expiry

Defaults (read the real values from `expiresIn` / `refreshTokenExpiresIn`): access **15 min**, refresh **30 days**, both opaque random strings (not JWTs: do not parse them).

```
@Serializable data class RefreshRequest(val refreshToken: String, val deviceId: String)

class TokenAuthenticator(
    private val tokens: TokenStore, private val installationId: String, private val refreshClient: OkHttpClient
) : Authenticator {
    private val lock = Any()
    override fun authenticate(route: Route?, response: Response): Request? {
        val path = response.request.url.encodedPath
        // never "refresh" a failed sign-in / refresh / logout call, and never loop
        if (path.endsWith("/auth") || path.endsWith("/auth/refresh") || path.endsWith("/auth/logout") ||
response.request.header("X-Retry") != null) return null
        val sent = response.request.header("Authorization")?.removePrefix("Bearer ")
        synchronized(lock) {                                    // SINGLE-FLIGHT: one refresh at a time
```

```
            val current = runBlocking { tokens.read() } ?: return null
            if (current.access != sent) return retry(response, current.access)   // someone already refreshed
            val body = json.encodeToString(RefreshRequest(current.refresh, installationId))
            val r = refreshClient.newCall(Request.Builder().url(Api.BASE_URL + "auth/refresh")
                .post(body.toRequestBody("application/json".toMediaType())).build()).execute()
            r.use {
                when {
                    it.isSuccessful -> {
                        val auth = json.decodeFromString<Envelope<MobileAuth>>(it.body!!.string()).data
                        runBlocking { tokens.save(auth) }         // persist BEFORE retrying
                        return retry(response, auth.accessToken)
                    }
                    it.code == 409 -> {                           // REFRESH_CONFLICT: another refresh just won
                        Thread.sleep(800)
                        val latest = runBlocking { tokens.read() }
                        return if (latest != null && latest.access != sent) retry(response, latest.access) else null
                    }
                    it.code == 401 || it.code == 403 -> { runBlocking { tokens.clear() }; SessionEvents.signedOut();
return null }
                    else -> return null                           // 429 / 5xx: keep tokens, surface the error
                }
            }
        }
    }
    private fun retry(r: Response, access: String) =
        r.request.newBuilder().header("Authorization", "Bearer $access").header("X-Retry", "1").build()
}
```

Rules:

- **Refresh before expiry is optional**; refreshing on `401` via the `Authenticator` is enough. If you refresh proactively, do it through the same single-flight path.
- **Never run two refreshes concurrently and never reuse an old refresh token.** A presented token that was already rotated (after a 5 s grace) revokes the whole family.
- **Lost response:** if the app sent `POST /auth/refresh` but never received the reply, the server may have rotated the token and the app cannot learn the new pair. A later retry with the old token fails and revokes the family → the only recovery is a fresh sign-in. Use a generous timeout (≈30 s) and handle "unrecoverable refresh" as signed-out.
- Outcomes: `401 REFRESH_TOKEN_INVALID|EXPIRED` → sign in again; `403 ACCOUNT_BLOCKED` → blocked screen; `409 REFRESH_CONFLICT` → re-read storage and retry once; `429` → back off.

Logout (idempotent, works even if the access token has expired):

```
@Serializable data class LogoutRequest(val refreshToken: String? = null)
@Serializable data class LogoutAllRequest(val keepPush: Boolean = false)

suspend fun logout() {
    val t = tokens.read()
    runCatching {   // best effort: always clear local state
        safe { authApi.logout(LogoutRequest(t?.refresh)) }          // Bearer is added when a token is stored
    }
    tokens.clear()
    FirebaseAuth.getInstance().signOut()
    // push registration of this installation is revoked server-side by /auth/logout
}
```

`POST /auth/logout-all` (body `{ "keepPush": false }`) signs out **every** device. After a password/identity change on another device, your tokens may be revoked: the next call returns `401` then refresh fails → signed out.

Session expiry / blocked: a suspended shop loses all sessions immediately (`401`, then refresh → `403 ACCOUNT_BLOCKED`). Show a blocked screen from `nextStep = "account_blocked"`; do not loop.

7. Loading shop data

Call `GET /users/me` after sign-in/refresh/app start, then `GET /shops/me`:

```
interface PrintoutApi {
    @GET("users/me")  suspend fun me(): Envelope<ProfileView>
    @GET("shops/me")  suspend fun shop(): Envelope<ShopsView>
    @GET("shops/me/qr") suspend fun qr(): Envelope<QrInfo>
}
@Serializable data class ProfileView(val user: UserProfile, val shops: List<ShopProfile>, val shop: ShopProfile?, val
nextStep: String, val showQr: Boolean)
@Serializable data class ShopsView(val shops: List<ShopProfile>, val shop: ShopProfile?, val nextStep: String, val
showQr: Boolean)
@Serializable data class QrInfo(val ready: Boolean, val publicUrl: String?, val qrCodeUrl: String?, val reason: String?)
```

`shops` has **zero or one** element (single-shop model; `shop == shops[0]`). Render the QR in-app from `qrCodeUrl` (`https://printoutbuddy.tech/p/<slug>`); only show it when `ready`.

Other settings: `GET/PUT /shop/settings` (name, address, public contact, brand colour, `acceptsOrders` toggle), `GET/PUT /shop/hours` plus temporary closure and emergency reopen endpoints (`weeklyHours` is `{ mon..sun: [{start:"HH:MM", end:"HH:MM|24:00"}] }`, `PUT` is partial).

8. Pricing rules: fetch and edit

Up to four rules exist per shop (A4 only): colour mode `bw|colour` × sides `single|duplex`. `pricePerSheetPaise` is an integer 1…1,000,000.

```
@Serializable data class PricingRule(
    val id: String, val paperSize: String, val colourMode: String, val sides: String,
    val pricePerSheetPaise: Int, val active: Boolean, val createdAt: String, val updatedAt: String
)
@Serializable data class CreateRule(val colourMode: String, val sides: String, val pricePerSheetPaise: Int, val active:
Boolean = true)
@Serializable data class UpdateRule(val pricePerSheetPaise: Int? = null, val active: Boolean? = null,
                                    val colourMode: String? = null, val sides: String? = null)

interface PricingApi {
    @GET("shop/pricing-rules")           suspend fun list(): Envelope<List<PricingRule>>
    @POST("shop/pricing-rules")          suspend fun create(@Body b: CreateRule): Envelope<PricingRule>      // 201
    @PUT("shop/pricing-rules/{id}")      suspend fun update(@Path("id") id: String, @Body b: UpdateRule):
Envelope<PricingRule>
    @DELETE("shop/pricing-rules/{id}")   suspend fun delete(@Path("id") id: String): Response<Unit>            // 204
}

suspend fun savePrice(existing: PricingRule?, colour: String, sides: String, paise: Int): PricingRule =
    if (existing != null) pricing.update(existing.id, UpdateRule(pricePerSheetPaise = paise)).data
    else try { pricing.create(CreateRule(colour, sides, paise)).data }
         catch (e: ApiException) {
             if (e.code != "DUPLICATE_PRICING_RULE") throw e
             val dup = pricing.list().data.first { it.colourMode == colour && it.sides == sides }   // someone created
it meanwhile
```

```
            pricing.update(dup.id, UpdateRule(pricePerSheetPaise = paise)).data
        }
```

- No version/ETag: **last write wins**. Refetch the list after each change.
- Existing orders keep their price snapshot; **new** quotes/orders use the current active rules immediately.
- Quote price = `sheets × pricePerSheetPaise`, `sheets = (sides == duplex ? ceil(pages/2) : pages) × copies`. The app should preview with that formula but never treat it as authoritative.

9. Orders and the live queue

```
@Serializable data class OrderItem(
    val id: String, val orderNumber: String, val status: String, val totalPaise: Int, val currency: String,
    val createdAt: String, val updatedAt: String, val customerDisplayNameOrReference: String?,
    val originalFilename: String, val pageCount: Int?, val selectedPageCount: Int?, val colourMode: String?,
    val sides: String?, val copies: Int?, val documentStatus: String, val deleteAfter: String?, val printInitiatedAt:
String?
)
@Serializable data class OrderPage(val items: List<OrderItem>, val nextCursor: String? = null)
@Serializable data class Counts(val pending: Int, val attention: Int)

interface OrdersApi {
    @GET("shop/orders") suspend fun list(
        @Query("active") active: String? = "1",   // hides COLLECTED / CANCELLED / EXPIRED
        @Query("print") print: String? = null,    // pending | initiated | attention
        @Query("sort") sort: String = "newest",
        @Query("cursor") cursor: String? = null, @Query("limit") limit: Int = 50): Envelope<OrderPage>
    @GET("shop/orders/counts") suspend fun counts(): Envelope<Counts>
    @GET("shop/orders/{id}")   suspend fun detail(@Path("id") id: String): Envelope<JsonElement>
}
```

- **Pagination:** pass back `nextCursor` verbatim (opaque). Absent `nextCursor` = last page. Do not mix cursors across different `sort`/filters.
- **Live queue strategy:** (1) on foreground: `GET /shop/orders?active=1` + `GET /shop/orders/counts`; (2) foreground real-time: SSE (§11); (3) background: FCM → on push, refetch; (4) reconnect/recovery: `GET /shop/notifications?after=<lastSeq>`. Poll `counts` every 30–60 s only as a fallback (limit 600/min per session is generous).
- **Print (the one shop action):** `POST /shop/orders/{id}/print-now` with a fresh UUID `clientRequestId`. The first success starts a 30-minute deletion countdown (`document.deleteAfter`), moves the order `NEW→ACCEPTED→PRINTING` and returns `access.url` (≤5 min). Open it with a PDF viewer / `PrintManager`. Repeats with the same or a new id are safe: they only re-issue access (`firstPrint: false`).
- **Reprint/view later:** `POST /shop/orders/{id}/document-access` (inline) or `.../document-download` (attachment). `410 DOCUMENT_UNAVAILABLE` means the file is gone (expired or past `deleteAfter`).
- **Cancel:** `POST /shop/orders/{id}/transitions { "toStatus":"CANCELLED", "clientRequestId": "<uuid>", "reason"?: "..." }`. It is the **only** status a shop can request. Do not call the retired `print-confirmation` (always `410`).
- **Order status in the new flow** ends at `PRINTING` after Print; `PRINTED/READY/COLLECTED` appear only on legacy orders. "Print initiated" ≠ paper printed.

```
@Serializable data class ClientRequest(val clientRequestId: String)
@Serializable data class PrintAccess(val url: String, val expiresAt: String, val contentDisposition: String, val
mimeType: String)
@Serializable data class PrintResult(val firstPrint: Boolean, val transitioned: Boolean, val access: PrintAccess)
// @POST("shop/orders/{id}/print-now") suspend fun printNow(@Path("id") id: String, @Body b: ClientRequest):
Envelope<PrintResult>
```

```
suspend fun printNow(orderId: String, requestId: String = UUID.randomUUID().toString()): PrintResult =
    orders.printNow(orderId, ClientRequest(requestId)).data       // retry with the SAME requestId
```

10. Customer tracking (customer flow only)

No authentication; the tracking token in the URL is the capability. An Android customer client (or a deep link from the web QR) uses:

```
@GET("public/orders/{token}") suspend fun track(@Path("token") token: String): Envelope<Tracking>

@Serializable data class Tracking(
    val serverTime: String, val orderNumber: String, val shopName: String, val status: String, val totalPaise: Int,
    val printIssue: Boolean, val timeline: List<TimelineEntry>, val document: TrackedDoc
)
@Serializable data class TimelineEntry(val status: String, val at: String)
@Serializable data class TrackedDoc(val fileName: String?, val pageCount: Int?, val status: String, val deleteAfter:
String?, val deletedAt: String?)
```

Poll every 5–15 s while the screen is visible and stop on terminal statuses; honour `429`. Treat `404 NOT_FOUND` as "unknown/expired link". Key fields: `status`, `timeline[]`, `printIssue` (true when the latest print job failed/needs attention), `document.deleteAfter/deletedAt`, `pickup`, `serverTime` (use it to compute the countdown, not the device clock). **Never log the token.** The upload flow is `GET /public/shops/{slug}` → `POST .../uploads/initiate` → `PUT uploadUrl` (resolve it against the base host if relative; raw bytes, header `Content-Type` must equal the declared MIME) → `POST .../uploads/{id}/complete` → `POST .../quotes` → `POST .../orders` (sequence D); use a fresh `clientRequestId` UUID per order and reuse it on retry.

11. Notifications: FCM, inbox and SSE

FCM registration: register after sign-in and **whenever the token changes**. The `installationId` must equal the one used at sign-in.

```
@Serializable data class FcmReg(val installationId: String, val token: String, val platform: String = "android")

class PbMessagingService : FirebaseMessagingService() {
    override fun onNewToken(token: String) {
        // Only if signed in; otherwise the token is sent with POST /auth next time (fcmToken)
        scope.launch { runCatching { safe { authApi.putFcm(FcmReg(installation.id(), token)) } } }
    }
    override fun onMessageReceived(m: RemoteMessage) {
        // data = { type: "order.new", orderId: "<id>" }; the notification title is generic ("New print order").
        // There is NO file name or customer data in the push: fetch details with your own token.
        if (m.data["type"] == "order.new") refreshQueueAndInbox()
    }
}
// On sign-out the server revokes this installation's registration. Explicit opt-out:
// DELETE users/me/fcm-token with body {"installationId": "<id>"}  (exactly one of installationId | token)
```

With Retrofit a DELETE with a body needs `@HTTP(method = "DELETE", path = "users/me/fcm-token", hasBody = true)`. Push is best effort and may be disabled server-side; the inbox below is the recovery path.

Inbox (durable):

```
@Serializable data class Notif(val id: String, val seq: String, val type: String, val createdAt: String, val readAt:
String?, val data: NotifData)
@Serializable data class NotifData(val orderId: String, val orderNumber: String)
```

```
@Serializable data class NotifPage(val items: List<Notif>, val nextCursor: String?, val hasMore: Boolean, val
latestCursor: String?, val unreadCount: Int)

// GET shop/notifications?after=<lastSeq>&limit=50   catch-up (newest first; seq is a decimal STRING, compare as
Long/BigInteger)
// GET shop/notifications?cursor=<oldestSeq>         page backwards
// POST shop/notifications/{id}/read, POST shop/notifications/read-all
```

Persist `lastSeenSeq` locally; on launch, reconnect, and every push, call `?after=lastSeenSeq` and merge by `id`.

SSE (optional, foreground only): `GET /shop/events`, `Authorization: Bearer`, header `Last-Event-ID` for replay. Events: `order.created`, `order.statusChanged`, `order.updated`, `printJob.updated`, `document.deletionScheduled`, `notification.created`; comment `: ping` every ≤25 s. Limits: 10 streams per shop, 5 per session, 30 connects/min (`429` with `Retry-After`). The bus is in memory on one API process and the retention worker cannot publish to it, so **never rely on SSE alone**.

```
val es = EventSources.createFactory(sseClient /* readTimeout = 0 */).newEventSource(
    Request.Builder().url(Api.BASE_URL + "shop/events").header("Authorization", "Bearer $access")
        .header("Accept", "text/event-stream").build(),
    object : EventSourceListener() {
        override fun onEvent(es: EventSource, id: String?, type: String?, data: String) { /* type = event name, data =
JSON */ }
        override fun onFailure(es: EventSource, t: Throwable?, r: Response?) { if (r?.code == 401)
refreshThenReconnect() else reconnectWithBackoff() }
    })
```

Close the stream in `onStop`; an access-token refresh requires reconnecting with the new token (the server also re-validates the session every heartbeat and closes the stream when it ends).

12. Error handling and retries

```
suspend inline fun <reified T> Call.await(): T { /* execute; map non-2xx via ApiErrorBody -> ApiException(status, code,
message, requestId, details, Retry-After) */ }
```

Status / code	**Meaning**	**App behaviour**
400 `VALIDATION_ERROR`	bad/unknown field; `details.fieldErrors`	show field errors; do not retry
401 `UNAUTHORIZED`	token missing/expired/revoked	refresh once (§6), retry; on failure → sign-in
401 `INVALID_CREDENTIALS`, `GOOGLE_TOKEN_INVALID`, `PHONE_TOKEN_INVALID`	sign-in failed	for Google: sign in with Google **again** (fresh token); do not auto-retry the same token
401 `REFRESH_TOKEN_INVALID` / `REFRESH_TOKEN_EXPIRED`	refresh failed	clear tokens → sign-in screen
403 `FORBIDDEN`	wrong role (e.g. admin on mobile)	show message
403 `ACCOUNT_BLOCKED` / `SHOP_SUSPENDED`	suspended	blocked screen
403 `SELF_ONBOARDING_DISABLED`	registration closed	"contact support"
403 `CSRF_INVALID`	you sent cookies	you should never see it with Bearer
404 `NOT_FOUND` / `ORDER_NOT_FOUND` / `PRINT_JOB_NOT_FOUND`	missing or another tenant's	remove from UI, refresh list
409 `REFRESH_CONFLICT`	concurrent refresh	wait ~1 s, re-read tokens
409 `DUPLICATE_PRICING_RULE`	rule exists	GET and PUT instead

Status / code	Meaning	App behaviour
409 `INVALID_STATUS_TRANSITION` / `ACTIVE_PRINT_JOB_EXISTS` / `DOCUMENT_UNAVAILABLE`	state changed	refetch the order, show current state
409 `EMAIL_UNAVAILABLE`, `ALREADY_ONBOARDED`, `IDEMPOTENCY_CONFLICT`	conflict	per §3 / use a new `clientRequestId` only for a *different* intent
410 `DOCUMENT_UNAVAILABLE` / `ENDPOINT_RETIRED`	file deleted / retired API	show "file removed"; remove the call
422 `NO_PRICING_RULE`, `INVALID_PAGE_RANGE`, `QUOTE_EXPIRED` …	business rule	show message
429 `RATE_LIMITED`	throttled	honour `Retry-After`; exponential backoff with jitter
5xx / `INTERNAL_ERROR` / 503 `*_UNAVAILABLE`	server/outage or feature off	backoff; keep UI usable

Retry policy: retry **GET** and naturally idempotent writes (`PUT`, `DELETE`) on network errors, `502/503/504`, `429`: 3 attempts, 0.5 s → 1 s → 2 s with ±20% jitter, cap 10 s, respecting `Retry-After`. Retry **POST** only when it carries an idempotency key (`clientRequestId` on `print-now`, `transitions`, `print-jobs`, `retry`, `reconcile`, public `orders`) and reuse the **same** key. Do **not** blindly retry `POST /auth`, `POST /auth/refresh` (see §6), `POST /shop/devices/pairing-codes` or `POST /shop/hours/reopen` (not idempotent). Always include `error.requestId` in bug reports.

13. Devices and printers (where appropriate)

The Windows agent prints; the **owner app only manages**. Useful owner endpoints (Bearer): `GET /shop/devices` (list with `presence`, `printers`), `POST /shop/devices/pairing-codes` (code `PB-XXXX-XXXX`, 10 min, single use; the owner types it into the Windows agent), `PATCH /shop/devices/{id}` (rename), `POST /shop/devices/{id}/revoke`, `PATCH /shop/devices/{id}/printers/{printerId}` (`{ authorized, isDefault }`), `POST /shop/orders/{id}/print-jobs` (queue to a Windows printer), `GET /shop/print-jobs`, and `cancel` / `retry` / `reconcile`.

- A print job targets an **authorized, present** printer; if none is default pass `printerId`. Possible conflicts: `PRINTER_NOT_AUTHORIZED`, `PRINTER_UNAVAILABLE`, `PRINTER_UNSUPPORTED_OPTIONS`, `ACTIVE_PRINT_JOB_EXISTS`.
- **No "printed" state exists.** `PRINT_INITIATED` = the spooler accepted it. `NEEDS_RECONCILIATION` requires a human decision via `reconcile` (`PRINTED` / `NOT_PRINTED_REQUEUE` / `NOT_PRINTED_GIVE_UP`); `retry` after a possibly-submitted job needs `acknowledgeDuplicateRisk: true` (show a "check the printer first" confirmation).
- **Device mode (`/device/*`) is a different identity** (`pbd_…` credential from `POST /device/pair`, `platform: "ANDROID"`), intended for a dedicated companion device, not for the owner app. Android device accounts can use `/device/orders*`, `/device/heartbeat`, `/device/push-token`, `/device/credential/rotate`; the printer/print-job device routes are **Windows-only** (`409 CONFLICT`). Whether the Android app should act as a paired device at all is **Requires confirmation** (the owner-session model in this guide is the supported default).

14. Offline and reconnection behaviour

- **Cache read models** (orders page, pricing rules, profile) in Room for display; mark them stale. Never cache access/refresh tokens outside the encrypted store, and never cache signed document URLs (≤5 min lifetime).
- **Mutations while offline:** keep a small outbox of *idempotent* actions (`print-now`, cancel, pricing edit) with their `clientRequestId`; replay on reconnect with the same ids. Expect `409 INVALID_STATUS_TRANSITION` / `DOCUMENT_UNAVAILABLE` on replay because the 30-minute / 24-hour deadlines keep running: surface it, do not retry forever.
- **On reconnect (and on every foreground):** (1) ensure a valid token (refresh if `accessExpiresAt` is near), (2) `GET /shop/notifications?after=lastSeq`, (3) `GET /shop/orders?active=1` and `GET /shop/orders/counts`, (4) reopen SSE, (5) re-register FCM if the token changed.
- **Server time:** use `serverTime` from `/public/orders/:token`, `/shop/hours` (`availability.serverTime`) and `/device/heartbeat` for countdowns; do not trust the device clock for `deleteAfter`.
- Connectivity probe: `GET https://printoutbuddy.tech/health` (`{"data":{"status":"ok"}}`). Do not use `/ready` from the app.

15. Implementation checklist

- [ ] OkHttp + kotlinx.serialization (`ignoreUnknownKeys`, `explicitNulls=false`); TLS only; no cookie jar for owner calls.
- [ ] Installation id generated once; used for sign-in, refresh (`deviceId`), onboarding, FCM.
- [ ] Fresh Google sign-in before every `POST /auth`; discard the Firebase session after the exchange.
- [ ] Tokens in Keystore-backed AEAD storage; backups excluded; never logged.
- [ ] Single-flight `Authenticator`; refresh result persisted before retrying; lost-refresh → sign-in again.
- [ ] `nextStep`-driven navigation; blocked/closed-registration/feature-off screens.
- [ ] UUID `clientRequestId` for every print/cancel/order mutation, reused on retry.
- [ ] Cursor pagination with opaque cursors; inbox catch-up by `seq`.
- [ ] FCM `onNewToken` → `PUT /users/me/fcm-token`; push handler only triggers a refetch.
- [ ] No calls to retired/admin/Windows-only endpoints; no billing UI.
- [ ] Support: log `requestId` from error bodies (never tokens, tracking tokens, FCM tokens or signed URLs).

16. Example requests and responses

All values are **fake placeholders** (ids, tokens, numbers). Shapes are those produced by the code. `<ACCESS>` / `<REFRESH>` stand for opaque 43-character tokens; never log real ones. Success bodies are wrapped in `{ "data": ... }`.

16.1 `POST /auth` (Google): existing owner

```
POST /api/v1/auth HTTP/1.1
Host: printoutbuddy.tech
Content-Type: application/json

{
  "loginMethod": "google",
  "firebaseIdToken": "<FIREBASE_ID_TOKEN>",
  "fcmToken": "<FCM_TOKEN>",
  "installationId": "Hq3mV9xT2bLwK8dN5rYzA1cE7uJ4oPfG",
  "platform": "android"
}
```

```
{
  "data": {
    "accessToken": "<ACCESS>",
    "refreshToken": "<REFRESH>",
    "tokenType": "Bearer",
    "expiresIn": 900,
    "refreshTokenExpiresIn": 2592000,
    "isShopRegisteredAtLeastOne": true,
    "showQr": true,
    "nextStep": "dashboard",
    "user": {
      "id": "cmaexampleuser0001", "role": "shop", "name": "Anita Sharma", "email": "anita@example.com",
      "phoneNumber": null, "loginMethods": ["google"], "emailVerified": true, "phoneVerified": false,
      "createdAt": "2026-10-20T09:15:00.000Z", "updatedAt": "2026-10-20T09:15:00.000Z", "lastLoginAt": "2026-10-
27T05:30:12.000Z"
    },
    "shops": [
      {
        "id": "cmaexampleshop0001", "ownerId": "cmaexampleuser0001", "shopName": "Anita Xerox & Print",
        "phoneNumber": null, "address": "12 MG Road, Pune", "pincode": "411001", "location": null,
```

```
        "status": "standard", "plan": null, "isVerified": null, "isActive": true, "showQr": true,
        "qrCodeUrl": "https://printoutbuddy.tech/p/anita-xerox-print-7k2a",
        "rateInfo": [ { "paperSize": "A4", "colourMode": "bw", "sides": "single", "pricePerSheetPaise": 200 } ],
        "services": { "colourModes": ["bw"], "sides": ["single"] }, "payment": null, "rating": null,
        "createdAt": "2026-10-20T09:15:00.000Z", "updatedAt": "2026-10-26T11:02:00.000Z",
        "slug": "anita-xerox-print-7k2a", "shopStatus": "ACTIVE", "acceptsOrders": true, "isDefault": true,
        "registeredBy": "self", "images": null, "referral": null, "onboardingStatus": "dashboard"
      }
    ],
    "shop": { "id": "cmaexampleshop0001", "note": "identical to shops[0]; abbreviated in this example only" },
    "onboardingAvailable": true
  }
}
```


(`shop` is the same object as `shops[0]`; the abbreviation above is only to keep the example short, the real response repeats the full object.)

16.2 `POST /auth` (Google): no account yet, restricted pair


```
{
  "data": {
    "accessToken": "<RESTRICTED_ACCESS>", "refreshToken": "<RESTRICTED_REFRESH>", "tokenType": "Bearer",
    "expiresIn": 900, "refreshTokenExpiresIn": 2592000,
    "isShopRegisteredAtLeastOne": false, "showQr": false, "nextStep": "create_shop",
    "user": { "id": null, "role": null, "name": null, "email": "newowner@example.com", "phoneNumber": null,
              "loginMethods": ["google"], "emailVerified": true, "phoneVerified": false,
              "createdAt": null, "updatedAt": null, "lastLoginAt": null },
    "shops": [], "shop": null, "onboardingAvailable": true
  }
}
```


16.3 `POST /shops/onboarding`


```
{
  "ownerName": "Ravi Kumar", "shopName": "Ravi Print Point", "address": "Shop 4, Station Road, Nashik",
  "pincode": "422001", "client": "mobile", "deviceId": "Hq3mV9xT2bLwK8dN5rYzA1cE7uJ4oPfG"
}
```


Response `201`: same body as 16.1 with the new owner (`loginMethods: ["google"]`, `registeredBy: "self"`, `rateInfo: []`, `shopStatus: "ACTIVE"`, `nextStep: "dashboard"`). Validation error example:


```
{ "error": { "code": "VALIDATION_ERROR", "message": "Request validation failed", "requestId": "0f1e2d3c-0000-4000-8000-
000000000000",
  "details": { "formErrors": [], "fieldErrors": { "pincode": ["Enter a valid 6-digit PIN code"] } } } }
```


16.4 `POST /auth/refresh`


```
{ "refreshToken": "<REFRESH>", "deviceId": "Hq3mV9xT2bLwK8dN5rYzA1cE7uJ4oPfG" }
```


Response `200`: same shape as 16.1 (a new `accessToken` **and** a new `refreshToken`; replace both). Failures:


```
{ "error": { "code": "REFRESH_TOKEN_INVALID", "message": "Your sign-in could not be renewed. Sign in again.",
"requestId": "..." } }
```

```
{ "error": { "code": "ACCOUNT_BLOCKED", "message": "This account is blocked. Contact Printout Buddy support.",
"requestId": "...", "details": { "nextStep": "account_blocked" } } }
```

16.5 Pricing rules

`GET /api/v1/shop/pricing-rules` returns `200`:

```
{ "data": [
  { "id": "cmarule000000001", "paperSize": "A4", "colourMode": "bw", "sides": "single", "pricePerSheetPaise": 200,
"active": true,
    "createdAt": "2026-10-20T09:20:00.000Z", "updatedAt": "2026-10-20T09:20:00.000Z" },
  { "id": "cmarule000000002", "paperSize": "A4", "colourMode": "colour", "sides": "duplex", "pricePerSheetPaise": 1000,
"active": true,
    "createdAt": "2026-10-20T09:21:00.000Z", "updatedAt": "2026-10-22T08:00:00.000Z" }
] }
```

`POST /api/v1/shop/pricing-rules` with body `{ "colourMode": "bw", "sides": "duplex", "pricePerSheetPaise": 150 }` returns `201 { "data": { "id": "...", "paperSize": "A4", ... } }`; a repeat of an existing combination returns `409`:

```
{ "error": { "code": "DUPLICATE_PRICING_RULE", "message": "A pricing rule for this colour mode and sides already
exists", "requestId": "..." } }
```

`PUT /api/v1/shop/pricing-rules/cmarule000000001` with `{ "pricePerSheetPaise": 250 }` returns `200` with the updated rule. `DELETE` returns `204`.

16.6 Live queue

`GET /api/v1/shop/orders?active=1&print=pending&limit=20` returns `200`:

```
{ "data": {
  "items": [
    { "id": "cmaorder00000001", "orderNumber": "ANITAX-0042", "status": "NEW", "totalPaise": 1200, "currency": "INR",
      "createdAt": "2026-10-27T05:41:10.000Z", "updatedAt": "2026-10-27T05:41:10.000Z",
      "customerDisplayNameOrReference": "Ravi", "originalFilename": "notes.pdf", "pageCount": 12, "selectedPageCount":
6,
      "colourMode": "bw", "sides": "duplex", "copies": 2, "documentStatus": "AVAILABLE", "deleteAfter": null,
      "printInitiatedAt": null, "paperSize": "A4", "pageSelection": { "mode": "ranges", "ranges": [ { "from": 1, "to": 6
} ] } }
  ],
  "nextCursor": "WyIyMDI2LTEwLTI3VDA1OjQxOjEwLjAwMFoiLCJjbWFvcmRlcjAwMDAwMDAxIl0"
} }
```

For the next page repeat the call with `&cursor=<nextCursor>`. `GET /api/v1/shop/orders/counts` returns `{ "data": { "pending": 3, "attention": 0 } }`.

16.7 Print and cancel

`POST /api/v1/shop/orders/cmaorder00000001/print-now`:

```
{ "clientRequestId": "6f1c2a7e-3b64-4c0e-9a55-1d2f8e7b9c10" }
```

Response `200`:

```
{ "data": {
  "order": { "id": "cmaorder00000001", "orderNumber": "ANITAX-0042", "status": "PRINTING" },
```

```
  "transitioned": true, "firstPrint": true,
  "document": { "status": "PRINTED_RETENTION", "printInitiatedAt": "2026-10-27T05:44:00.000Z", "deleteAfter": "2026-10-27T06:14:00.000Z" },
  "access": { "url": "https://<object-store-or-api-host>/<signed-path>?<signature>", "expiresAt": "2026-10-27T05:49:00.000Z",
              "contentDisposition": "inline", "mimeType": "application/pdf" }
} }
```

`POST /api/v1/shop/orders/cmaorder00000001/transitions` with `{ "toStatus": "CANCELLED", "reason": "Customer asked", "clientRequestId": "<uuid>" }` returns `200 { "data": { "order": { "id": "...", "orderNumber": "...", "status": "CANCELLED", "totalPaise": 1200, "currency": "INR", "createdAt": "...", "updatedAt": "..." } } }`. A conflict looks like:

```
{ "error": { "code": "INVALID_STATUS_TRANSITION", "message": "Invalid order transition: PRINTING -> CANCELLED", "requestId": "...", "details": { "from": "PRINTING", "to": "CANCELLED" } } }
```

16.8 Notifications and push

`PUT /api/v1/users/me/fcm-token` with `{ "installationId": "Hq3mV9xT2bLwK8dN5rYzA1cE7uJ4oPfG", "token": "<FCM_TOKEN>", "platform": "android" }` returns `200 { "data": { "registered": true, "installationId": "Hq3m...", "platform": "android" } }`.

`GET /api/v1/shop/notifications?after=120&limit=50` returns `200`:

```
{ "data": {
  "items": [ { "id": "cmanotif0000001", "seq": "123", "type": "order.created", "createdAt": "2026-10-27T05:41:10.000Z", "readAt": null,
               "data": { "orderId": "cmaorder00000001", "orderNumber": "ANITAX-0042" } } ],
  "nextCursor": null, "hasMore": false, "latestCursor": "123", "unreadCount": 1
} }
```

The FCM data message the app receives is `{ "type": "order.new", "orderId": "cmaorder00000001" }` with a generic notification title; nothing else is sent.

16.9 Errors you must handle everywhere

```
{ "error": { "code": "UNAUTHORIZED", "message": "Authentication required", "requestId": "..." } }
```

```
{ "error": { "code": "RATE_LIMITED", "message": "Too many requests. Please slow down and retry shortly.", "requestId": "..." } }
```

(`RATE_LIMITED` also carries a `Retry-After` header in seconds.)

```
{ "error": { "code": "SHOP_SUSPENDED", "message": "This shop is suspended", "requestId": "..." } }
```

16.10 Customer flow

```
POST /api/v1/public/shops/anita-xerox-print-7k2a/uploads/initiate
Content-Type: application/json

{ "fileName": "notes.pdf", "byteSize": 482113, "declaredMimeType": "application/pdf" }
```

```
{ "data": { "uploadId": "dGhpc0lzQW5FeGFtcGxlSWQ", "uploadUrl": "https://printoutbuddy.tech/api/v1/public/uploads/dGhpc0lzQW5FeGFtcGxlSWQ/content?token=<UPLOAD_TOKEN>",
```

```
  "requiredHeaders": { "content-type": "application/pdf" }, "expiresAt": "2026-10-27T05:56:00.000Z",
  "limits": { "acceptedMimeTypes": ["application/pdf", "image/jpeg", "image/png"], "maxBytes": 52428800, "maxPdfPages":
200, "imagePageCount": 1 } } }
```


`uploadUrl` is absolute when the server has `PUBLIC_API_BASE` configured and a **relative path (`/api/v1/public/uploads/...`) when it does not**: always resolve it against `https://printoutbuddy.tech` before use (**Requires confirmation** which form production returns). `PUT <uploadUrl>` with the raw bytes and `Content-Type: application/pdf` returns `{ "data": { "uploadId": "...", "byteSize": 482113 } }`. Then `POST .../uploads/<uploadId>/complete` (no body) returns `{ "data": { "documentId": "...", "detectedMimeType": "application/pdf", "byteSize": 482113, "pageCount": 12, "documentStatus": "AVAILABLE", "uploadedAt": "...", "expiresAt": "..." } }`.


```
POST /api/v1/public/shops/anita-xerox-print-7k2a/quotes
Content-Type: application/json

{ "documentId": "dGhpc0lzQW5FeGFtcGxlSWQ",
  "printOptions": { "paperSize": "A4", "colourMode": "bw", "sides": "duplex", "copies": 2, "pageSelection": { "mode":
"ranges", "ranges": [ { "from": 1, "to": 6 } ] } } }
```



```
{ "data": { "quoteId": "<SIGNED_QUOTE>",
  "printOptions": { "paperSize": "A4", "colourMode": "bw", "sides": "duplex", "copies": 2, "pageSelection": { "mode":
"ranges", "ranges": [ { "from": 1, "to": 6 } ] } },
  "selectedPageCount": 6, "sheetsPerCopy": 3, "totalSheets": 6, "unitPricePaise": 200, "totalPaise": 1200, "currency":
"INR", "expiresAt": "2026-10-27T05:55:00.000Z" } }
```



```
POST /api/v1/public/shops/anita-xerox-print-7k2a/orders
Content-Type: application/json

{ "quoteId": "<SIGNED_QUOTE>", "customerDisplayNameOrReference": "Ravi", "clientRequestId": "0d9a6f52-7a0e-4f4f-8f8b-
3c2d1e0b9a11" }
```



```
{ "data": { "orderNumber": "ANITAX-0042", "trackingToken": "<TRACKING_TOKEN>", "status": "NEW", "totalPaise": 1200,
"currency": "INR" } }
```


`GET /api/v1/public/orders/<TRACKING_TOKEN>` returns `200`:

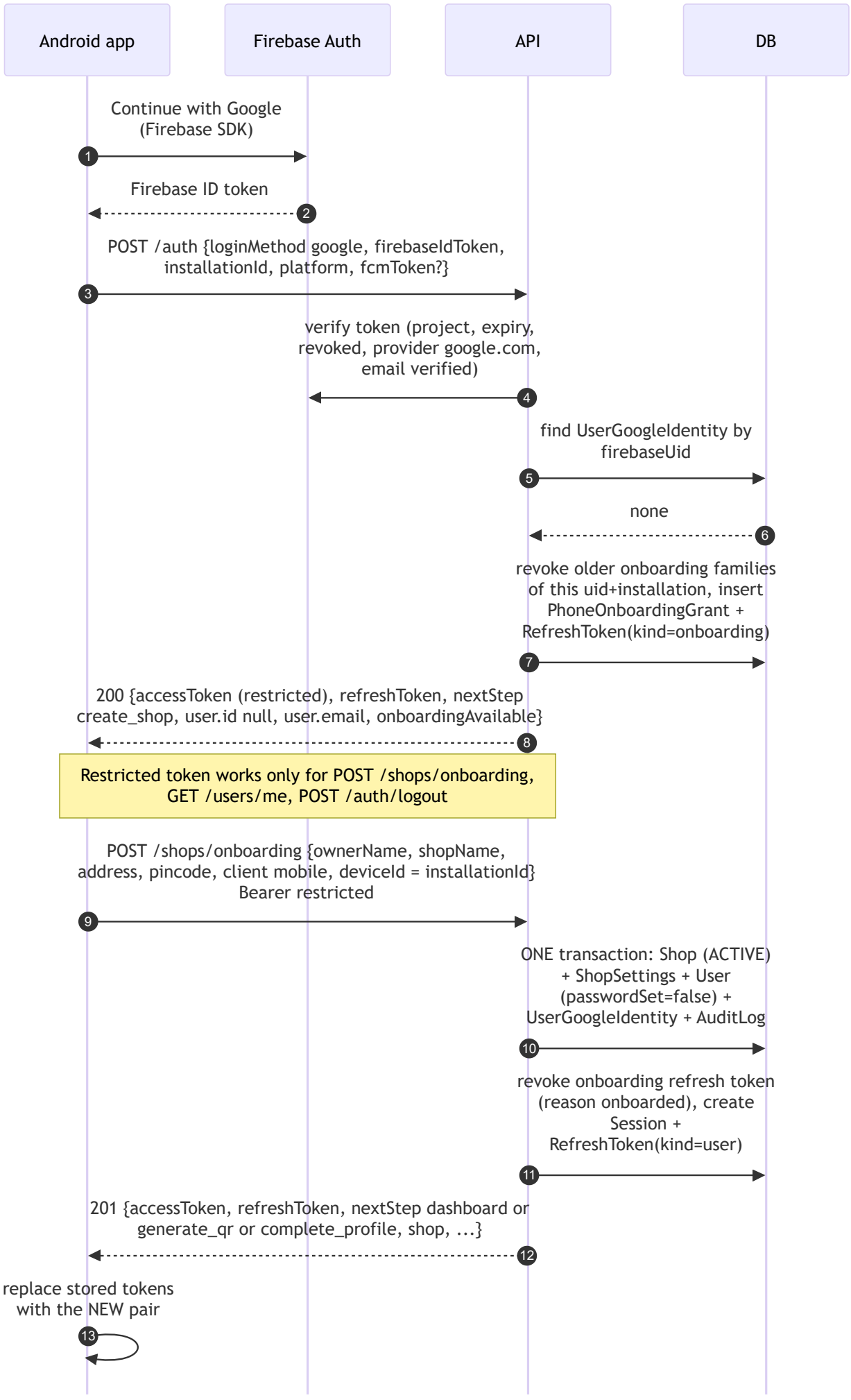

```
{ "data": {
  "serverTime": "2026-10-27T05:46:00.000Z", "retentionMinutes": 30, "shopSlug": "anita-xerox-print-7k2a",
  "orderNumber": "ANITAX-0042", "shopName": "Anita Xerox & Print",
  "pickup": { "address": "12 MG Road, Pune", "contact": null }, "printIssue": false,
  "status": "PRINTING", "totalPaise": 1200, "currency": "INR",
  "createdAt": "2026-10-27T05:41:10.000Z", "updatedAt": "2026-10-27T05:44:00.000Z",
  "printOptions": { "paperSize": "A4", "colourMode": "bw", "sides": "duplex", "copies": 2, "pageSelection": { "mode":
"ranges", "ranges": [ { "from": 1, "to": 6 } ] } },
  "selectedPageCount": 6,
  "document": { "fileName": "notes.pdf", "pageCount": 12, "status": "PRINTED_RETENTION", "deleteAfter": "2026-10-
27T06:14:00.000Z", "deletedAt": null },
  "documentDeleteAfter": "2026-10-27T06:14:00.000Z",
  "timeline": [ { "status": "NEW", "at": "2026-10-27T05:41:10.000Z" }, { "status": "ACCEPTED", "at": "2026-10-
27T05:44:00.000Z" }, { "status": "PRINTING", "at": "2026-10-27T05:44:00.000Z" } ]
} }
```

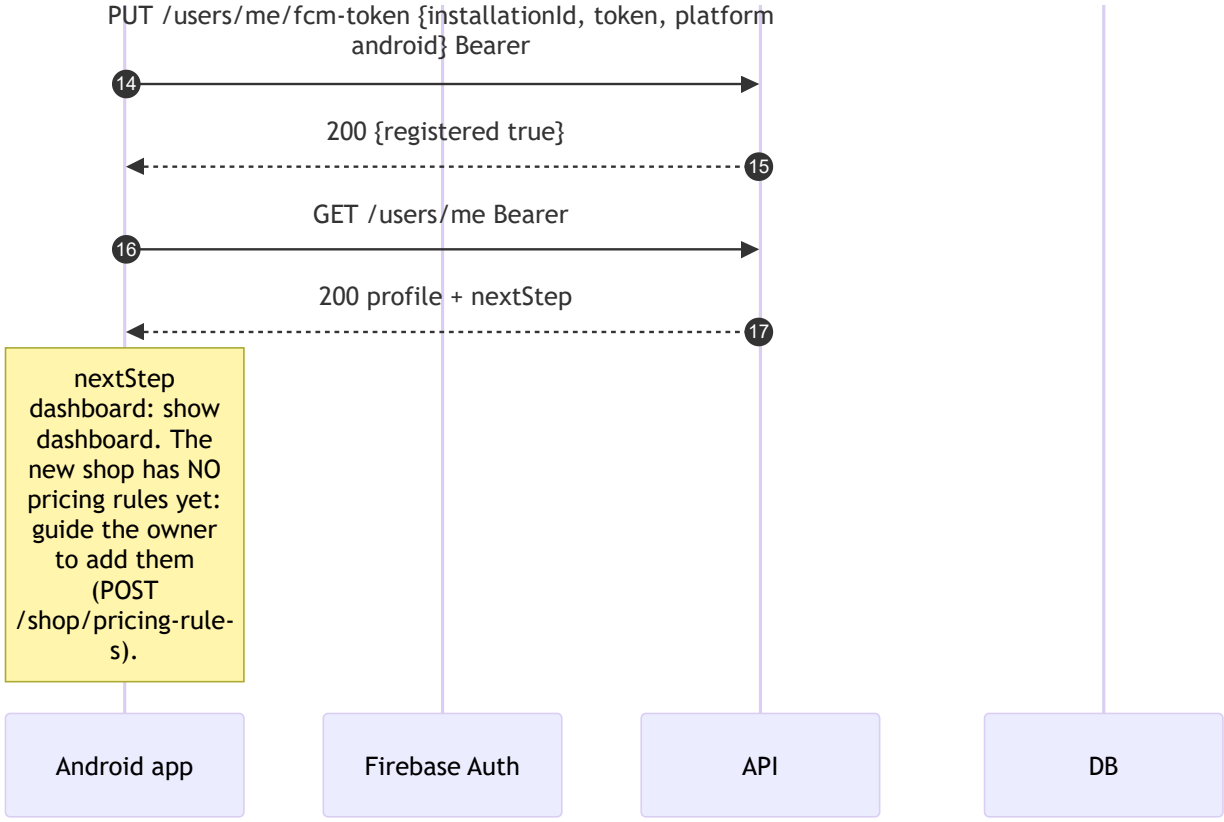
Sequence diagrams

All flows are transcribed from the route code on `origin/main`. `API` = Fastify (`/api/v1`), `DB` = PostgreSQL, `FB` = Firebase Auth, `Store` = private object storage. Android examples use the **mobile bearer contract**: no cookies, no CSRF header.

A. New Google user → create shop → dashboard

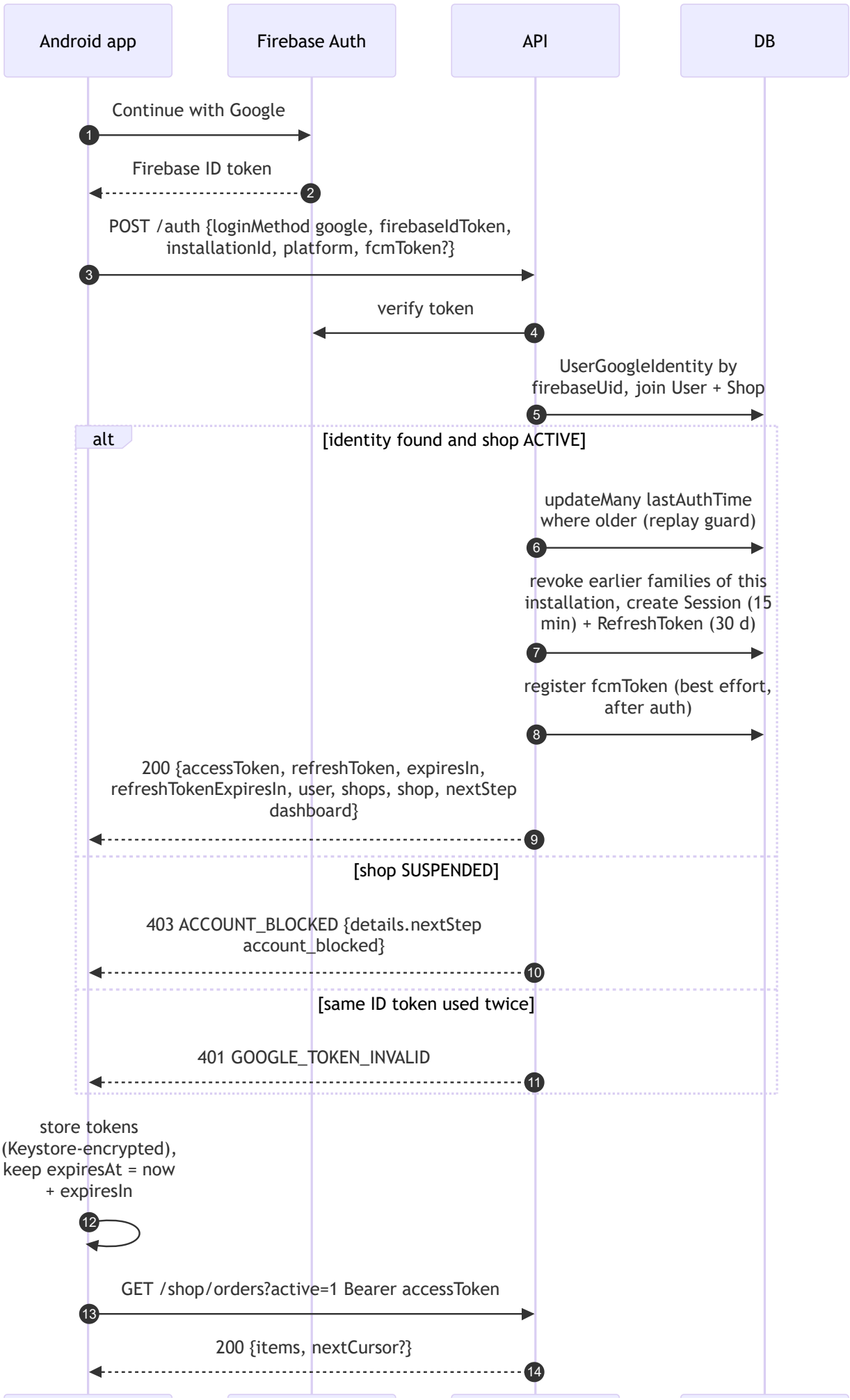
First-time user: Google proves the identity, but no account exists, so the API issues a **restricted** token pair that can only create the shop.

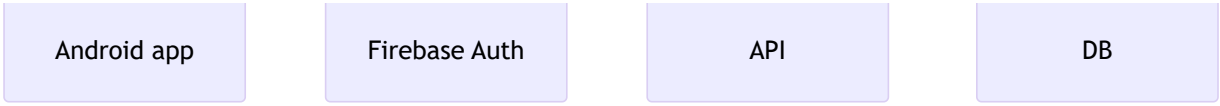

Android app
Firebase Auth
API
DB
Continue with Google (Firebase SDK)
1
Firebase ID token
2
POST /auth {loginMethod google, firebaseIdToken, installationId, platform, fcmToken?}
3
verify token (project, expiry, revoked, provider google.com, email verified)
4
find UserGoogleIdentity by firebaseUid
5
none
6
revoke older onboarding families of this uid+installation, insert PhoneOnboardingGrant + RefreshToken(kind=onboarding)
7
200 {accessToken (restricted), refreshToken, nextStep create_shop, user.id null, user.email, onboardingAvailable}
8
Restricted token works only for POST /shops/onboarding, GET /users/me, POST /auth/logout
POST /shops/onboarding {ownerName, shopName, address, pincode, client mobile, deviceId = installationId}
Bearer restricted
9
ONE transaction: Shop (ACTIVE) + ShopSettings + User (passwordSet=false) + UserGoogleIdentity + AuditLog
10
revoke onboarding refresh token (reason onboarded), create Session + RefreshToken(kind=user)
11
201 {accessToken, refreshToken, nextStep dashboard or generate_qr or complete_profile, shop, ...}
12
replace stored tokens with the NEW pair
13



Failure branches: `403 SELF_ONBOARDING_DISABLED` (feature flag off; the app should show "registration not open"), `409 ALREADY_ONBOARDED`, `409 EMAIL_UNAVAILABLE` (the Google email already belongs to another account: sign in with the existing method and *link* Google from Settings), `429`. A retry of the same onboarding within 2 minutes returns the same shop with `200`.

B. Existing Google user → authenticate → dashboard

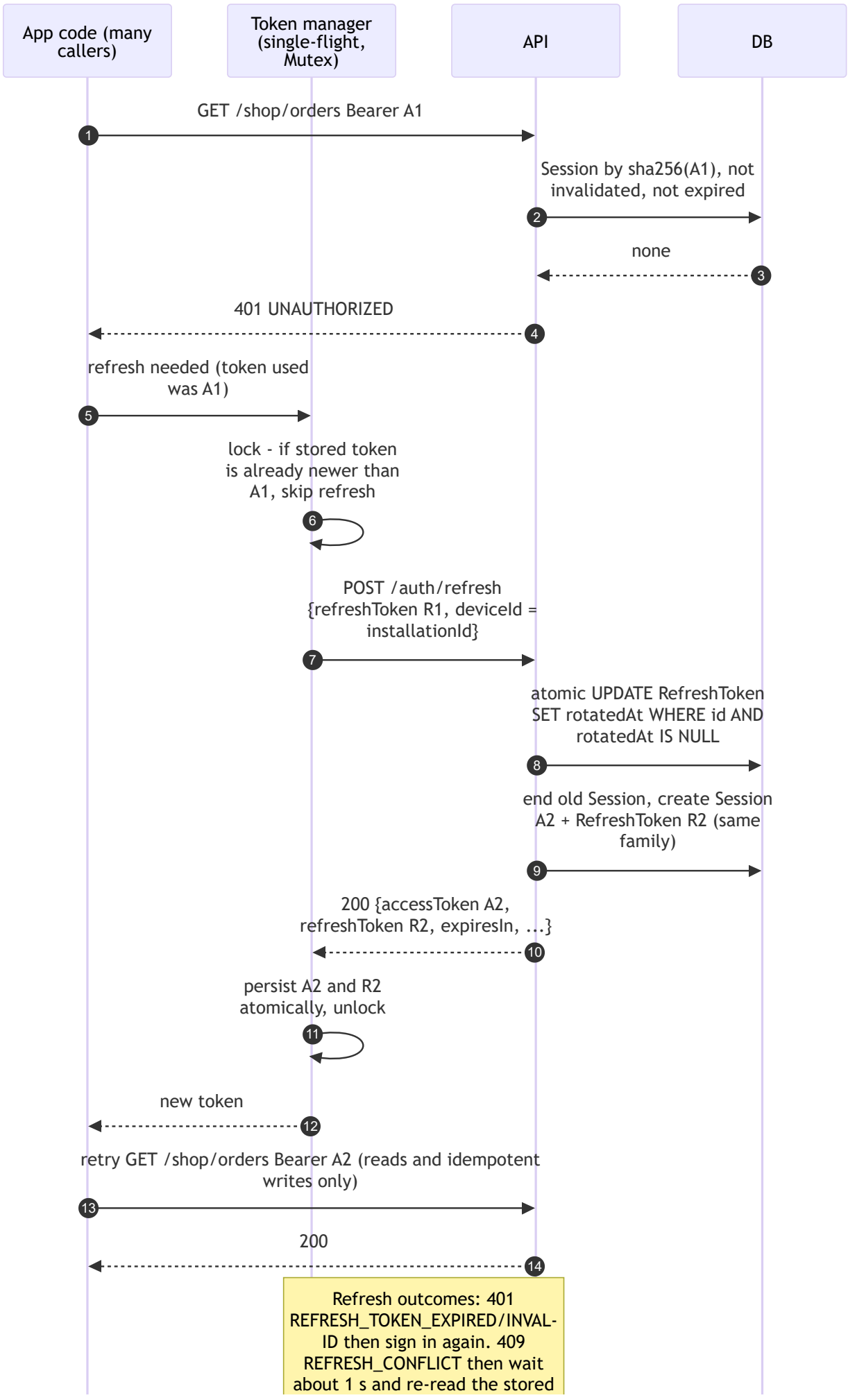

Android app
Firebase Auth
API
DB
Continue with Google
1
Firebase ID token
2
POST /auth {loginMethod google, firebaseIdToken, installationId, platform, fcmToken?}
3
verify token
4
UserGoogleIdentity by firebaseUid, join User + Shop
5
alt
[identity found and shop ACTIVE]
updateMany lastAuthTime where older (replay guard)
6
revoke earlier families of this installation, create Session (15 min) + RefreshToken (30 d)
7
register fcmToken (best effort, after auth)
8
200 {accessToken, refreshToken, expiresIn, refreshTokenExpiresIn, user, shops, shop, nextStep dashboard}
9
[shop SUSPENDED]
403 ACCOUNT_BLOCKED {details.nextStep account_blocked}
10
[same ID token used twice]
401 GOOGLE_TOKEN_INVALID
11
store tokens (Keystore-encrypted), keep expiresAt = now + expiresIn
12
GET /shop/orders?active=1 Bearer accessToken
13
200 {items, nextCursor?}
14

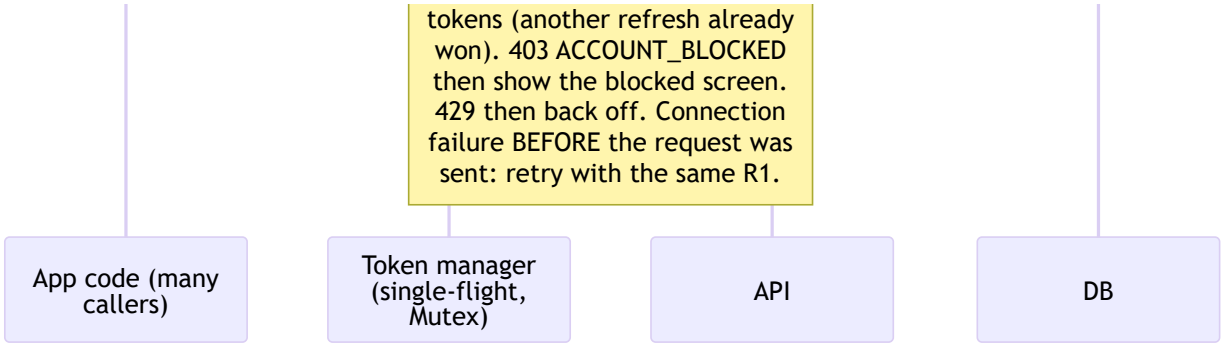


An existing owner whose Google account has **never been linked** gets the restricted pair (as in flow A), *not* a session, even if the Google email equals their account email. They must sign in with their existing method (`loginMethod: "email"` or `"otp"`) and then link Google (`POST /users/me/identities/google`).

C. Access token expires → refresh → retry safely

Refresh tokens are **single-use** and rotate; two concurrent refreshes would revoke the family if mishandled, so refresh is **single-flight**.


App code (many callers)
Token manager (single-flight, Mutex)
API
DB
GET /shop/orders Bearer A1
1
Session by sha256(A1), not invalidated, not expired
2
none
3
401 UNAUTHORIZED
4
refresh needed (token used was A1)
5
lock - if stored token is already newer than A1, skip refresh
6
POST /auth/refresh {refreshToken R1, deviceId = installationId}
7
atomic UPDATE RefreshToken SET rotatedAt WHERE id AND rotatedAt IS NULL
8
end old Session, create Session A2 + RefreshToken R2 (same family)
9
200 {accessToken A2, refreshToken R2, expiresIn, ...}
10
persist A2 and R2 atomically, unlock
11
new token
12
retry GET /shop/orders Bearer A2 (reads and idempotent writes only)
13
200
14
Refresh outcomes: 401 REFRESH_TOKEN_EXPIRED/INVAL-ID then sign in again. 409 REFRESH_CONFLICT then wait about 1 s and re-read the stored

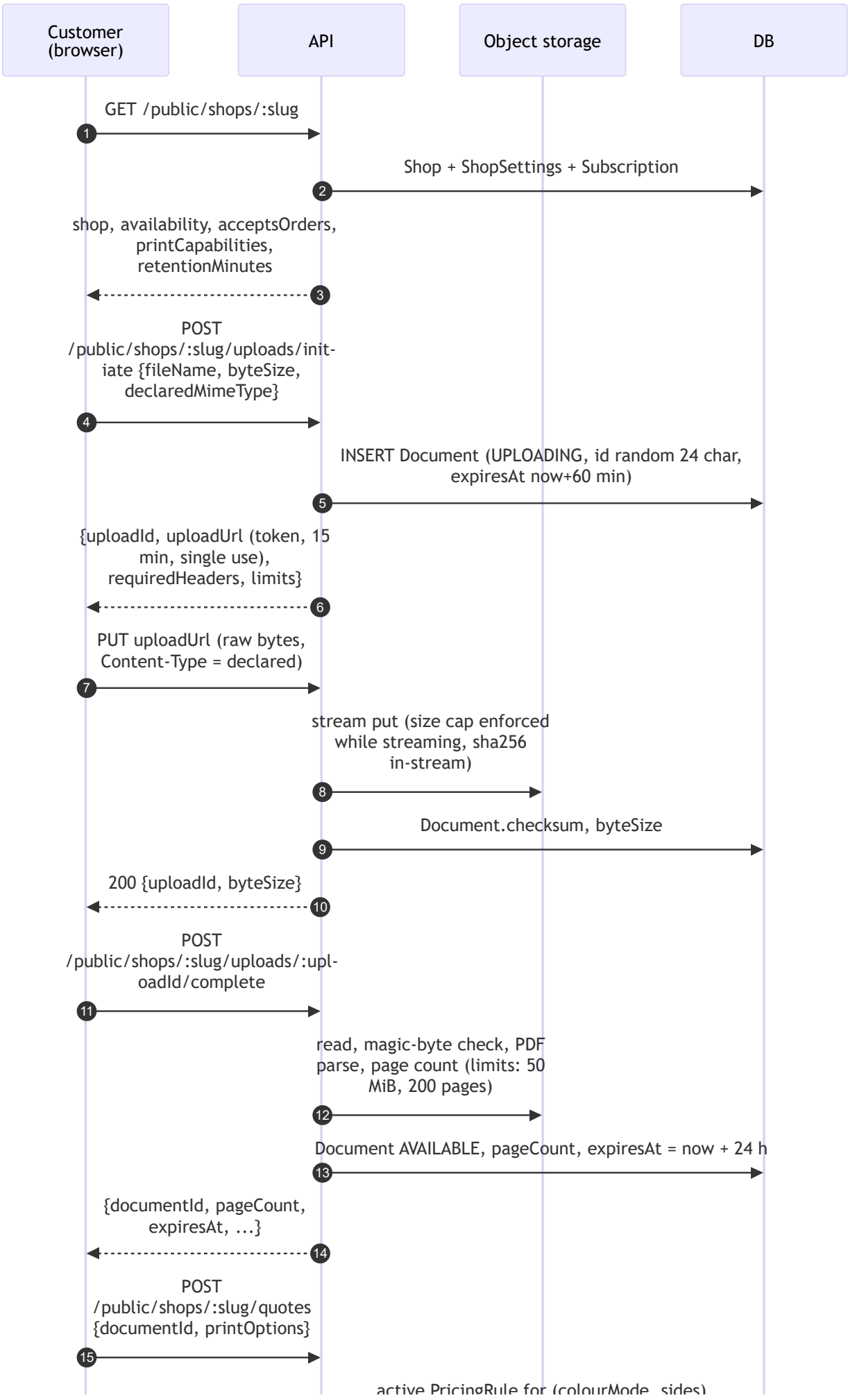


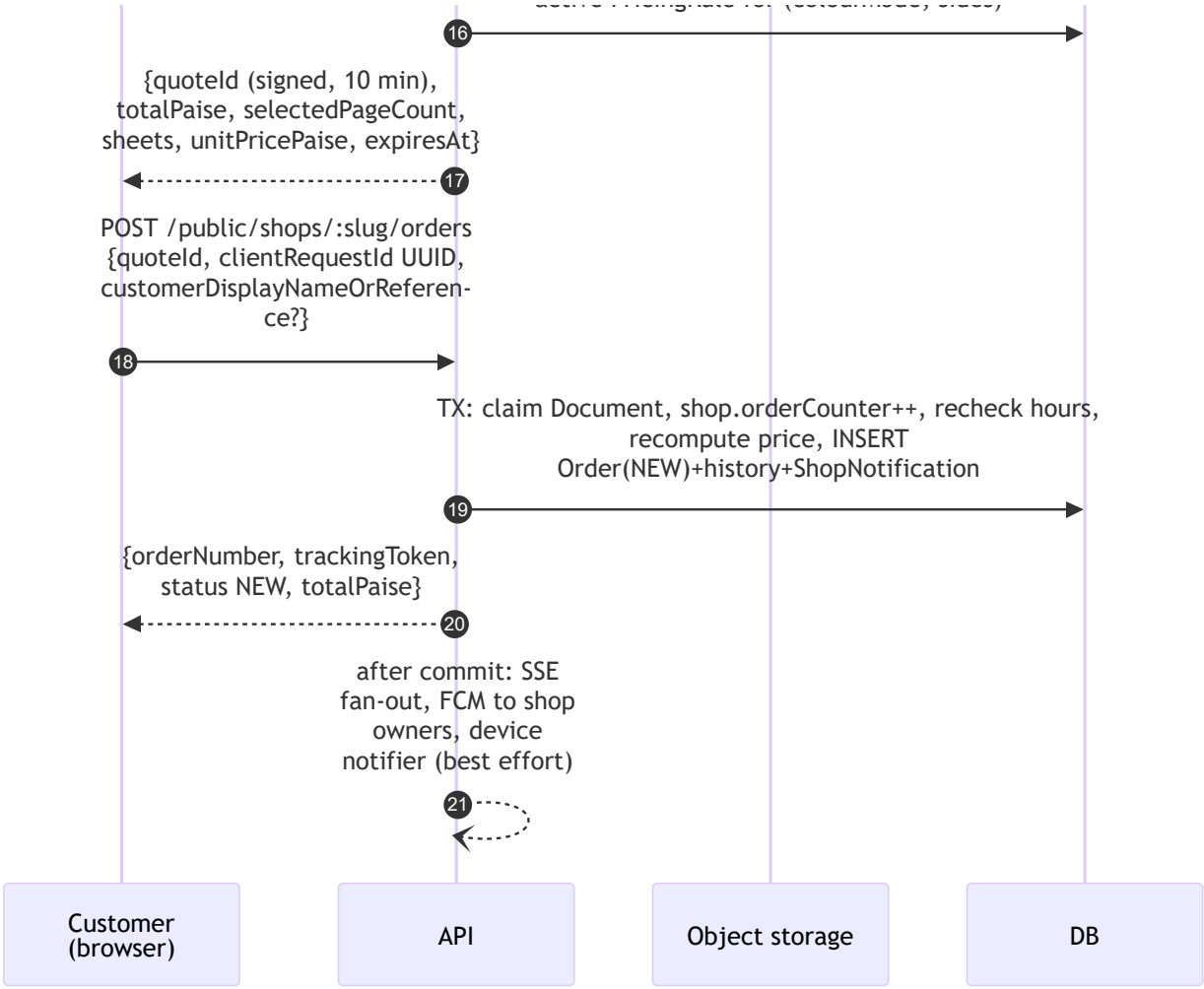
Lost refresh response (known sharp edge). The server rotates R1 *before* the response reaches the app. If the response is lost (timeout after the request was sent), the app still holds R1, which is now spent: re-presenting it within `REFRESH_REUSE_GRACE_SECONDS` (5 s) returns `409 REFRESH_CONFLICT`, and later returns `401 REFRESH_TOKEN_INVALID` **and revokes the whole token family**. There is no way to recover the new pair, so the only safe outcome is to sign in again. Keep the refresh call's timeout generous (about 30 s) and treat an unrecoverable refresh as "signed out", not as a crash.

Retry rule for **mutating** calls after a 401: only retry when the request carries an idempotency key (`clientRequestId`) or is naturally idempotent (PUT/DELETE of a known id). `POST /shop/orders/:id/print-now`, `transitions`, `print-jobs` all take a UUID `clientRequestId`: reuse the **same** UUID on retry.

D. Customer uploads PDF → gets quote → creates order

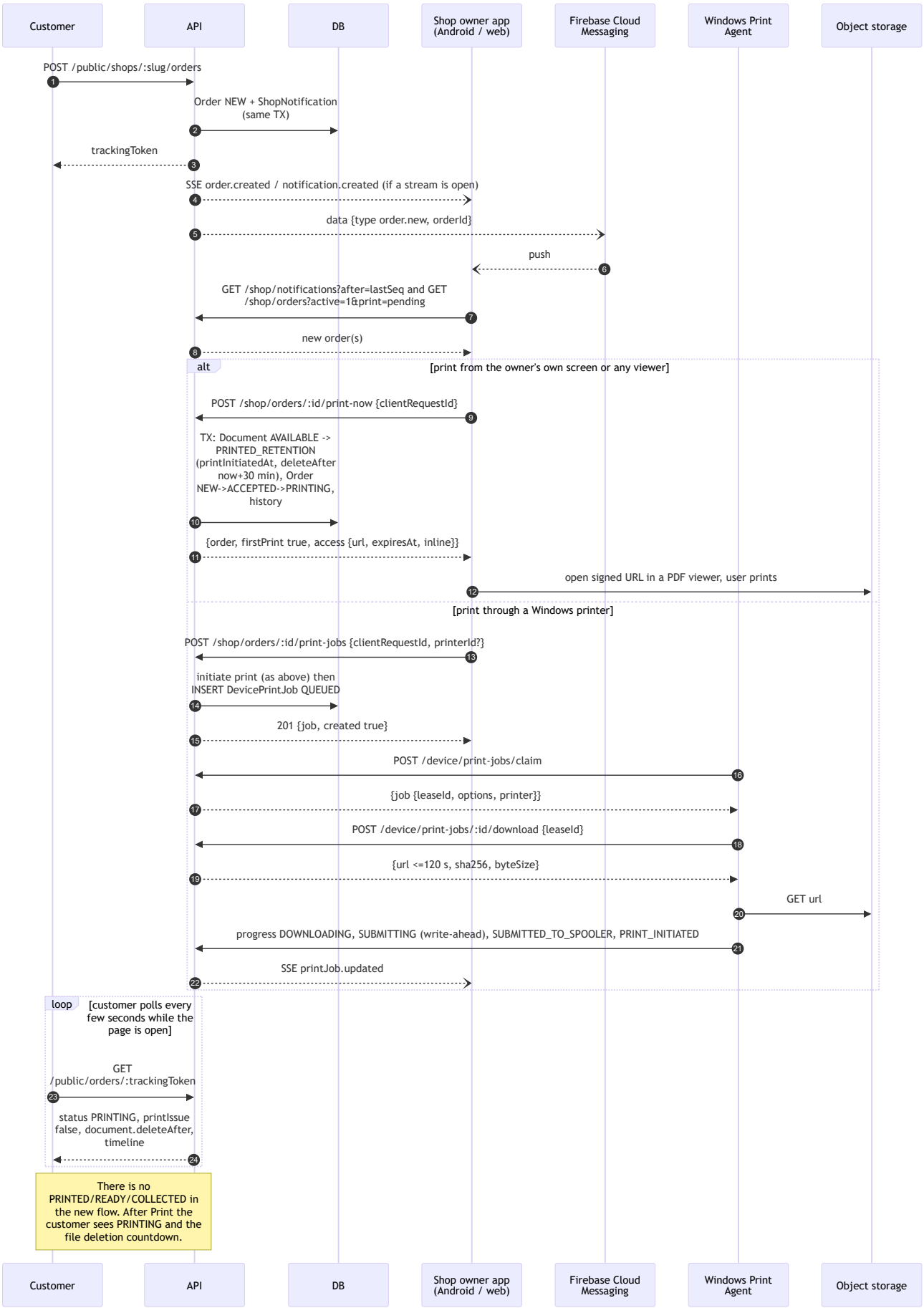
Customers are anonymous; there is no login. (This is the web flow; an Android customer client would follow the same sequence.)


Customer (browser)
API
Object storage
DB
GET /public/shops/:slug
1
Shop + ShopSettings + Subscription
2
shop, availability, acceptsOrders, printCapabilities, retentionMinutes
3
POST /public/shops/:slug/uploads/initiate {fileName, byteSize, declaredMimeType}
4
INSERT Document (UPLOADING, id random 24 char, expiresAt now+60 min)
5
{uploadId, uploadUrl (token, 15 min, single use), requiredHeaders, limits}
6
PUT uploadUrl (raw bytes, Content-Type = declared)
7
stream put (size cap enforced while streaming, sha256 in-stream)
8
Document.checksum, byteSize
9
200 {uploadId, byteSize}
10
POST /public/shops/:slug/uploads/:uploadId/complete
11
read, magic-byte check, PDF parse, page count (limits: 50 MiB, 200 pages)
12
Document AVAILABLE, pageCount, expiresAt = now + 24 h
13
{documentId, pageCount, expiresAt, ...}
14
POST /public/shops/:slug/quotes {documentId, printOptions}
15
active PricingRule for (colourMode, sides)

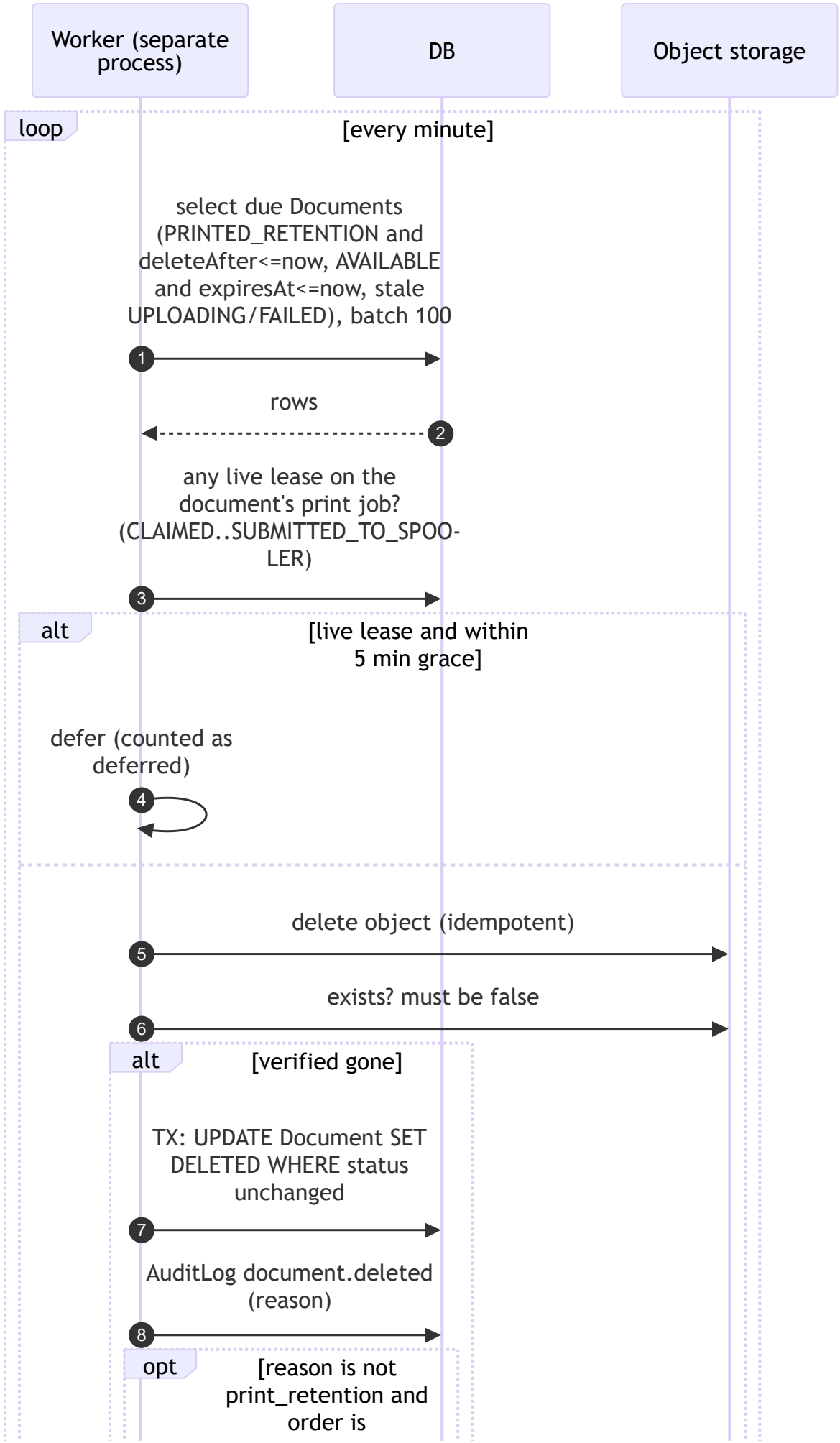


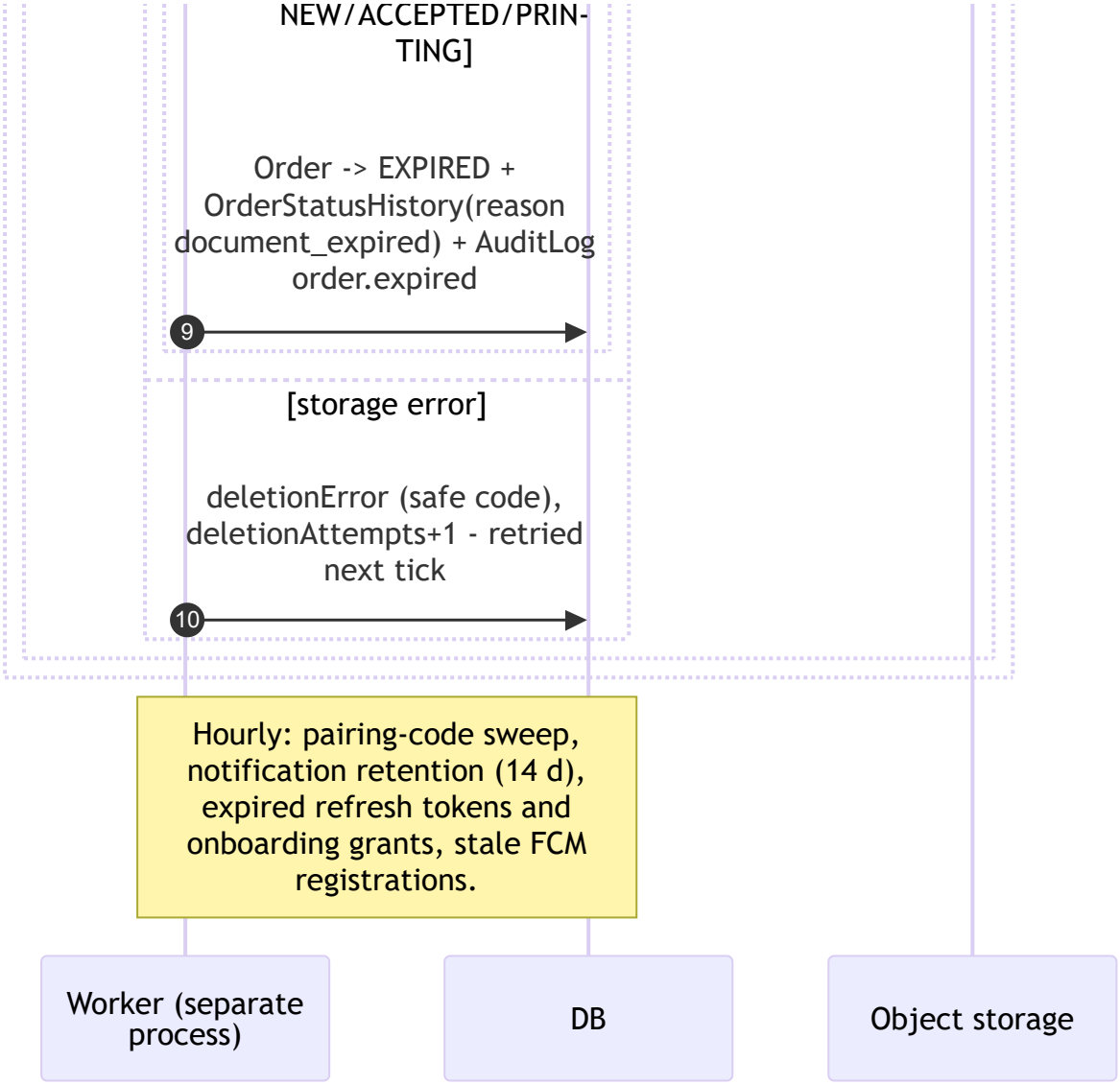
Repeat the final POST with the same `clientRequestId` and the original order is returned (idempotent).

E. Shop receives order → prints → customer tracking updates



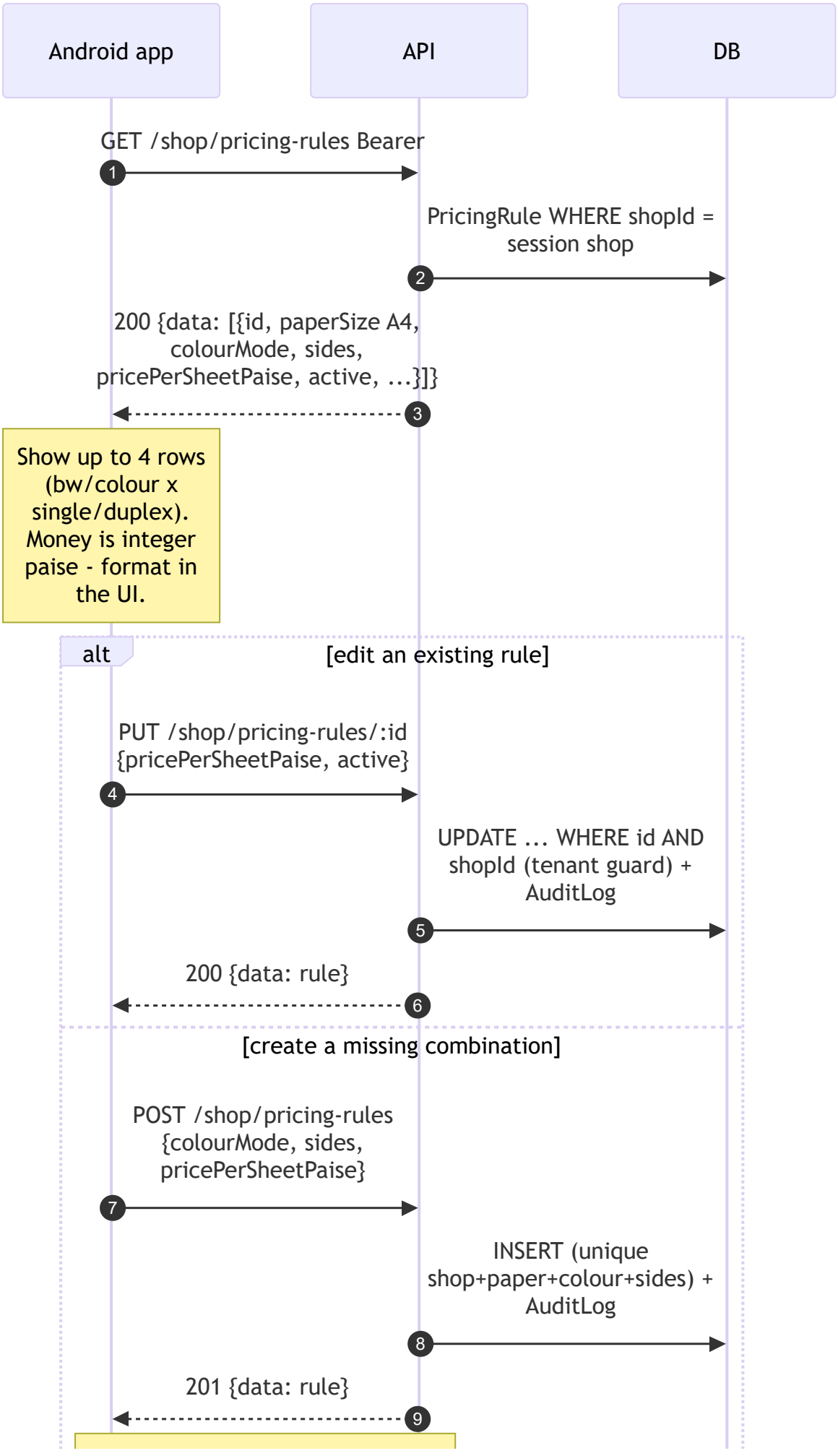
F. Document retention expiry → worker deletes file

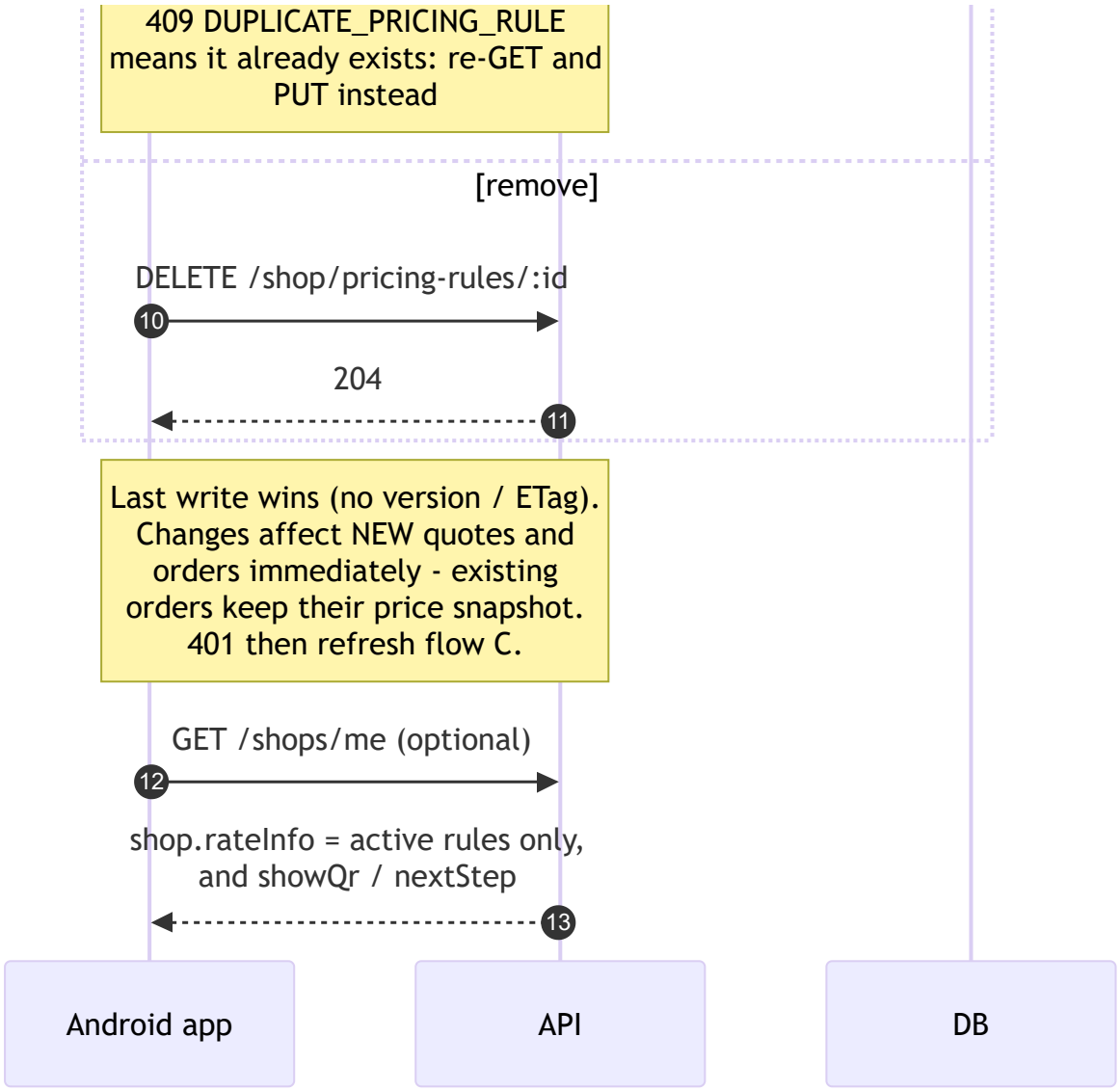

Worker (separate process)
DB
Object storage
loop
[every minute]
select due Documents (PRINTED_RETENTION and deleteAfter<=now, AVAILABLE and expiresAt<=now, stale UPLOADING/FAILED), batch 100
1
rows
2
any live lease on the document's print job? (CLAIMED..SUBMITTED_TO_SPOO-LER)
3
alt
[live lease and within 5 min grace]
defer (counted as deferred)
4
delete object (idempotent)
5
exists? must be false
6
alt
[verified gone]
TX: UPDATE Document SET DELETED WHERE status unchanged
7
AuditLog document.deleted (reason)
8
opt
[reason is not print_retention and order is



Orders and metadata are never deleted. API clients see the effect as `document.status = DELETED`, `fileName = null` on the tracking page, and `410 DOCUMENT_UNAVAILABLE` from document-access. The worker cannot push SSE (different process): clients refetch.

G. Android fetches and edits shop pricing rules


Android app
API
DB
GET /shop/pricing-rules Bearer
1
PricingRule WHERE shopId = session shop
2
200 {data: [{id, paperSize A4, colourMode, sides, pricePerSheetPaise, active, ...}]}
3
Show up to 4 rows (bw/colour x single/duplex). Money is integer paise - format in the UI.
alt
[edit an existing rule]
PUT /shop/pricing-rules/:id {pricePerSheetPaise, active}
4
UPDATE ... WHERE id AND shopId (tenant guard) + AuditLog
5
200 {data: rule}
6
[create a missing combination]
POST /shop/pricing-rules {colourMode, sides, pricePerSheetPaise}
7
INSERT (unique shop+paper+colour+sides) + AuditLog
8
201 {data: rule}
9



System architecture

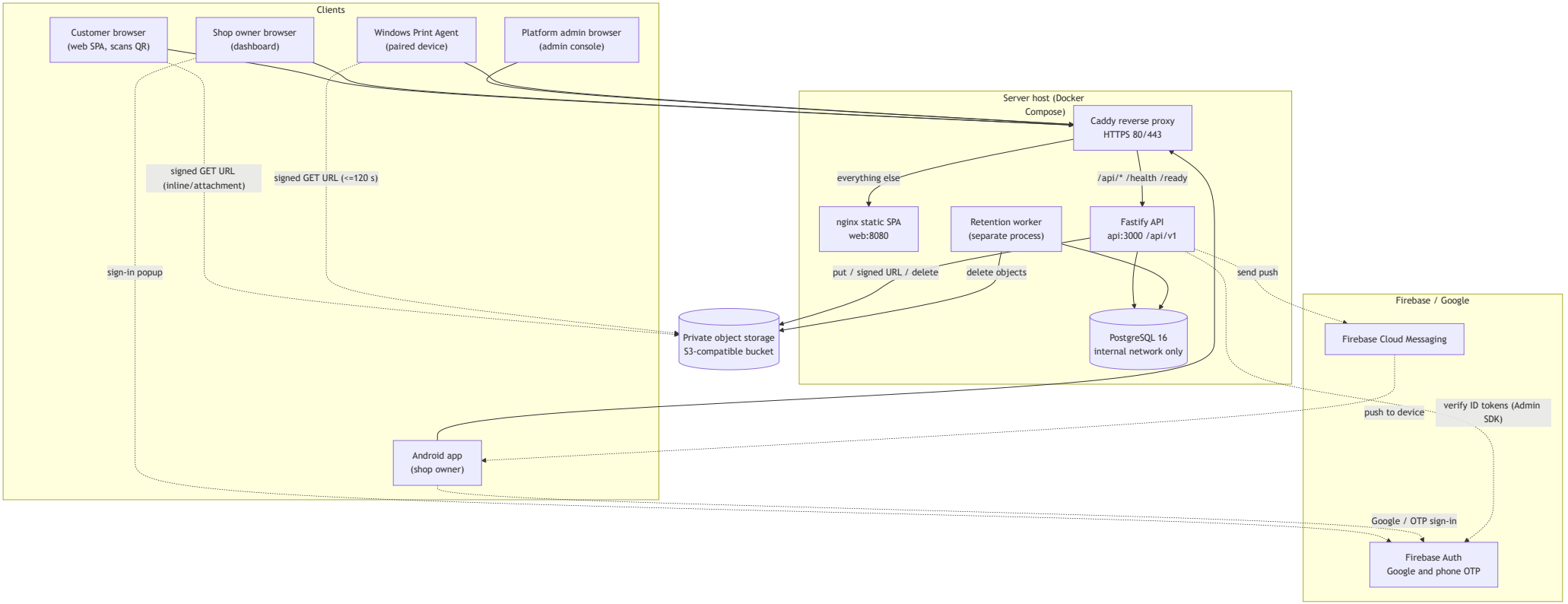
ER diagrams show data relationships only. This document shows **who talks to whom**. Everything is derived from the code and from `deploy/` (Caddy + Docker Compose); items that depend on production configuration are marked **Requires confirmation**.

0. Components at a glance

Component	Technology	Role
Deployment proxy	Caddy (`deploy/Caddyfile`)	The only public listener (80/443). Automatic HTTPS. Routes `/api/*`, `/health`, `/ready` to the API and everything else to the web container. Streams uploads (no request buffering), never buffers SSE, overwrites `X-Forwarded-For`.
Web frontend	React SPA (`apps/web`), served by nginx (`web:8080`)	Customer pages (`/p/<slug>`, tracking), shop-owner dashboard, admin console. Calls the same `/api/v1`.
API	Node + Fastify (`apps/api`), single instance, port 3000	All business logic, auth, signed URLs, SSE hub, FCM sender. Prefix `/api/v1`.
Database	PostgreSQL 16 (`postgres`, internal network only)	Prisma ORM; 32 models.
Object storage	S3-compatible private bucket (`STORAGE_DRIVER=s3`) or local disk (`local`)	Customer documents. Never public; reached only through short-lived signed URLs. Production driver: **Requires confirmation**.
Background worker	Same image, `worker.js` (separate process)	Retention: deletes expired documents, expires orders, sweeps pairing codes, notifications, refresh tokens, grants and FCM rows. Cannot push SSE (separate process).
Windows Print Agent	Node/PowerShell app (`apps/windows-agent`)	Paired device. Reports printers, leases print jobs, downloads the PDF via signed URL, hands it to the Windows spooler.
Android app	Kotlin (not in `main`; see below)	Shop-owner app: user Bearer tokens, FCM, optional foreground SSE.
Firebase	Firebase Auth + FCM (project id from `FIREBASE_PROJECT_ID`)	Sign-in with Google / phone OTP (client SDK) and push delivery. The API verifies ID tokens with the Admin SDK and sends FCM with it.

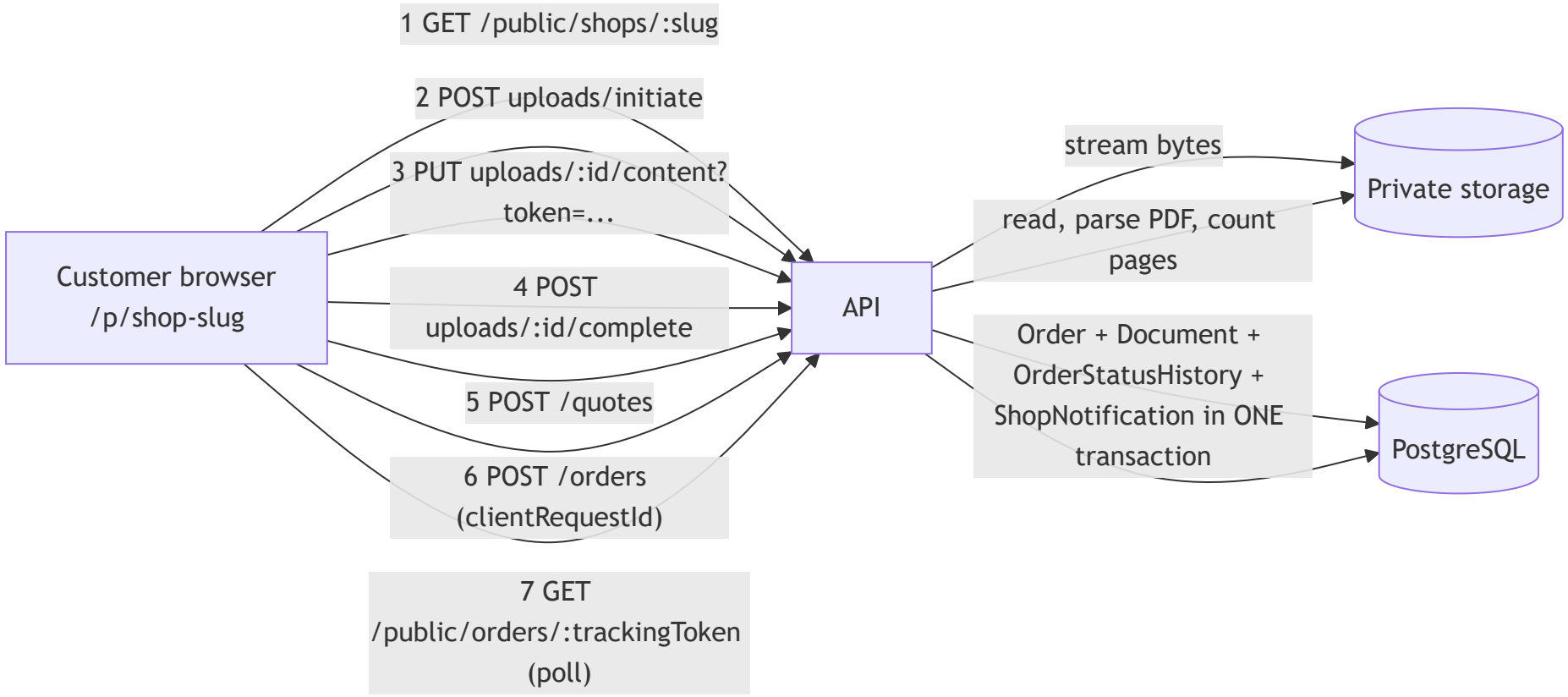
> **Branch status.** On `origin/main` (`080c6bd`): web app, API (including Phase A: live queue filters, customer tracking, privacy hardening), Windows agent, worker, mobile auth/profile/FCM APIs, Google sign-up/onboarding (PR #39), free-platform change (PR #40). **Not on `main`:** the Android application source (`feature/daksh-android-companion`, plus `phase3/android*`), and any Phase B work (`feature/daksh-phase-b-smart-print-printers` currently has zero commits beyond `main`). This documentation describes `main` only.

1. Whole system

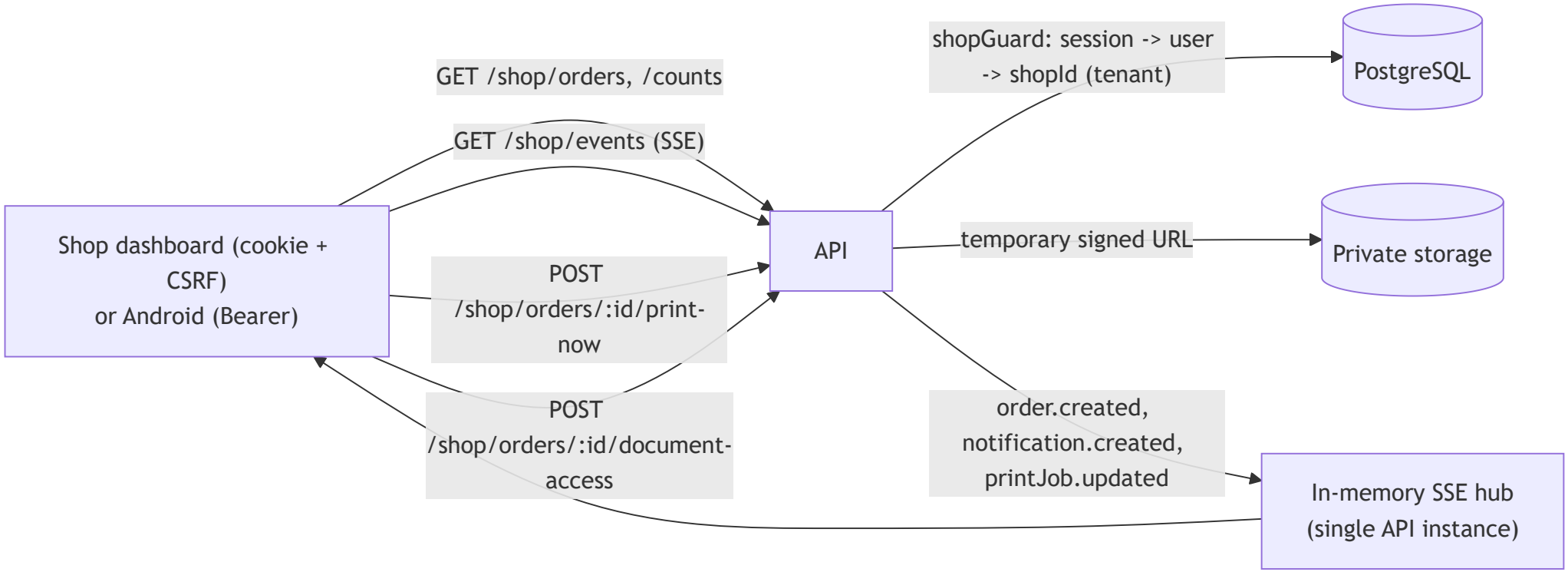


Notes: the browser PUTs uploads to **the API** (`PUT /public/uploads/:id/content`, streamed straight into storage); documents are read back through **signed URLs** pointing at the object store (S3 driver) or at `/api/v1/internal/documents/:key` (local driver).

2. Customer web → API → database / storage

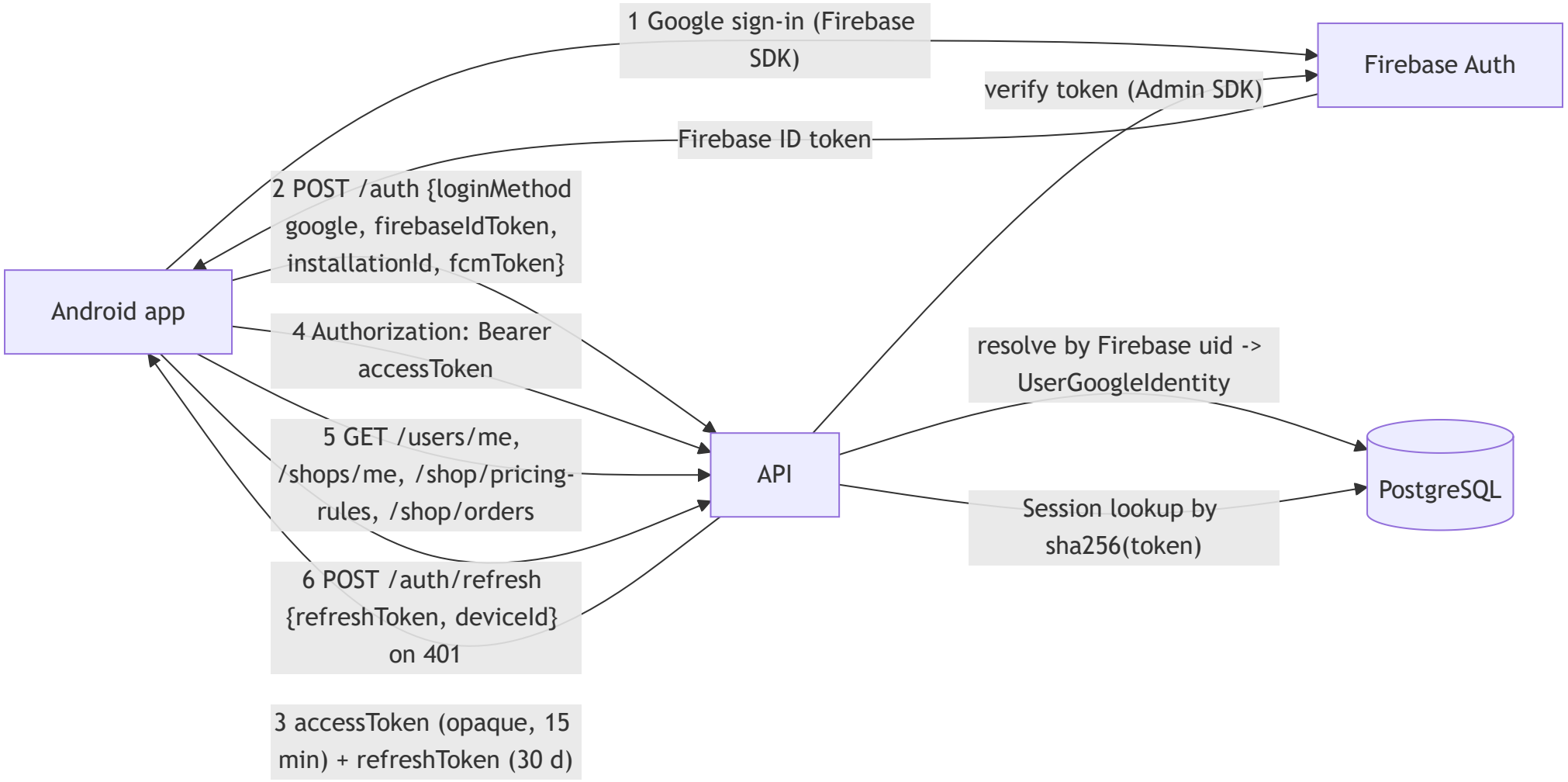


3. Shop dashboard → API → shop orders



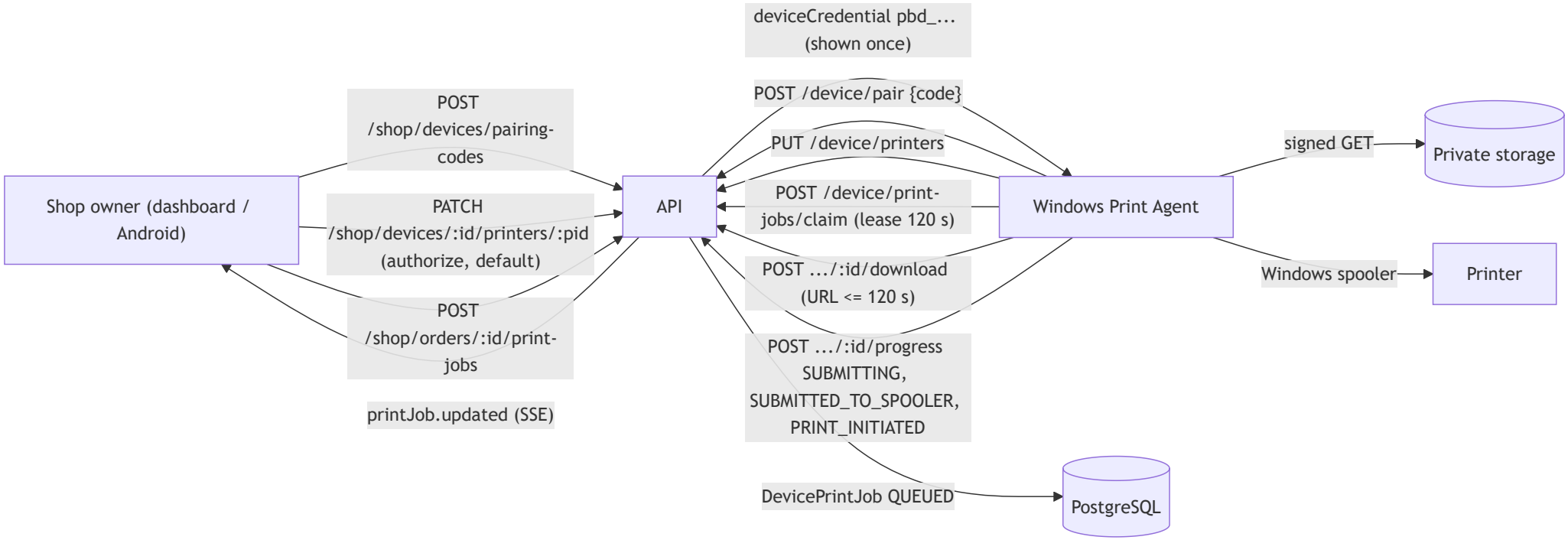
The tenant (`shopId`) is **always derived from the authenticated session**; no shop route accepts a shop id from the client.

4. Android app → mobile auth → API → shop data

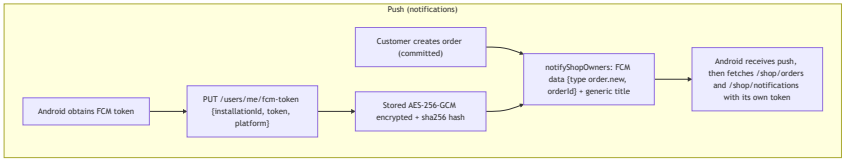


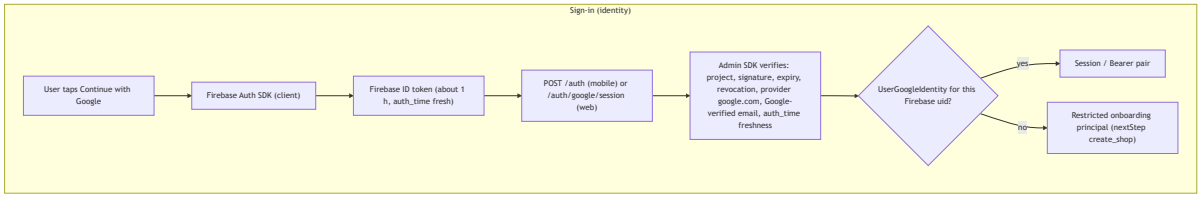
The Firebase ID token is used **once**, at sign-in. Every later call carries the API's own opaque access token. Firebase ID tokens are not accepted by `/shop/*`, `/users/*` or `/shops/*`.

5. Windows Print Agent → device APIs → print-job lifecycle

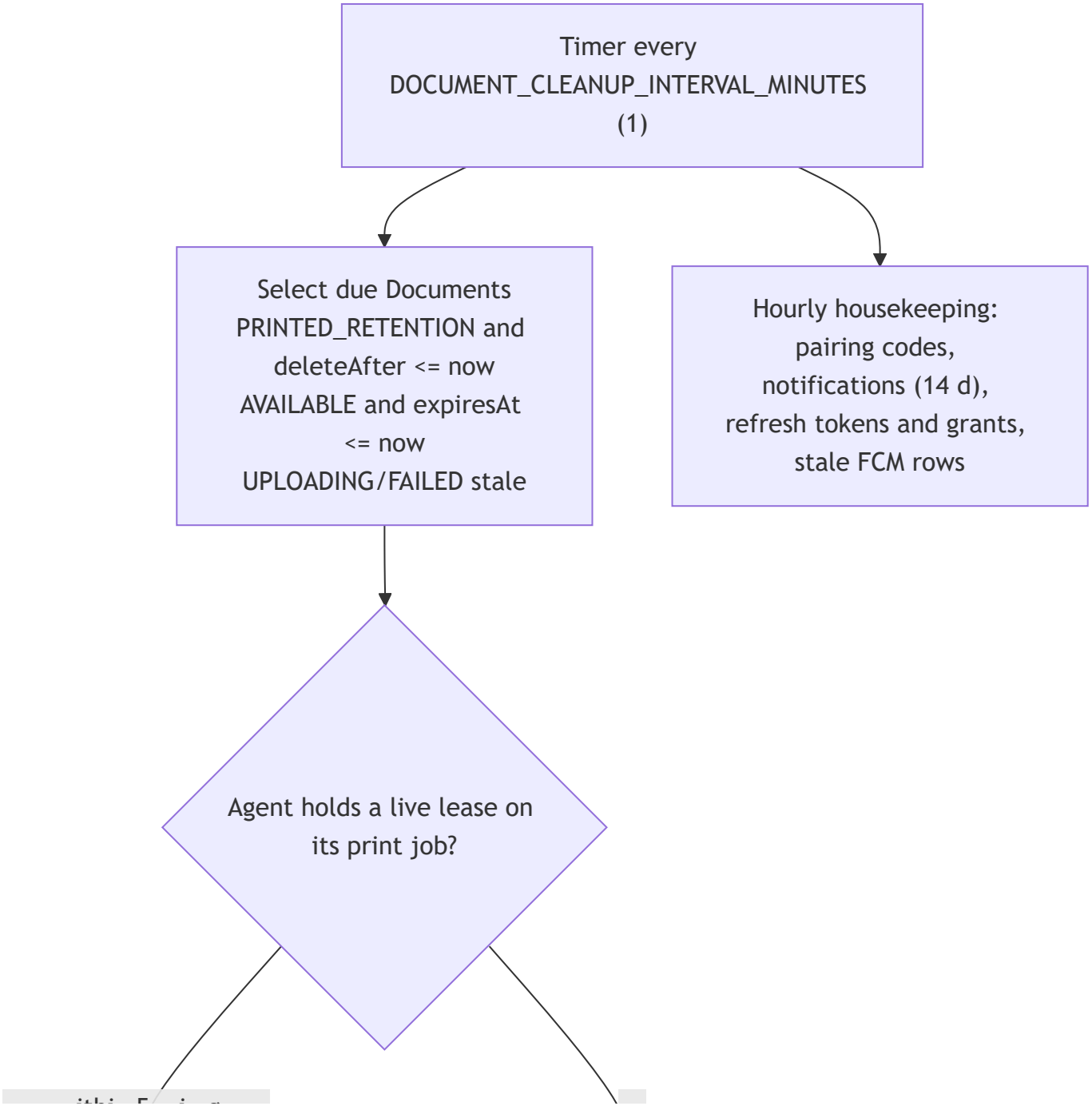


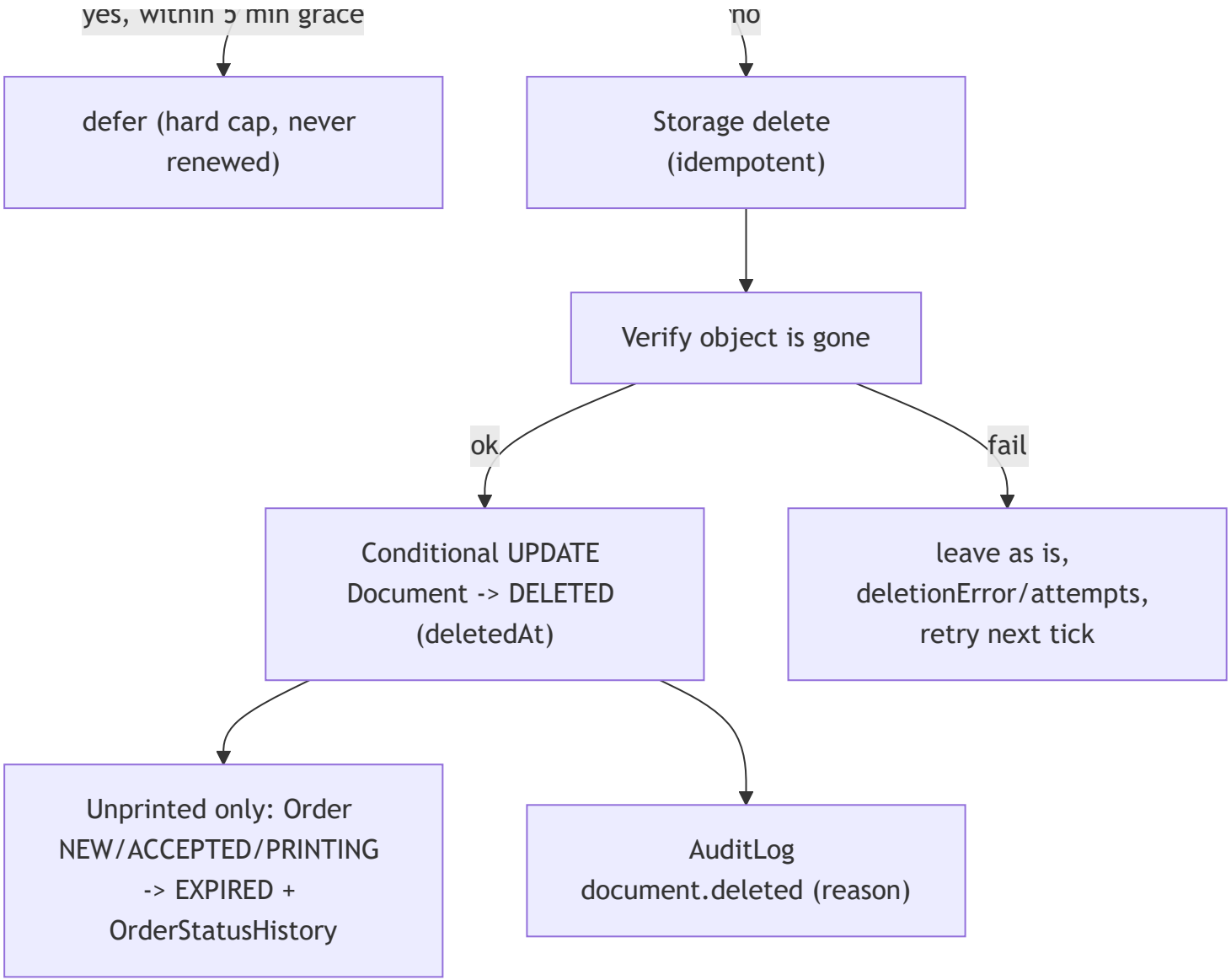
6. Firebase → Google sign-in / FCM → backend





7. Worker → file retention → object storage cleanup


Timer every DOCUMENT_CLEANUP_INTERVAL_MINUTES (1)
Select due Documents PRINTED_RETENTION and deleteAfter <= now AVAILABLE and expiresAt <= now UPLOADING/FAILED stale
Hourly housekeeping: pairing codes, notifications (14 d), refresh tokens and grants, stale FCM rows
Agent holds a live lease on its print job?



8. Trust boundaries

Boundary	Enforced by
Internet → API	Caddy TLS, per-IP rate limits, body-size caps, Helmet headers, CORS limited to `WEB_ORIGIN` (browsers only: native apps are unaffected)

Boundary	Enforced by
Tenant isolation	`shopId` from session/device row; every query includes it; foreign ids are indistinguishable from unknown ids (404)
Database	Postgres on an `internal: true` Docker network (no route to the internet, no published port)
Storage	Bucket is private; only the API and worker hold credentials; clients get ≤300 s signed URLs bounded by the document's retention deadline
Worker	Separate process and container with only storage-egress network access

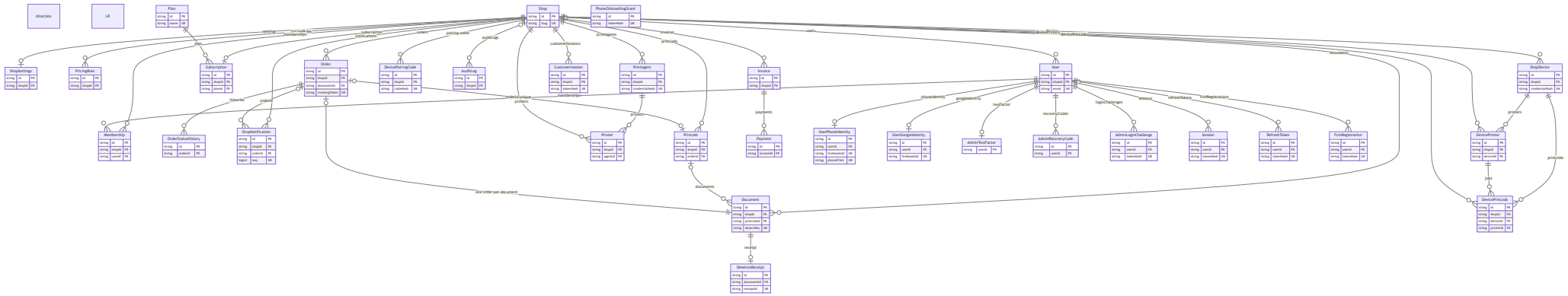
Full database ER diagram

Source: `apps/api/prisma/schema.prisma` on `origin/main` (`080c6bd`), PostgreSQL 16, 12 migrations (`20261003120000_initial` … `20261023100000_user_profile_onboarding`). **32 models, 14 enums.** Nothing here is invented: every entity, key and relationship is in the schema.

How to read the diagrams: `PK` primary key, `FK` foreign key, `UK` unique. `||--o{` = exactly one → zero-or-more; `|o--o{` = optional parent (nullable FK) → zero-or-more; `||--o|` = one → zero-or-one. Enum-typed columns show the enum name as their type; the enums are listed below. The full diagram shows keys only; the domain diagrams carry the important fields.

Full system diagram (all 32 models, keys only)

> This diagram is deliberately complete and therefore wide (every model hangs off `Shop` / `User`). Zoom in the rendered Markdown (GitHub/VS Code preview); for reading, use the per-domain diagrams linked in the table below, which show the important fields at a legible size.



Model → domain document

#	Model	Domain doc	In active use on `main`?
1	Shop	shop-pricing	Yes
2	ShopSettings	shop-pricing	Yes
3	PricingRule	shop-pricing	Yes
4	Plan	legacy-billing	Admin API only
5	Subscription	legacy-billing	Admin API; intake gate

#	Model	Domain doc	In active use on `main`?
6	Invoice	legacy-billing	**No** (schema only)
7	Payment	legacy-billing	**No** (schema only)
8	Membership	shop-pricing	**No** (schema only; not used for authorization)
9	User	auth	Yes
10	UserPhoneIdentity	auth	Yes
11	UserGoogleIdentity	auth	Yes
12	Session	auth	Yes
13	RefreshToken	auth	Yes
14	PhoneOnboardingGrant	auth	Yes
15	AdminTwoFactor	admin-security	Yes
16	AdminRecoveryCode	admin-security	Yes
17	AdminLoginChallenge	admin-security	Yes
18	AuditLog	admin-security	Yes
19	Document	orders-documents	Yes
20	Order	orders-documents	Yes
21	OrderStatusHistory	orders-documents	Yes
22	DeletionReceipt	orders-documents	**No** (schema only)
23	CustomerSession	orders-documents	**No** (schema only)
24	ShopDevice	devices-printers	Yes
25	DevicePairingCode	devices-printers	Yes
26	DevicePrinter	devices-printers	Yes
27	DevicePrintJob	devices-printers	Yes
28	PrintAgent	devices-printers	**No** (schema only)
29	Printer	devices-printers	**No** (schema only)
30	PrintJob	devices-printers	**No** (schema only; only `Document.printJobId` points at it)
31	ShopNotification	notifications	Yes
32	FcmRegistration	notifications	Yes

8 of the 32 models are schema-only "target architecture" tables that no route, worker or domain code reads or writes (verified by searching `apps/api/src` for any Prisma call on them): `Invoice`, `Payment`, `Membership`, `DeletionReceipt`, `CustomerSession`, `PrintAgent`, `Printer`, `PrintJob`. The other **24 are in active use**. `Plan` and `Subscription` are used only by the admin API and the order-intake gate.

Enums

Enum	Values	Used by
`Role`	`SHOP_OWNER`, `PLATFORM_ADMIN`	User.role
`ShopStatus`	`ACTIVE`, `SUSPENDED`, `PILOT`, `PAST_DUE`, `CANCELLED`	Shop.status. **Only `ACTIVE` and `SUSPENDED` are ever set by application code today**; `PILOT` / `PAST_DUE` / `CANCELLED` are reserved (an admin can set them through `PUT /admin/shops/:id`).
`OrderStatus`	`NEW`, `ACCEPTED`, `PRINTING`, `PRINTED`, `READY`, `COLLECTED`, `CANCELLED`, `EXPIRED`	Order.status, OrderStatusHistory
`DocumentStatus`	`UPLOADING`, `AVAILABLE`, `PRINTED_RETENTION`, `DELETED`, `FAILED`	Document.status
`ColourMode`	`bw`, `colour`	PricingRule.colourMode
`PrintSides`	`single`, `duplex`	PricingRule.sides
`SubscriptionStatus`	`ACTIVE`, `SUSPENDED`, `CANCELLED`	Subscription.status
`AfterHoursPolicy`	`ACCEPT`, `BLOCK`	ShopSettings.afterHoursPolicy
`DevicePlatform`	`ANDROID`, `WINDOWS`	ShopDevice.platform
`DeviceStatus`	`ACTIVE`, `REVOKED`	ShopDevice.status
`MembershipRole`	`OWNER`, `STAFF`	Membership.role
`PrintJobStatus`	`UPLOADING`, `QUEUED`, `PRINTING`, `PRINTED`, `FAILED`, `CANCELLED`, `EXPIRED`	PrintJob.status (unused table)
`PrinterConnectivity`	`ONLINE`, `OFFLINE`, `ERROR`, `UNKNOWN`	DevicePrinter.connectivity
`DevicePrintJobStatus`	`QUEUED`, `CLAIMED`, `DOWNLOADING`, `SUBMITTING`, `SUBMITTED_TO_SPOOLER`, `PRINT_INITIATED`, `FAILED`, `CANCEL_REQUESTED`, `CANCELED`, `NEEDS_RECONCILIATION`	DevicePrintJob.status

Logical references that are NOT foreign keys

These columns hold another table's id but have **no FK constraint** in the schema, so the database does not enforce them (and they survive deletion of the target):

Column	Points at
`DevicePrintJob.orderId`, `DevicePrintJob.documentId`	Order, Document

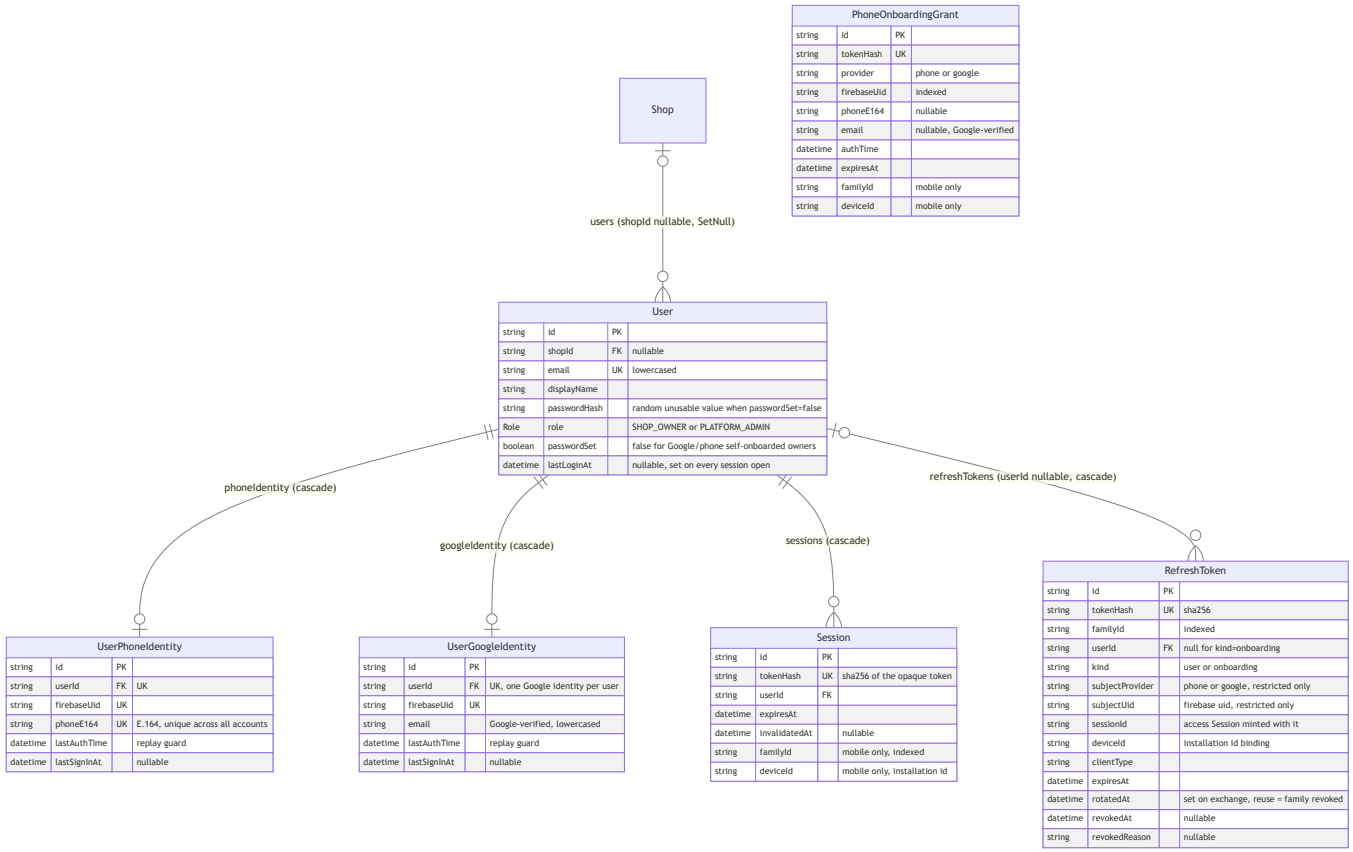
Column	Points at
`OrderStatusHistory.actorUserId`, `actorDeviceId`	User, ShopDevice
`AuditLog.actorUserId`, `actorDeviceId`, `targetId`	User, ShopDevice, polymorphic target
`DevicePairingCode.usedByDeviceId`, `createdByUserId`	ShopDevice, User
`ShopDevice.revokedByUserId`	User
`DevicePrintJob.requestedByUserId`, `resolvedByUserId`	User
`Session.familyId` ↔ `RefreshToken.familyId`	the mobile token family (a shared value, not a table)
`Session.deviceId`, `RefreshToken.deviceId` ↔ `FcmRegistration.installationId`	the app installation id chosen by the client
`RefreshToken.sessionId`	the access `Session` minted with it
`PhoneOnboardingGrant.*`	standalone: no relation to any other table (`firebaseUid`, `familyId` are plain strings)

Facts for reviewers

- `Shop` is the tenant root. Every shop-scoped table carries `shopId`, except `User.shopId` (nullable, `SetNull`), `AuditLog.shopId` (nullable), and child tables that inherit tenancy through their parent (`OrderStatusHistory`, `Payment`, `AdminRecoveryCode`, …).
- Delete semantics: orders, documents, print jobs and invoices use `Restrict` (a shop that has them cannot be deleted); settings, pricing, devices, notifications and sessions use `Cascade`.
- A user belongs to exactly one shop (`User.shopId`). `Membership` exists but is **not** used for authorization.

ER: authentication and identity

Models: `User`, `UserPhoneIdentity`, `UserGoogleIdentity`, `Session`, `RefreshToken`, `PhoneOnboardingGrant` (plus `Shop` for context). Admin 2FA tables are in er-admin-security.md; push registrations in er-notifications.md.



Constraints

Table	Unique	Foreign keys (onDelete)	Notable indexes
User	`email`	`shopId → Shop` (SetNull, nullable)	
UserGoogleIdentity	`userId`, `firebaseUid`	`userId → User` (Cascade)	
UserPhoneIdentity	`userId`, `firebaseUid`, `phoneE164`	`userId → User` (Cascade)	
Session	`tokenHash`	`userId → User` (Cascade)	`userId`, `familyId`
RefreshToken	`tokenHash`	`userId → User` (Cascade, nullable)	`familyId`, `userId`, `subjectUid`, `expiresAt`
PhoneOnboardingGrant	`tokenHash`	none	`familyId`, `firebaseUid`, `expiresAt`

How the pieces fit

- **Web session:** `Session` row (no `familyId`/`deviceId`); the cookie holds the raw token, the DB holds only `sha256(token)`. TTL `SESSION_DAYS` (7).
- **Mobile session:** the **access token is a** `Session` with `familyId` + `deviceId` and a short TTL (`ACCESS_TOKEN_TTL_MINUTES`, 15). The **refresh token is a** `RefreshToken` in the same `familyId`, single-use, rotated on every `POST /auth/refresh`. Two different secrets, both stored only as SHA-256 hashes.

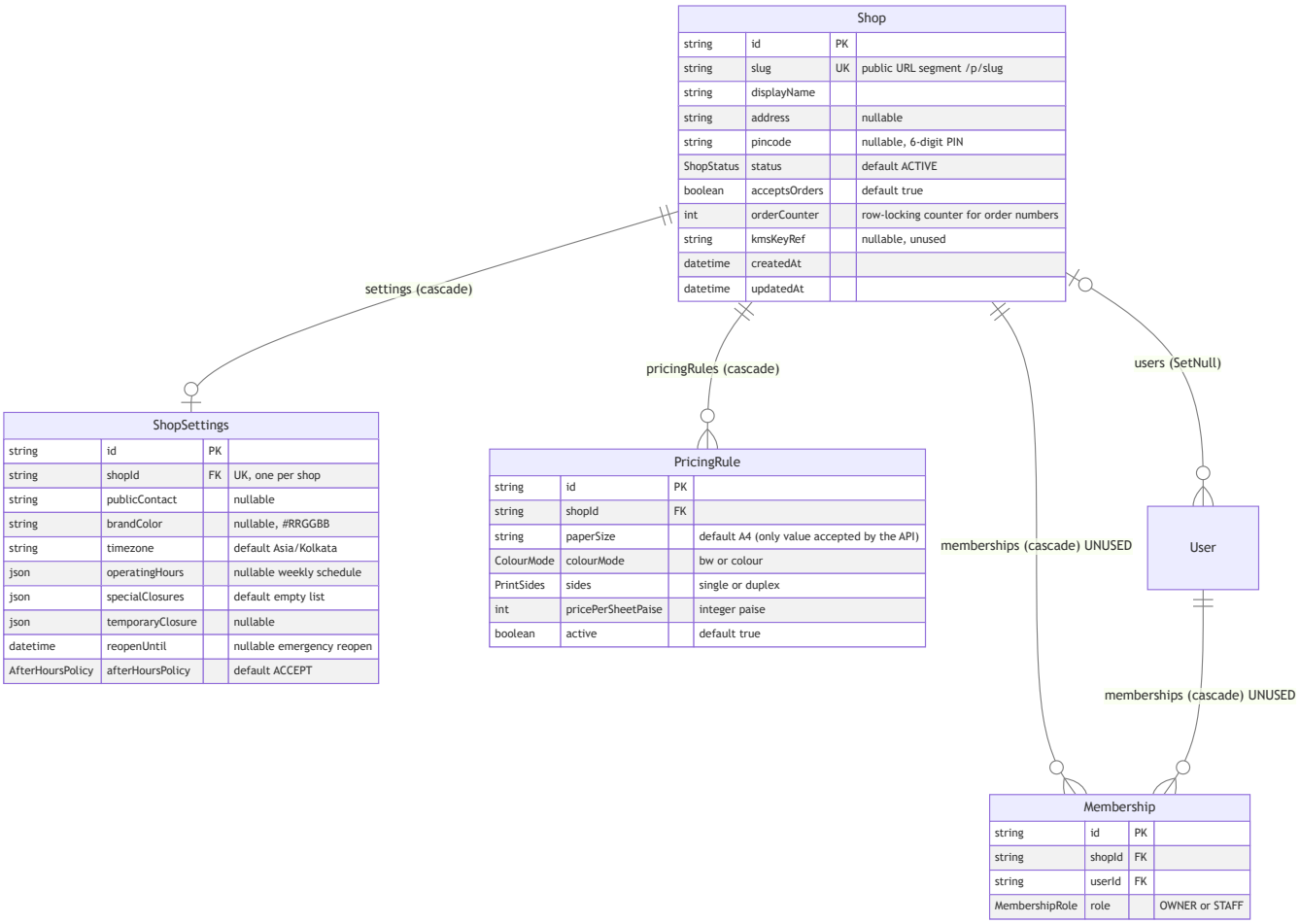
- **Restricted principal** (verified Firebase Google/phone identity with no account): `PhoneOnboardingGrant` (the raw value *is* the access token on mobile) + a `RefreshToken` with `kind = 'onboarding'` and `userId = null`. It can only call `POST /shops/onboarding`, `GET /users/me` and logout. After a successful onboarding the onboarding refresh row is revoked with reason `onboarded` and the app receives a normal pair.
- **Identity resolution is by Firebase uid only:** `UserGoogleIdentity.firebaseUid` / `UserPhoneIdentity.firebaseUid`. A matching email or phone never links or signs in an account; linking is an explicit, re-authenticated owner action.
- `Shop.users` can hold several users, but the business rule is one `SHOP_OWNER` per shop (`GET /shops/me` returns the oldest owner as `ownerId`).

Not in this diagram

The Firebase ID token, Firebase's own refresh token and the FCM token are **never stored** as such (FCM tokens are stored encrypted, see notifications). The Firebase Admin SDK verifies ID tokens server-side; nothing about a Firebase session lives in these tables except `firebaseUid`.

ER: shop and pricing

Models: `Shop`, `ShopSettings`, `PricingRule`, `Membership` (unused), `User` (for context). Subscription/plan tables are in er-legacy-billing.md.



Constraints

Table	Unique	Foreign keys	Notes
Shop	`slug`	none	Tenant root. `orderCounter` is incremented under the shop row lock when an order is created.
ShopSettings	`shopId`	`shopId → Shop` (Cascade)	Created lazily (`upsert`) and with onboarding.
PricingRule	`(shopId, paperSize, colourMode, sides)`	`shopId → Shop` (Cascade)	Max 4 effective rules (A4 only): bw/colour × single/duplex.
Membership	`(shopId, userId)`	`shopId → Shop`, `userId → User` (Cascade)	**Unused.** Do not treat as authorization.

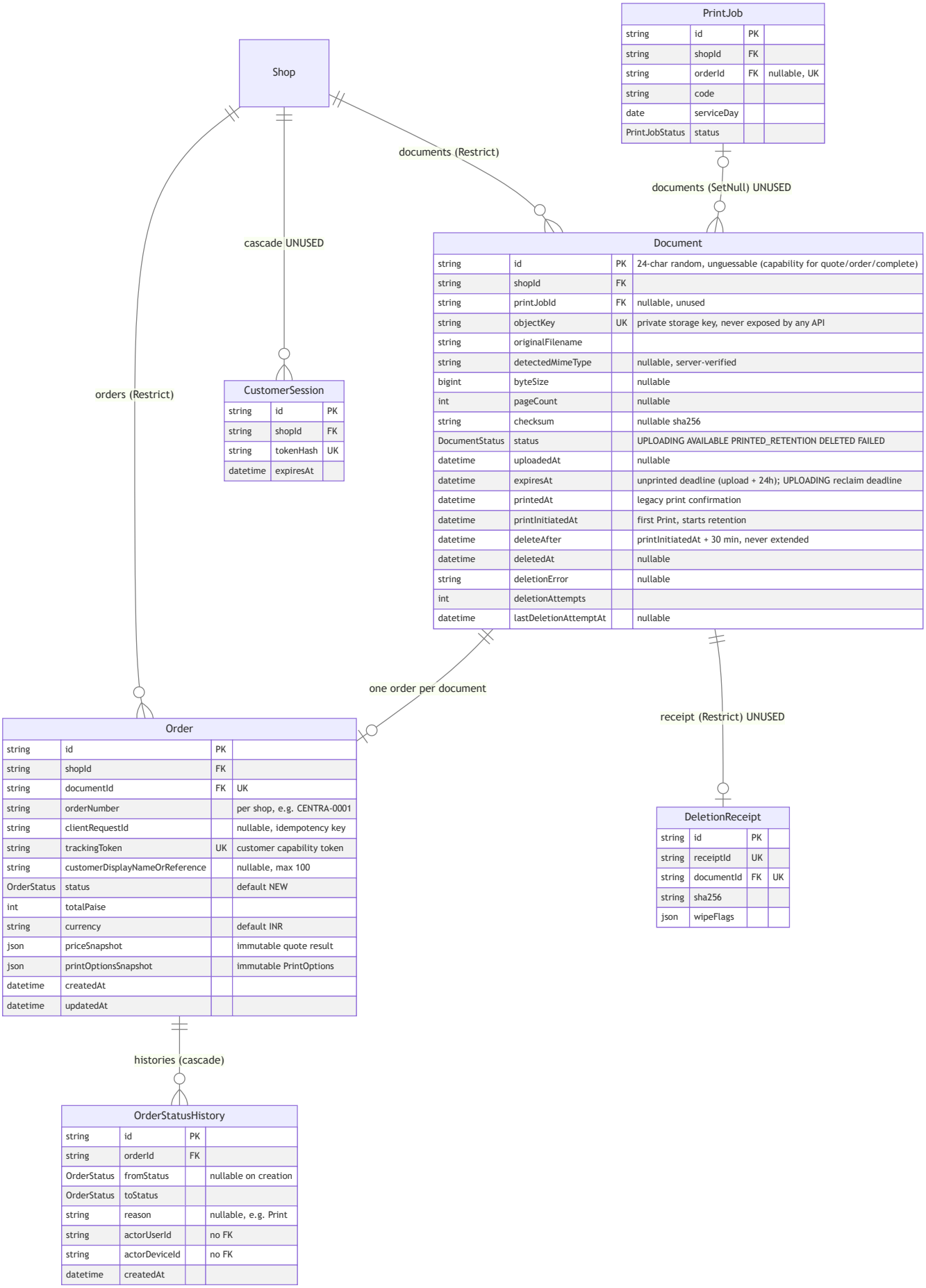
Behaviour that matters to an integrator

- **Pricing rule semantics:** a quote needs an *active* rule matching `(colourMode, sides)`. Sheets = `duplex ? ceil(pages/2) : pages`, times `copies`; total = sheets × `pricePerSheetPaise`. Missing rule → `422 NO_PRICING_RULE`. The cap is 2,000,000,000 paise per order.
- **Rule edits never change existing orders:** `Order.priceSnapshot` / `printOptionsSnapshot` freeze the price at order time (see er-orders-documents.md). The order endpoint also **recomputes** the price from the shop's *current* active rules; the quote total is not authoritative.

- **Operating hours** live in `ShopSettings` JSON columns (`operatingHours`, `specialClosures`, `temporaryClosure`) interpreted in `timezone`; `afterHoursPolicy = BLOCK` makes the public intake return `409 SHOP_CLOSED` when closed.
- **Open vs accepting:** the public site accepts orders only if `Shop.status = ACTIVE` AND `Shop.acceptsOrders` AND (no subscription OR subscription `ACTIVE`) AND hours allow it.
- **Self-onboarded shops** (since PR #40) are created `ACTIVE` with `acceptsOrders = true`, a `ShopSettings` row, **no** `Subscription`, and **no** pricing rules: the owner must add rules before customers can get a quote (`422 NO_PRICING_RULE`).

ER: orders and documents

Models: `Order`, `OrderStatusHistory`, `Document`, plus the schema-only `DeletionReceipt`, `CustomerSession`, `PrintJob` (shown dashed in prose, drawn normally below). Device print dispatch is in er-devices-printers.md.


Shop
PrintJob
string id PK
string shopId FK
string orderId FK nullable, UK
string code
date serviceDay
PrintJobStatus status
documents (Restrict)
documents (SetNull) UNUSED
cascade UNUSED
orders (Restrict)
CustomerSession
string id PK
string shopId FK
string tokenHash UK
datetime expiresAt
Document
string id PK 24-char random, unguessable (capability for quote/order/complete)
string shopId FK
string printJobId FK nullable, unused
string objectKey UK private storage key, never exposed by any API
string originalFilename
string detectedMimeType nullable, server-verified
bigint byteSize nullable
int pageCount nullable
string checksum nullable sha256
DocumentStatus status UPLOADING AVAILABLE PRINTED_RETENTION DELETED FAILED
datetime uploadedAt nullable
datetime expiresAt unprinted deadline (upload + 24h); UPLOADING reclaim deadline
datetime printedAt legacy print confirmation
datetime printInitiatedAt first Print, starts retention
datetime deleteAfter printInitiatedAt + 30 min, never extended
datetime deletedAt nullable
string deletionError nullable
int deletionAttempts
datetime lastDeletionAttemptAt nullable
one order per document
receipt (Restrict) UNUSED
Order
string id PK
string shopId FK
string documentId FK UK
string orderNumber per shop, e.g. CENTRA-0001
string clientRequestId nullable, idempotency key
string trackingToken UK customer capability token
string customerDisplayNameOrReference nullable, max 100
OrderStatus status default NEW
int totalPaise
string currency default INR
json priceSnapshot immutable quote result
json printOptionsSnapshot immutable PrintOptions
datetime createdAt
datetime updatedAt
DeletionReceipt
string id PK
string receiptId UK
string documentId FK UK
string sha256
json wipeFlags
histories (cascade)
OrderStatusHistory
string id PK
string orderId FK
OrderStatus fromStatus nullable on creation
OrderStatus toStatus
string reason nullable, e.g. Print
string actorUserId no FK
string actorDeviceId no FK
datetime createdAt

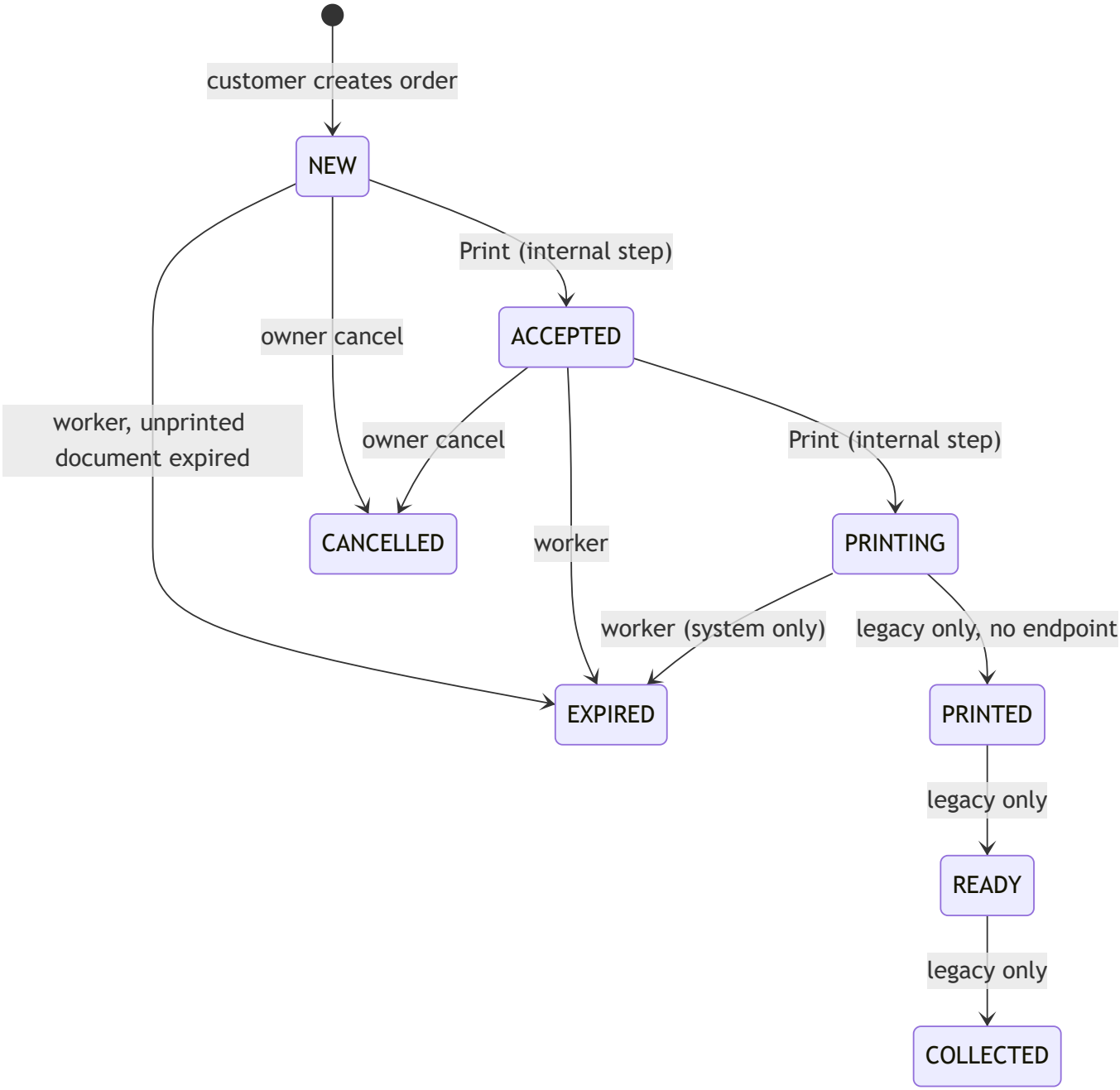
Constraints

Table	Unique	Foreign keys (onDelete)	Indexes
Document	`objectKey`	`shopId → Shop` (Restrict), `printJobId → PrintJob` (SetNull)	`(shopId, status)`, `(status, deleteAfter)`, `(status, expiresAt)`, `(status, createdAt)`, `printJobId`
Order	`documentId`, `trackingToken`, `(shopId, orderNumber)`, `(shopId, clientRequestId)`	`shopId → Shop` (Restrict), `documentId → Document` (no action)	`(shopId, status, createdAt)`, `(shopId, createdAt)`
OrderStatusHistory	none	`orderId → Order` (Cascade)	`(orderId, createdAt)`

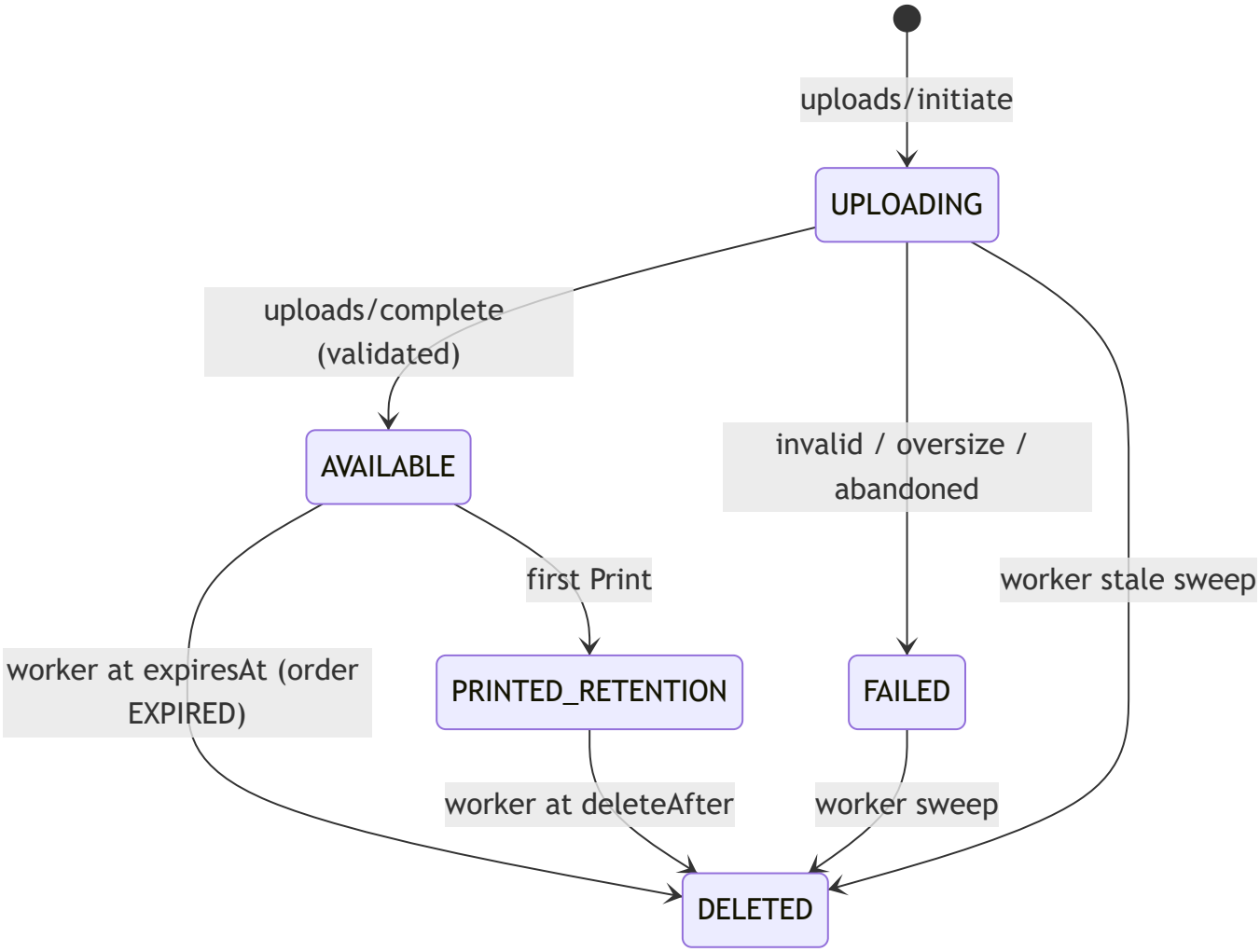
`(shopId, clientRequestId)` is the order-creation idempotency key (a NULL `clientRequestId` never collides). `documentId` unique enforces **one order per document**.

Lifecycles

Order status (`OrderStatus`): `NEW → ACCEPTED → PRINTING` is driven only by Print (`print-now` / `POST /device/orders/:id/print` / print-job enqueue). Owners may only **cancel** (`NEW|ACCEPTED → CANCELLED`). `PRINTING → EXPIRED` is system-only (retention worker). `PRINTED`, `READY`, `COLLECTED` are **legacy**: no endpoint creates them any more, old rows remain readable. Terminal: `COLLECTED`, `CANCELLED`, `EXPIRED`.



Document status: `UPLOADING` (initiate) → `AVAILABLE` (complete, `expiresAt = +24 h`) → `PRINTED_RETENTION` (first Print: `printInitiatedAt = now`, `deleteAfter = now + 30 min`) → `DELETED` (worker, only after the object is verified gone). `FAILED` = rejected/abandoned upload. Unprinted documents are deleted at `expiresAt` and their order becomes `EXPIRED`.



JSON column shapes

`Order.printOptionsSnapshot` = `{ paperSize: "A4", colourMode: "bw"|"colour", sides: "single"|"duplex", copies: 1..1000, pageSelection: {mode:"all"} | {mode:"ranges", ranges:[{from,to}]} }` (ranges are normalised: sorted, merged, in bounds).

`Order.priceSnapshot` = `{ printOptions, selectedPageCount, sheetsPerCopy, totalSheets, unitPricePaise, totalPaise, currency: "INR" }`.

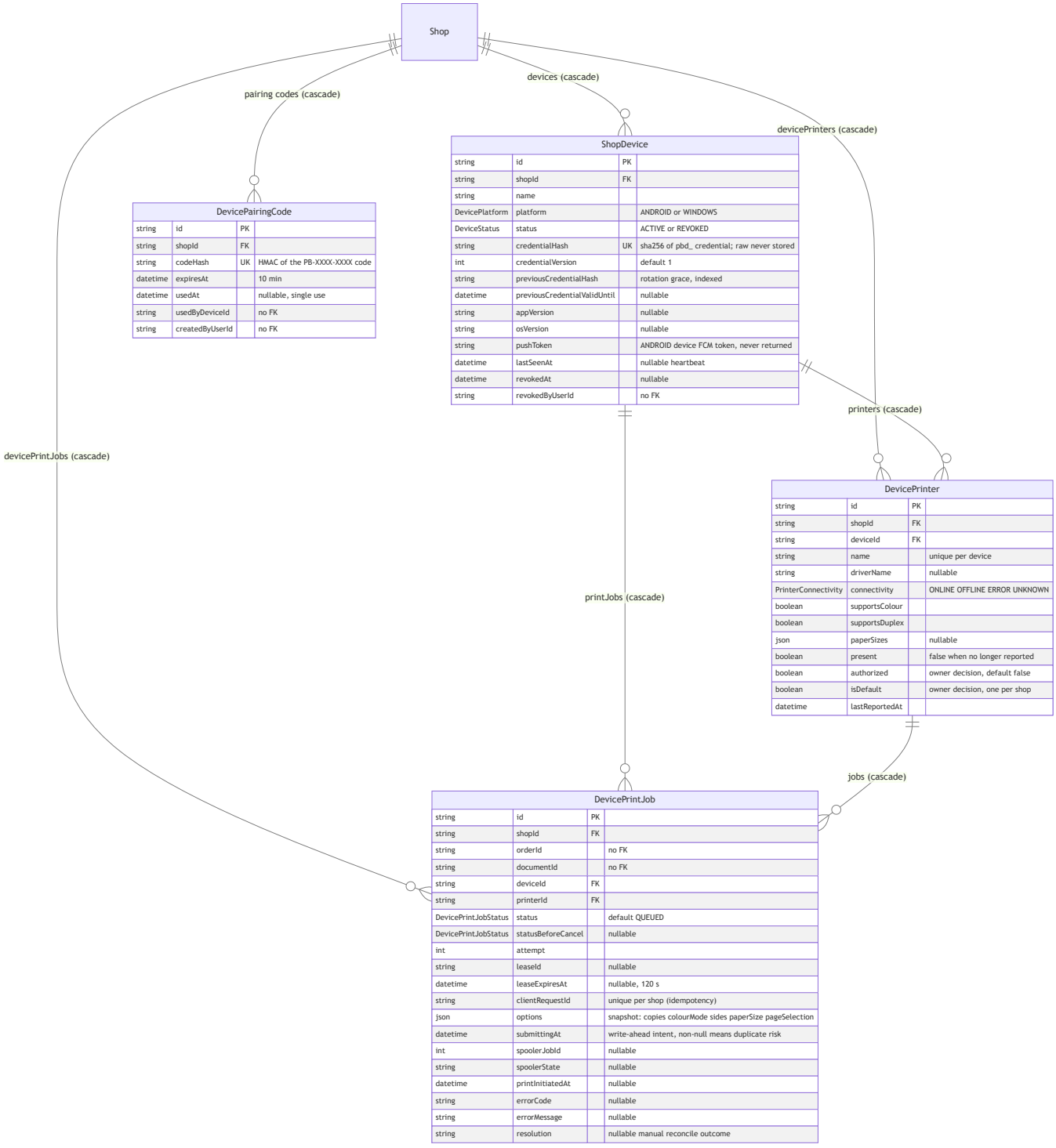
Privacy-relevant facts

- `objectKey`, bucket names and storage URLs are **never** in any API response; the only document access is a short-lived signed URL (≤300 s and never later than `deleteAfter`/`expiresAt`).
- Orders and their metadata are **never deleted** by retention: only the file is. After deletion `document.fileName` is hidden from the customer tracking response.
- `DeletionReceipt`, `CustomerSession`, `PrintJob` are not used by any code path today.

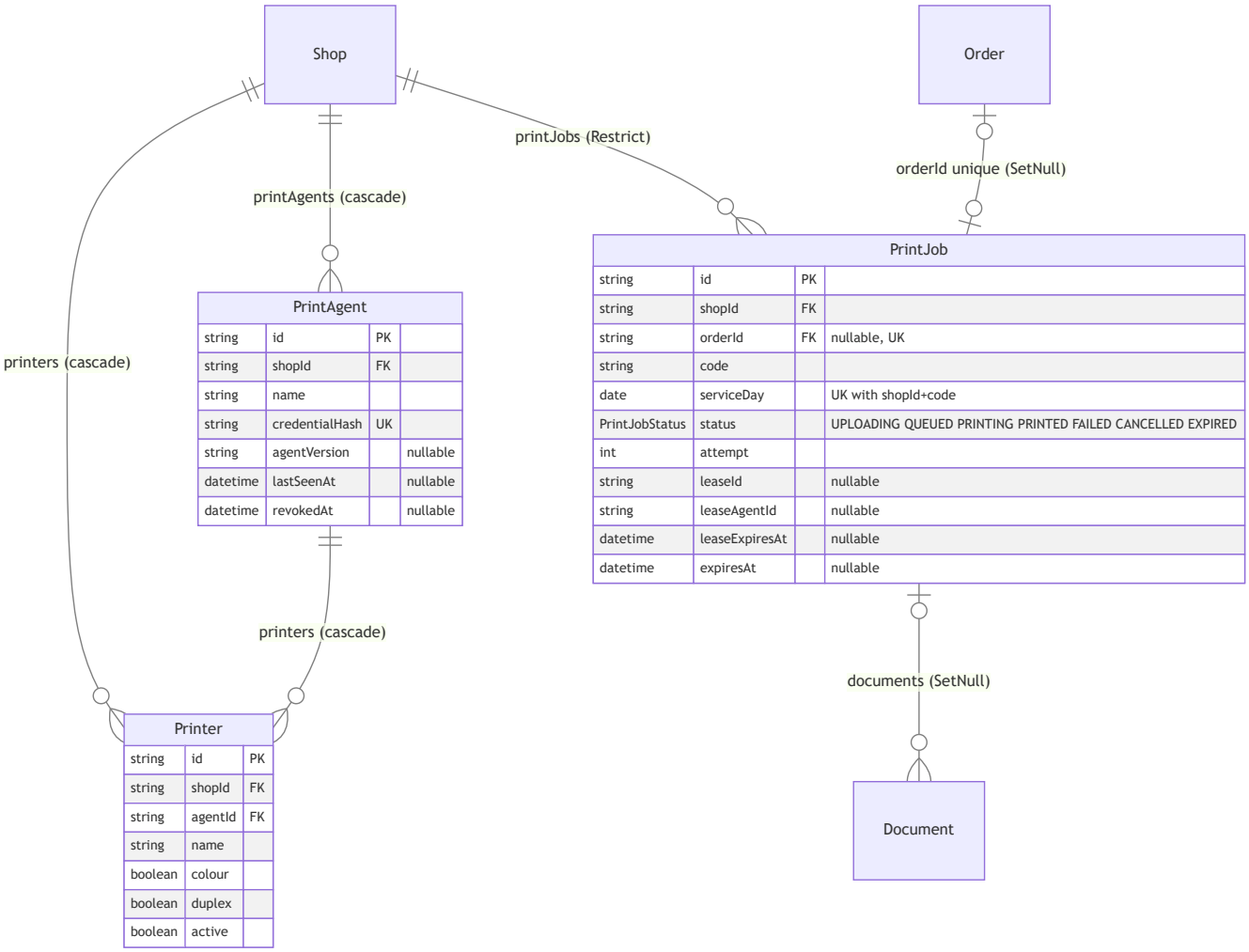
ER: devices, printers and print jobs

Two parallel model families exist. **Only the first is live:**

Family	Models	Status on `main`
Device foundation + Windows agent	`ShopDevice`, `DevicePairingCode`, `DevicePrinter`, `DevicePrintJob`	**In use** (pairing, heartbeat, printers, claim/progress, shop queue)
Target architecture	`PrintAgent`, `Printer`, `PrintJob`	**Schema only**: no route/worker touches them



Schema-only target tables (not used):

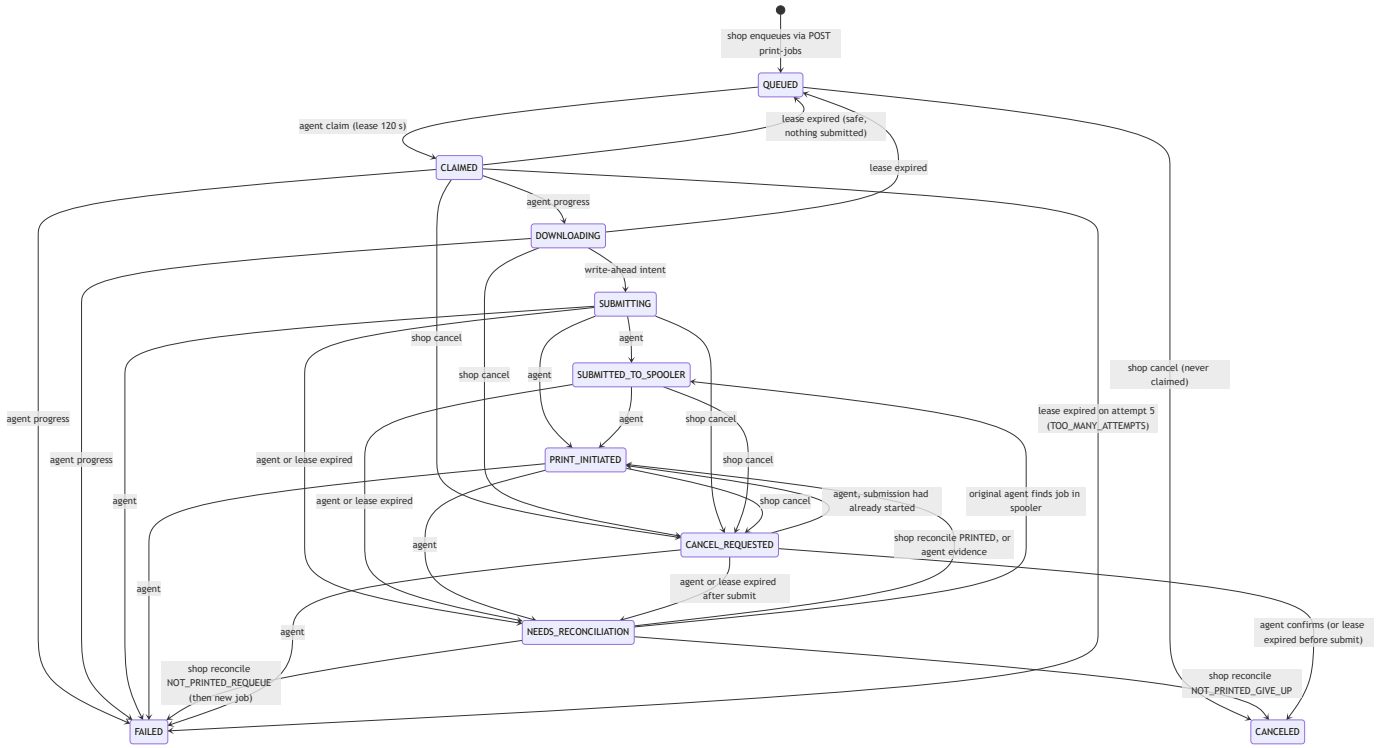


Constraints

Table	Unique	Foreign keys (onDelete)	Indexes
ShopDevice	`credentialHash`	`shopId → Shop` (Cascade)	`(shopId, status)`, `previousCredentialHash`
DevicePairingCode	`codeHash`	`shopId → Shop` (Cascade)	`(shopId, expiresAt)`
DevicePrinter	`(deviceId, name)`	`shopId → Shop`, `deviceId → ShopDevice` (Cascade)	`(shopId, isDefault)`
DevicePrintJob	`(shopId, clientRequestId)`	`shopId → Shop`, `deviceId → ShopDevice`, `printerId → DevicePrinter` (all Cascade)	`(shopId, orderId)`, `(deviceId, status)`, `(shopId, status, createdAt)`
PrintAgent / Printer / PrintJob	`credentialHash` / none / `orderId`, `(shopId, serviceDay, code)`	see diagram	

Print-job state machine (`DevicePrintJobStatus`)

There is **deliberately no "PRINTED" state**: neither the spooler nor the platform can confirm paper. `PRINT_INITIATED` means "the spooler accepted the job and began processing it". Any job that may have reached the spooler is never auto re-queued: ambiguity ends in `NEEDS_RECONCILIATION` and a human decides.



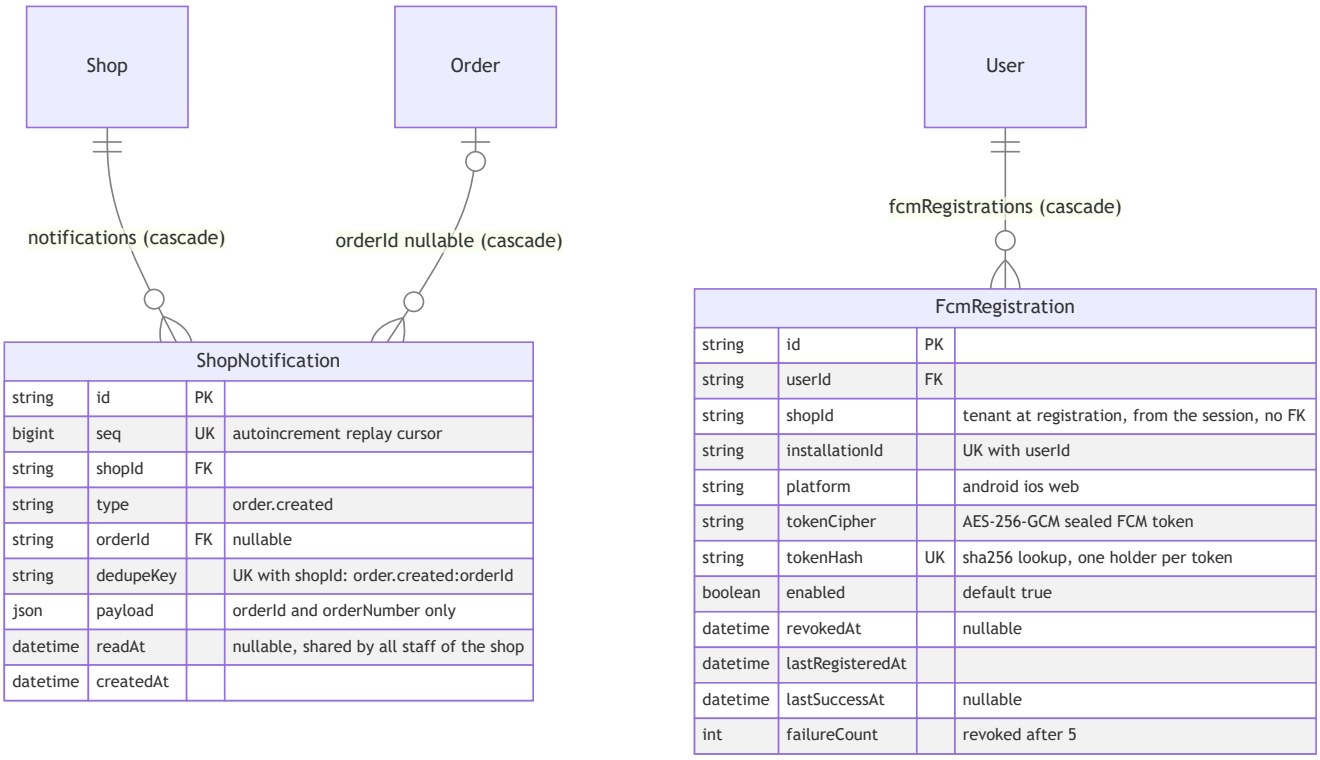
Constants (from `domain/print-jobs.ts`): lease 120 s, max 5 attempts (then `FAILED TOO_MANY_ATTEMPTS`), document download URL ≤120 s, one in-flight job per device, blocking statuses (`QUEUED, CLAIMED, DOWNLOADING, SUBMITTING, SUBMITTED_TO_SPOOLER, CANCEL_REQUESTED, NEEDS_RECONCILIATION`) block another job for the same order (`409 ACTIVE_PRINT_JOB_EXISTS`).

Android relevance

An **Android phone running the owner app** uses a *user* bearer token and never creates `ShopDevice` rows. `ShopDevice.platform = ANDROID` exists for a **paired companion device** (pairing code → `pbd_` credential → `/device/*`). The print queue (`DevicePrinter` / `DevicePrintJob`) is Windows-only: Android devices get `409 CONFLICT` on printer/print-job device routes.

ER: notifications (in-app inbox, SSE, FCM)

Models: `ShopNotification` (durable inbox/outbox) and `FcmRegistration` (push targets). Also touching: `ShopDevice.pushToken` (device-credential push for paired Android devices, a separate mechanism).



ShopNotification			
string	id	PK	
bigint	seq	UK	autoincrement replay cursor
string	shopId	FK	
string	type		order.created
string	orderId	FK	nullable
string	dedupeKey		UK with shopId: order.created:orderId
json	payload		orderId and orderNumber only
datetime	readAt		nullable, shared by all staff of the shop
datetime	createdAt		

FcmRegistration			
string	id	PK	
string	userId	FK	
string	shopId		tenant at registration, from the session, no FK
string	installationId		UK with userId
string	platform		android ios web
string	tokenCipher		AES-256-GCM sealed FCM token
string	tokenHash	UK	sha256 lookup, one holder per token
boolean	enabled		default true
datetime	revokedAt		nullable
datetime	lastRegisteredAt		
datetime	lastSuccessAt		nullable
int	failureCount		revoked after 5

Constraints

Table	Unique	Foreign keys (onDelete)	Indexes
ShopNotification	`seq`, `(shopId, dedupeKey)`	`shopId → Shop` (Cascade), `orderId → Order` (Cascade)	`(shopId, seq)`, `(shopId, readAt)`, `createdAt`
FcmRegistration	`tokenHash`, `(userId, installationId)`	`userId → User` (Cascade)	`shopId`

Delivery model

1. `POST /public/shops/:slug/orders` writes the `Order` **and** its `ShopNotification` row in the **same transaction** (transactional outbox; `dedupeKey` makes it idempotent). Committed order ⇔ committed notification.
2. After commit, three best-effort signals fan out: (a) the SSE hub (`notification.created` + `order.created`) for open `/shop/events` streams, (b) **FCM push** to the shop owners' registered installations (payload `{ type: "order.new", orderId }` plus a generic title: no file names, customer data, URLs or tokens), (c) the device notifier for paired Android devices (`NEW_PRINT_REQUEST`).
3. **The database is the source of truth.** A client that missed SSE/push recovers with `GET /shop/notifications?after=<lastSeq>` (and `GET /shop/orders`). Notifications are hidden after `NOTIFICATION_RETENTION_DAYS` (14) and swept by the worker.

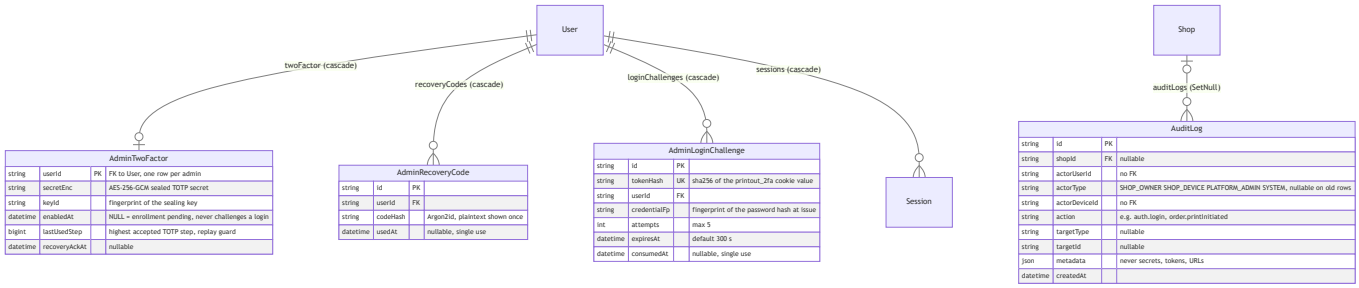
FCM rules

- Registration is **per (user, installation)**: `PUT /users/me/fcm-token` upserts; the same token presented under another account/installation revokes the older registration.

- The token is stored **only** encrypted (`tokenCipher`) plus a SHA-256 `tokenHash`; it is never returned by any API, never logged and never put in audit metadata.
- Push is sent only to registrations of **this shop** whose user is still a `SHOP_OWNER` of that shop; permanent FCM errors (`registration-token-not-registered`, `invalid-registration-token`, `invalid-argument`) revoke immediately; transient failures count up to `FCM_FAILURE_REVOKE_AFTER` (5). Registrations unused for `FCM_STALE_DAYS` (60) are ignored and swept.
- Logout / logout-all / refresh-family revocation removes the installation's registration (logout-all can keep push with `keepPush: true`).
- Whether push is actually enabled in production (`FCM_PUSH_ENABLED`, default **false**) and whether `FIREBASE_PROJECT_ID` is set is **Requires confirmation** (production env is not in the repository).

ER: admin and security

Models: `AdminTwoFactor`, `AdminRecoveryCode`, `AdminLoginChallenge`, `AuditLog` (+ `User`, `Shop`, `Session` for context).



Constraints

Table	Unique	Foreign keys (onDelete)	Indexes
AdminTwoFactor	`userId` (PK)	`userId → User` (Cascade)	
AdminRecoveryCode	none	`userId → User` (Cascade)	`(userId, usedAt)`
AdminLoginChallenge	`tokenHash`	`userId → User` (Cascade)	`userId`, `expiresAt`
AuditLog	none	`shopId → Shop` (SetNull)	`(shopId, createdAt)`, `createdAt`

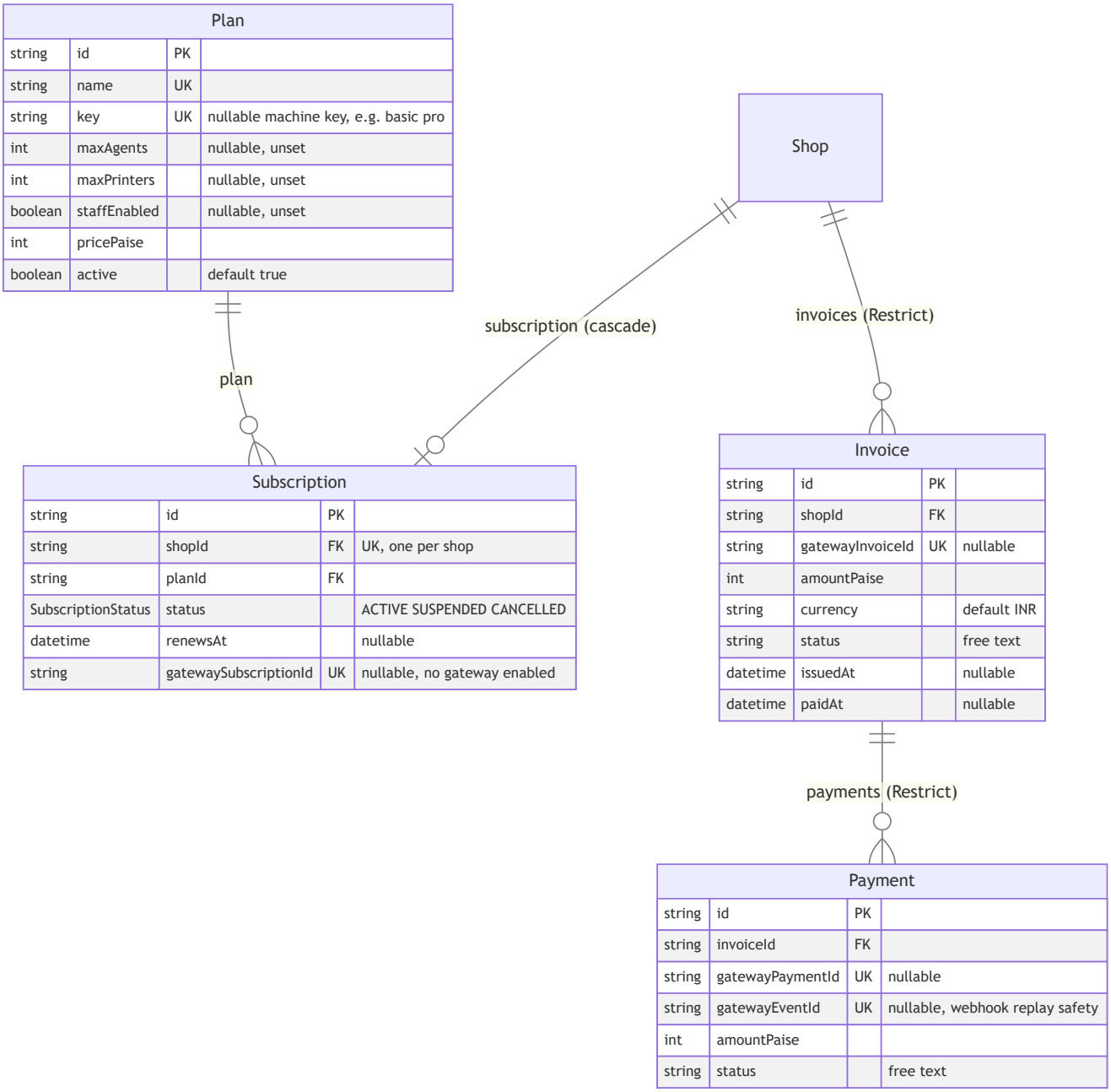
Security design recorded in the schema

- **Admin boundary:** `Role.PLATFORM_ADMIN` vs `SHOP_OWNER` on `User.role`. Admin routes sit behind `adminGuard` (role check + CSRF on mutations). Admins have no `shopId`; they can read **metadata only**: the admin API never returns filenames, object keys, tracking tokens or document URLs.
- **2FA (TOTP):** an enrolled admin's `passwordHash` is stored with a `tfa1$` prefix ("rollback guard": an older API that does not understand 2FA fails closed instead of accepting the password alone). A password-verified sign-in only yields an `AdminLoginChallenge`, which is **not** a session; any route that does not look for the challenge cookie rejects it.
- **Platform admins can never sign in through the mobile `POST /auth`, Google, or phone paths** (`403 FORBIDDEN`): the Android app is for shop owners only.
- **AuditLog** rows are only ever created by application code (no route or worker updates or deletes them); every sensitive mutation writes a row inside its transaction. Metadata is explicitly free of secrets (phone and email are masked, e.g. `+91••••••••10` and `a***@gmail.com`).
- Suspension (`PUT /admin/shops/:id` with `status: SUSPENDED`) invalidates all `Session` rows of the shop's users and revokes their `RefreshToken`s in one transaction; devices of a suspended shop get `403 SHOP_SUSPENDED`.

ER: legacy billing (plans, subscriptions, invoices, payments)

The platform is free (PR #40): no payment is taken, there is no checkout and no customer-facing billing API. These tables are retained for compatibility and a future decision.

Model	Used by code?	How
`Plan`	Yes (admin only)	`GET/POST/PUT /admin/plans`
`Subscription`	Yes (admin + gate)	`GET /admin/subscriptions`, `PUT /admin/subscriptions/:shopId`; the **public-intake and device gate** treats *no subscription* or `ACTIVE` as allowed (`domain/eligibility.ts`)
`Invoice`	**No**	schema only
`Payment`	**No**	schema only



Constraints

Table	Unique	Foreign keys (onDelete)
Plan	`name`, `key`	none
Subscription	`shopId`, `gatewaySubscriptionId`	`shopId → Shop` (Cascade), `planId → Plan` (no action)
Invoice	`gatewayInvoiceId`	`shopId → Shop` (Restrict)
Payment	`gatewayPaymentId`, `gatewayEventId`	`invoiceId → Invoice` (Restrict)

Notes for the Android developer

- **Do not build billing UI.** `ShopProfile.status` (`standard`/`premium`) and `plan` in `GET /shops/me` are derived from `Subscription`/`Plan` and are `standard` / `null` for every self-onboarded shop.
- **Revenue in the app is estimated order value only** (`GET /shop/analytics*`): customer payments are cash at the counter and are not recorded anywhere. `Payment`/`Invoice` say nothing about customer orders.
- An admin can still create a subscription for a shop; a `SUSPENDED` or `CANCELLED` subscription makes the shop reject new customer intake (`409 SHOP_UNAVAILABLE`) and makes its paired devices answer `403 SUBSCRIPTION_INACTIVE`.

API inventory (all registered routes)

> Source of truth: the Fastify route registrations in `apps/api/src` on `origin/main` (commit `080c6bd`). Every row below was transcribed from code; the route list is cross-checked mechanically against a scan of the sources (see README). **113 registered routes** (111 under `/api/v1` + `/health` and `/ready`). **1 retired** (documented as deprecated, not active).

Conventions

Base URL (production): `https://printoutbuddy.tech/api/v1` (`/health` and `/ready` live at the root of the same host).

Success envelope: `{ "data": <payload> }`. Exceptions: `204 No Content` (no body), CSV exports, the SSE stream, and the raw upload/download bytes.

Error envelope (every non-2xx):

```
{ "error": { "code": "VALIDATION_ERROR", "message": "Request validation failed", "requestId": "<uuid>", "details": { } }
}
```

`code` is a stable machine value (see the list in android-integration-guide.md); `message` is human text and may change; `requestId` is also what support needs. `details` is present only for some errors (validation `fieldErrors`, `details.nextStep = "account_blocked"`, `details.maxBytes`, `details.nextOpenAt` for `SHOP_CLOSED`).

Unknown routes → `404 NOT_FOUND`. **Never cached:** every response carries `Cache-Control: no-store` unless a route sets otherwise.

Strict bodies: almost every request body is a strict schema: unknown keys → `400 VALIDATION_ERROR`. Do not send extra fields. (Exceptions: `POST /shops/onboarding` ignores unknown keys.)

Dates are ISO-8601 UTC strings. **Money** is integer **paise** (`pricePerSheetPaise`, `totalPaise`; currency always `INR`). **Ids** are opaque strings (cuid; document ids are 24-char random). Business days are **Asia/Kolkata**.

Authentication mechanisms

Key	Mechanism
None	Unauthenticated. Per-IP rate limit.
Owner session	Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Any session	Any signed-in user (owner or admin). Bearer, or cookie + `X-CSRF-Token` on non-GET.
Session or restricted grant	A full session (owner/admin) or the restricted onboarding principal (verified Google/phone identity with no account yet).
Restricted onboarding grant	Android: the restricted `accessToken` returned by `POST /auth` when `nextStep = create_shop` (sent as `Authorization: Bearer`). Web: `printout_onboarding` cookie. Not a session.
Admin session	`PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Device credential	`Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Pending 2FA challenge cookie	`printout_2fa` cookie (path `/api/v1/auth/login/2fa`) issued by `/auth/login` for admins with 2FA. Not a session.
Signed URL	HMAC `exp` + `sig` query parameters issued by the API (local storage driver only).
Upload token	`token` query parameter from `uploads/initiate` (HMAC, 15 min, single use).
Session, or refresh token in body	Owner session (Bearer / cookie + CSRF), or a restricted grant, or `{ refreshToken }` in the body when the access token already expired.
Tracking token (path)	The unguessable `trackingToken` in the URL is the capability.

Owner routes accept **either** transport: Android uses `Authorization: Bearer` and is **exempt from CSRF**; a browser uses the cookie and must send `X-CSRF-Token` on every POST/PUT/PATCH/DELETE (`403 CSRF_INVALID` otherwise). The **Firebase ID token is never accepted by** `/shop/*`**,** `/users/*`**, or** `/shops/*`: it is only exchanged at `POST /auth` (or the web equivalents).

Rate limits (code defaults, per process, per minute)

Bucket	Limit	Key
Login/exchange/onboarding (`LOGIN_RATE_LIMIT_MAX`)	10	client IP
Refresh	30	client IP (+ 60 per refresh family)
Public (`PUBLIC_RATE_LIMIT_MAX`)	60 (uploads/initiate 20, upload PUT 30)	client IP
shopRead	600	session id
shopMutation	60	session id
status (transitions, print-now)	120	session id
documentAccess	60	session id
shopExport (CSV)	6	session id
sseConnect	30	session id
deviceRead / deviceMutation	300 / 120	device id
devicePair	10	client IP
adminRead / adminMutation	600 / 60	session id
IP ceiling in front of every guarded scope	3000 (= shopRead × 5)	client IP

Exceeded → `429 RATE_LIMITED` with a `Retry-After` header (seconds). Counters are in memory on the single API process. The production values of the `RATE_LIMIT_*` env overrides are **Requires confirmation** (they may differ from the defaults).

Status labels used below

Label	Meaning
Verified from code	Method, path, guards, schemas and error codes were read from the route source on `main`.
Deprecated	Registered but retired: it answers an error and must not be called.
Not implemented	Planned in docs/API_TARGET_SPEC.md but absent from the code: listed in Not implemented.
Requires confirmation	Behaviour depends on production configuration or an inner shape that is not pinned by a schema.

1. Authentication and onboarding

Method	Path	Auth	Android
POST	`/api/v1/auth`	None	YES
POST	`/api/v1/auth/refresh`	None	YES
POST	`/api/v1/auth/logout`	Session, or refresh token in body	YES
POST	`/api/v1/auth/logout-all`	Any session	YES
GET	`/api/v1/auth/session`	Any session	NO
POST	`/api/v1/auth/login`	None	NO
POST	`/api/v1/auth/login/2fa`	Pending 2FA challenge cookie	NO
POST	`/api/v1/auth/login/2fa/cancel`	None	NO
POST	`/api/v1/auth/google/session`	None	NO
POST	`/api/v1/auth/phone/session`	None	NO
GET	`/api/v1/auth/phone/config`	None	Optional
POST	`/api/v1/auth/firebase/exchange`	None	NO
POST	`/api/v1/shops/onboarding`	Restricted onboarding grant	YES

Method	Path	Auth	Android
GET	`/api/v1/users/me/identities`	Owner session	YES
POST	`/api/v1/users/me/identities/google`	Owner session	YES
POST	`/api/v1/users/me/identities/google/unlink`	Owner session	YES
GET	`/api/v1/shop/account/phone`	Owner session	Optional
POST	`/api/v1/shop/account/phone`	Owner session	Optional
POST	`/api/v1/shop/account/phone/unlink`	Owner session	Optional

POST /api/v1/auth

Unified MOBILE sign-in. Exchanges a Firebase Google/phone ID token (or email+password) for a Bearer access token + rotating refresh token.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Request JSON	`{ loginMethod: "otp" | "email" | "google"; phoneNumber?: string; firebaseIdToken?: string; email?: string; password?: string; fcmToken?: string; installationId: string; platform: "android" | "ios" | "web" }` `firebaseIdToken` (20-4096 chars) required for `otp` and `google`; `email` + `password` required for `email`. `installationId` = 16-64 chars `[A-Za-z0-9_-]`, random and stable per install. `fcmToken` is optional, registered only after authentication succeeds, and a push failure never fails sign-in. Body is strict (unknown keys → 400).
Response JSON	`{ data: MobileAuthResponse }`
HTTP status	success 200
Application errors	429 RATE_LIMITED, 400 VALIDATION_ERROR, 401 INVALID_CREDENTIALS (email), 401 GOOGLE_TOKEN_INVALID, 401 PHONE_TOKEN_INVALID, 403 FORBIDDEN (admin or non-owner), 403 ACCOUNT_BLOCKED (suspended shop, `details.nextStep=account_blocked`), 403 PHONE_REVERIFY_REQUIRED, 503 GOOGLE_LOGIN_UNAVAILABLE, 503 PHONE_LOGIN_UNAVAILABLE
Pagination	—
Idempotency	Not idempotent: each success revokes earlier token families of the same `installationId` and issues a new pair. A Firebase ID token can be exchanged once (replay protection on `auth_time`).
Rate limit	`LOGIN_RATE_LIMIT_MAX` = 10/min per IP; plus in-memory failed-login throttles (5 per IP+email / 15 min; account-wide 20).
Tables read	User, Shop, UserGoogleIdentity, UserPhoneIdentity, ShopSettings, Subscription, Plan, PricingRule, AuditLog
Tables written	Session, RefreshToken, PhoneOnboardingGrant (new identity), FcmRegistration (optional), UserGoogleIdentity/UserPhoneIdentity (lastAuthTime), User.lastLoginAt, AuditLog
Android	YES: the only Android sign-in endpoint.
Notes	A Google identity not linked to any account returns the **restricted pair** with `nextStep: "create_shop"` and `user.id: null`; use it only for `POST /shops/onboarding`, `GET /users/me` and logout.

POST /api/v1/auth/refresh

Rotate the refresh token: returns a new access token AND a new refresh token (single-use rotation with reuse detection).

Status	Verified from code

Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Request JSON	`{ refreshToken: string; deviceId: string }` `deviceId` must equal the `installationId` used at sign-in (16-64 chars). Body limit 4096 bytes; anything larger returns the same 401 as a forged token. Refresh token travels ONLY in the body.
Response JSON	`{ data: MobileAuthResponse }`
HTTP status	success 200
Application errors	401 REFRESH_TOKEN_INVALID, 401 REFRESH_TOKEN_EXPIRED, 409 REFRESH_CONFLICT (concurrent/just-rotated, retry after the first finishes), 403 ACCOUNT_BLOCKED, 403 FORBIDDEN, 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	—
Idempotency	Single-use. Re-presenting a rotated token within `REFRESH_REUSE_GRACE_SECONDS` (5) → 409 (nothing revoked); later → whole family revoked + 401. Always single-flight on the client.
Rate limit	30/min per IP (`LOGIN_RATE_LIMIT_MAX*3`), per-family budget 60, failure throttle per IP.
Tables read	RefreshToken, User, Shop, UserPhoneIdentity, UserGoogleIdentity
Tables written	RefreshToken (rotatedAt / new row), Session (old invalidated, new created), AuditLog
Android	YES
Notes	A restricted (onboarding) refresh token stays restricted until the identity is linked to a user; the next refresh after onboarding yields a normal pair. 15 min access / 30 day refresh are the code defaults (`ACCESS_TOKEN_TTL_MINUTES`, `REFRESH_TOKEN_TTL_DAYS`); trust `expiresIn` / `refreshTokenExpiresIn` in the response.

POST /api/v1/auth/logout

End this session. For a mobile session also revokes its refresh family and the installation's FCM registration.

Status	Verified from code
Authentication	Session, or refresh token in body: Owner session (Bearer / cookie + CSRF), or a restricted grant, or `{ refreshToken }` in the body when the access token already expired.
Role / scope	any
Request JSON	`{ refreshToken?: string }` Body optional. An expired-access-token app can still sign out by sending `{ refreshToken }` with no Authorization header (idempotent: unknown/revoked token → 204).
Response JSON	no body
HTTP status	success 204
Application errors	401 UNAUTHORIZED (no valid credential at all), 403 CSRF_INVALID (cookie), 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation 60/min per session.
Tables read	Session, RefreshToken, PhoneOnboardingGrant
Tables written	Session.invalidatedAt, RefreshToken.revokedAt, FcmRegistration (revoked), PhoneOnboardingGrant (deleted), AuditLog
Android	YES

POST /api/v1/auth/logout-all

Sign out of every device: ends all sessions and refresh families of the caller; removes push registrations unless `keepPush`.

Status	Verified from code
Authentication	Any session: Any signed-in user (owner or admin). Bearer, or cookie + `X-CSRF-Token` on non-GET.
Role / scope	any
Request JSON	`{ keepPush?: boolean }`
Response JSON	no body
HTTP status	success 204
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation 60/min per session.
Tables read	Session, RefreshToken
Tables written	Session, RefreshToken, FcmRegistration, AuditLog
Android	YES

GET /api/v1/auth/session

Web session probe: returns the web-shaped session payload including a fresh CSRF token.

Status	Verified from code
Authentication	Any session: Any signed-in user (owner or admin). Bearer, or cookie + `X-CSRF-Token` on non-GET.
Role / scope	any
Request body	none
Response JSON	`{ data: { user: { id: string; displayName: string; role: "SHOP_OWNER" | "PLATFORM_ADMIN" }; shop: { id: string; slug: string; displayName: string } | null; csrfToken: string; retentionMinutes: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe (GET).
Rate limit	shopRead 600/min per session.
Tables read	Session, User, Shop
Tables written	—
Android	NO: use `GET /users/me` (richer, same auth).

POST /api/v1/auth/login

WEB email+password sign-in (cookie session + CSRF token). Admins with 2FA get `{ twoFactorRequired: true }` plus a pending-challenge cookie instead of a session.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Request JSON	`{ email: string; password: string }`
Response JSON	`{ data: { user: { id: string; displayName: string; role: "SHOP_OWNER" | "PLATFORM_ADMIN" }; shop: { id: string; slug: string; displayName: string } | null; csrfToken: string; retentionMinutes: number } }`
HTTP status	success 200
Application errors	429 RATE_LIMITED, 400 VALIDATION_ERROR, 401 INVALID_CREDENTIALS, 403 SHOP_SUSPENDED, 503 TWO_FACTOR_UNAVAILABLE
Pagination	—
Idempotency	Not idempotent: rotates the cookie session.
Rate limit	10/min per IP + failed-login throttle.
Tables read	User, Shop, AdminTwoFactor
Tables written	Session, AdminLoginChallenge, User.lastLoginAt, AuditLog
Android	NO: sets a cookie and returns a CSRF token. Use `POST /auth` with `loginMethod: "email"`.

POST /api/v1/auth/login/2fa

Second step of an admin sign-in (TOTP or recovery code).

Status	Verified from code
Authentication	Pending 2FA challenge cookie: `printout_2fa` cookie (path `/api/v1/auth/login/2fa`) issued by `/auth/login` for admins with 2FA. Not a session.
Role / scope	PLATFORM_ADMIN
Request JSON	`{ method: "totp" | "recovery"; code: string }`
Response JSON	`{ data: { user: any; shop: any | null; csrfToken: string; retentionMinutes: number; recoveryCodesRemaining?: number } }`
HTTP status	success 200
Application errors	429 RATE_LIMITED, 400 VALIDATION_ERROR, 401 INVALID_TWO_FACTOR_CODE, 401 TWO_FACTOR_CHALLENGE_INVALID, 503 TWO_FACTOR_UNAVAILABLE
Pagination	—
Idempotency	Challenge is single-use; each attempt is counted before the code is checked (max 5).
Rate limit	10/min per IP.
Tables read	AdminLoginChallenge, AdminTwoFactor, AdminRecoveryCode, User
Tables written	AdminLoginChallenge, AdminTwoFactor.lastUsedStep, AdminRecoveryCode.usedAt, Session, AuditLog
Android	NO (admin web only)

POST /api/v1/auth/login/2fa/cancel

Abandon a pending admin 2FA challenge held by this browser.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Request body	none
Response JSON	no body
HTTP status	success 204
Application errors	—
Pagination	—
Idempotency	Idempotent.
Rate limit	none beyond global.
Tables read	AdminLoginChallenge
Tables written	AdminLoginChallenge.consumedAt
Android	NO (admin web only)

POST /api/v1/auth/google/session

WEB Google sign-in: verified Firebase Google ID token → session cookie, or restricted onboarding cookie when no account is linked.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Request JSON	`{ idToken: string }`
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	429 RATE_LIMITED, 400 VALIDATION_ERROR, 401 GOOGLE_TOKEN_INVALID, 403 FORBIDDEN (bad Origin / admin), 403 SHOP_SUSPENDED, 503 GOOGLE_LOGIN_UNAVAILABLE
Pagination	—
Idempotency	Token single-use (`auth_time` replay check).
Rate limit	10/min per IP + failure throttle.
Tables read	UserGoogleIdentity, User, Shop
Tables written	Session, PhoneOnboardingGrant, UserGoogleIdentity.lastAuthTime, AuditLog
Android	NO: use `POST /auth` (`loginMethod: "google"`).

POST /api/v1/auth/phone/session

WEB phone-OTP sign-in for an already-linked owner (cookie session). Does NOT create onboarding grants.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—

Request JSON	`{ idToken: string }`
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	429 RATE_LIMITED, 400 VALIDATION_ERROR, 401 PHONE_TOKEN_INVALID, 403 PHONE_NOT_LINKED, 403 PHONE_REVERIFY_REQUIRED, 403 SHOP_SUSPENDED, 503 PHONE_LOGIN_UNAVAILABLE
Pagination	—
Idempotency	Token single-use.
Rate limit	10/min per IP.
Tables read	UserPhoneIdentity, User, Shop
Tables written	Session, UserPhoneIdentity.lastAuthTime, AuditLog
Android	NO: use `POST /auth` (`loginMethod: "otp"`).

GET /api/v1/auth/phone/config

Phone sign-in availability and allowed country codes.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Request body	none
Response JSON	`{ data: { enabled: boolean; allowedCallingCodes: string[]; defaultCallingCode: "+91"; resendCooldownSeconds: number } }`
HTTP status	success 200
Application errors	—
Pagination	—
Idempotency	Safe.
Rate limit	none beyond global.
Tables read	—
Tables written	—
Android	Optional (feature detection)

POST /api/v1/auth/firebase/exchange

Phone-OTP token exchange for web (cookie) or mobile (`client: "mobile"`, flat Bearer pair). Kept for the web app.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Request JSON	`{ idToken: string; client?: "web" | "mobile"; deviceId?: string }` `deviceId` required when `client = mobile`. `client` defaults to `web`.
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	429 RATE_LIMITED, 400 VALIDATION_ERROR, 401 PHONE_TOKEN_INVALID, 403 PHONE_REVERIFY_REQUIRED, 403 ACCOUNT_BLOCKED / SHOP_SUSPENDED, 503 PHONE_LOGIN_UNAVAILABLE

Pagination	—
Idempotency	Token single-use.
Rate limit	10/min per IP.
Tables read	UserPhoneIdentity, User, Shop
Tables written	Session, RefreshToken, PhoneOnboardingGrant, AuditLog
Android	NO: `POST /auth` is the documented mobile entry (same logic).

POST /api/v1/shops/onboarding

Create the caller's shop (first-time Google/phone user). Creates User + Shop + ShopSettings + verified identity in ONE transaction and returns a full session/credential pair.

Status	Verified from code
Authentication	Restricted onboarding grant: Android: the restricted `accessToken` returned by `POST /auth` when `nextStep = create_shop` (sent as `Authorization: Bearer`). Web: `printout_onboarding` cookie. Not a session.
Role / scope	restricted principal
Request JSON	`{ ownerName: string; email?: string; shopName: string; address: string; pincode: string; client?: "web" | "mobile"; deviceId?: string }` `ownerName` 1-100; `shopName` 2-100; `address` 5-300; `pincode` `^[1-9][0-9]{5}$`. Google principals: `email` omitted or equal to the verified email; phone principals: `email` required. `client: "mobile"` requires `deviceId` (= the `installationId`). Unknown keys are NOT rejected (body is not strict).
Response JSON	`{ data: MobileAuthResponse }`
HTTP status	success 201 (created) / 200 (idempotent repeat)
Application errors	401 UNAUTHORIZED, 400 VALIDATION_ERROR, 403 SELF_ONBOARDING_DISABLED (`SHOP_SELF_ONBOARDING_ENABLED=false`), 409 ALREADY_ONBOARDED, 409 EMAIL_UNAVAILABLE, 409 GOOGLE_IDENTITY_UNAVAILABLE, 409 PHONE_UNAVAILABLE, 409 CONFLICT (slug exhausted), 429 RATE_LIMITED
Pagination	—
Idempotency	Naturally idempotent per verified identity: unique constraints yield one shop; a retry within a 2-minute grace window returns the same shop (200).
Rate limit	10/min per IP + per (firebase uid, IP) failure budget.
Tables read	PhoneOnboardingGrant, UserGoogleIdentity, UserPhoneIdentity, User
Tables written	Shop, ShopSettings, User, UserGoogleIdentity or UserPhoneIdentity, Session, RefreshToken, AuditLog
Android	YES (first-time Google users)
Notes	**Code vs docs:** the code (since PR #40) creates the shop with `status: ACTIVE` and `acceptsOrders: true` (QR immediately ready; no plan/subscription). `docs/MOBILE_AUTH_API.md` and `docs/API_CONTRACT.md` still describe `PILOT` + `acceptsOrders=false`; those passages are stale. The new owner has `passwordSet=false` (no password sign-in).

GET /api/v1/users/me/identities

Which sign-in methods are linked (masked values only).

Status	Verified from code

Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none
Response JSON	`{ data: { password: { set: boolean }; phone: { linked: boolean; phoneMasked?: string }; google: { linked: boolean; emailMasked?: string }; googleAvailable: boolean } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	shopRead 600/min.
Tables read	User, UserPhoneIdentity, UserGoogleIdentity
Tables written	—
Android	YES (Settings)

POST /api/v1/users/me/identities/google

Link (or replace) a Google identity on the signed-in owner. Requires re-authentication AND a fresh Google token.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ firebaseIdToken: string; reauth: { currentPassword?: string; firebaseIdToken?: string } }` `reauth` must contain EXACTLY ONE of `currentPassword` or a fresh `firebaseIdToken` of an identity already linked (for password-less accounts).
Response JSON	`{ data: { linked: true; emailMasked: string; replaced: boolean; otherSessionsEnded: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 403 INVALID_CURRENT_PASSWORD, 403 REAUTH_FAILED, 401 GOOGLE_TOKEN_INVALID, 409 GOOGLE_IDENTITY_UNAVAILABLE, 503 GOOGLE_LOGIN_UNAVAILABLE
Pagination	—
Idempotency	Token single-use; linking the same identity twice with a stale token → 401 GOOGLE_TOKEN_INVALID.
Rate limit	shopMutation 60/min + `acctchg` password budget.
Tables read	User, UserGoogleIdentity, UserPhoneIdentity
Tables written	UserGoogleIdentity, Session (others ended), RefreshToken (others revoked), AuditLog
Android	YES (Settings)
Notes	Ends the OTHER sessions/refresh families of the user; the current session stays.

POST /api/v1/users/me/identities/google/unlink

Unlink Google (refused if it is the only remaining sign-in method).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ reauth: { currentPassword?: string; firebaseIdToken?: string } }`
Response JSON	`{ data: { linked: false; otherSessionsEnded: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 NOT_FOUND, 409 LAST_SIGNIN_METHOD, 403 INVALID_CURRENT_PASSWORD, 403 REAUTH_FAILED
Pagination	—
Idempotency	Second call → 404.
Rate limit	shopMutation 60/min.
Tables read	User, UserGoogleIdentity, UserPhoneIdentity
Tables written	UserGoogleIdentity (deleted), Session, RefreshToken, AuditLog
Android	YES (Settings)

GET /api/v1/shop/account/phone

Linked-phone status.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none
Response JSON	`{ data: { linked: boolean; available: boolean; phoneMasked?: string; linkedAt?: string(ISO-8601); lastSignInAt?: string(ISO-8601) | null } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	shopRead.
Tables read	UserPhoneIdentity
Tables written	—
Android	Optional

POST /api/v1/shop/account/phone

Link or replace the owner's verified phone (fresh OTP token + proof of ownership).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ idToken: string; currentPassword?: string; reauthIdToken?: string }` Exactly one of `currentPassword` / `reauthIdToken`.
Response JSON	`{ data: { linked: true; phoneMasked: string; replaced: boolean; otherSessionsEnded: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 403 INVALID_CURRENT_PASSWORD, 403 REAUTH_FAILED, 401 PHONE_TOKEN_INVALID, 409 PHONE_UNAVAILABLE, 503 PHONE_LOGIN_UNAVAILABLE
Pagination	—
Idempotency	Token single-use.
Rate limit	shopMutation 60/min.
Tables read	User, UserPhoneIdentity
Tables written	UserPhoneIdentity, Session, RefreshToken, AuditLog
Android	Optional

POST /api/v1/shop/account/phone/unlink

Unlink the phone (refused if it is the last sign-in method).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ currentPassword?: string; reauthIdToken?: string }`
Response JSON	`{ data: { linked: false; otherSessionsEnded: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 NOT_FOUND, 409 LAST_SIGNIN_METHOD, 403 INVALID_CURRENT_PASSWORD
Pagination	—
Idempotency	Second call → 404.
Rate limit	shopMutation 60/min.
Tables read	User, UserPhoneIdentity, UserGoogleIdentity

Tables written	UserPhoneIdentity (deleted), Session, RefreshToken, AuditLog
Android	Optional

2. User profile

Method	Path	Auth	Android
GET	`/api/v1/users/me`	Session or restricted grant	YES
PATCH	`/api/v1/users/me`	Any session	YES
GET	`/api/v1/shops/me`	Owner session	YES
PATCH	`/api/v1/shops/me`	Owner session	YES
GET	`/api/v1/shops/me/qr`	Owner session	YES

GET /api/v1/users/me

The caller's profile + shop + `nextStep`. Works for suspended shops (returns `account_blocked`) and for the restricted onboarding principal.

Status	Verified from code
Authentication	Session or restricted grant: A full session (owner/admin) or the restricted onboarding principal (verified Google/phone identity with no account yet).
Role / scope	any / restricted
Request body	none
Response JSON	`{ data: ProfileView }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	shopRead 600/min.
Tables read	User, UserPhoneIdentity, UserGoogleIdentity, Shop, ShopSettings, Subscription, Plan, PricingRule, AuditLog, PhoneOnboardingGrant
Tables written	—
Android	YES: call at app start / after refresh to drive navigation by `nextStep`.

PATCH /api/v1/users/me

Update own display name; changing email also needs the current password and ends other sessions.

Status	Verified from code
Authentication	Any session: Any signed-in user (owner or admin). Bearer, or cookie + `X-CSRF-Token` on non-GET.
Role / scope	any
Request JSON	`{ name?: string; email?: string; currentPassword?: string }` At least one of `name` / `email`. `email` change: owner with a real password only (403 EMAIL_CHANGE_NOT_ALLOWED if `passwordSet=false`).
Response JSON	`{ data: ProfileView }`
HTTP status	success 200

Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 403 INVALID_CURRENT_PASSWORD, 403 EMAIL_CHANGE_NOT_ALLOWED, 409 EMAIL_UNAVAILABLE
Pagination	—
Idempotency	Idempotent for the same values.
Rate limit	shopMutation.
Tables read	User
Tables written	User, Session (others ended on email change), RefreshToken, AuditLog
Android	YES

GET /api/v1/shops/me

The owner's shop(s): `shops` has zero or one element (single-shop data model).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none
Response JSON	`{ data: { shops: ShopProfile[]; shop: ShopProfile | null; nextStep: string; isShopRegisteredAtLeastOne: boolean; showQr: boolean } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	shopRead.
Tables read	User, Shop, ShopSettings, Subscription, Plan, PricingRule, AuditLog
Tables written	—
Android	YES (includes `rateInfo` = active pricing rules)

PATCH /api/v1/shops/me

Edit shop name, address, PIN code.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ shopName?: string; address?: string; pincode?: string }` At least one field; strict body.
Response JSON	`{ data: { shops: ShopProfile[]; shop: ShopProfile | null; nextStep: string; isShopRegisteredAtLeastOne: boolean; showQr: boolean } }`

HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Idempotent for the same values.
Rate limit	shopMutation.
Tables read	Shop
Tables written	Shop, AuditLog
Android	YES

GET /api/v1/shops/me/qr

QR readiness and the public URL to encode. The QR is virtual (`<WEB_ORIGIN>/p/<slug>`), never stored.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	query `shopId?` (must equal the caller's own shop, else 404)
Request body	none
Response JSON	`{ data: { ready: boolean; publicUrl: string | null; qrCodeUrl: string | null; reason: "shop_pending_activation" | "shop_suspended" | "shop_inactive" | "invalid_slug" | null } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 404 NOT_FOUND
Pagination	—
Idempotency	Safe.
Rate limit	shopRead.
Tables read	Shop
Tables written	—
Android	YES (render QR in-app)

3. Shop settings

Method	Path	Auth	Android
GET	`/api/v1/shop/settings`	Owner session	YES
PUT	`/api/v1/shop/settings`	Owner session	YES
GET	`/api/v1/shop/hours`	Owner session	YES
PUT	`/api/v1/shop/hours`	Owner session	YES
PUT	`/api/v1/shop/hours/temporary-closure`	Owner session	YES
DELETE	`/api/v1/shop/hours/temporary-closure`	Owner session	YES

Method	Path	Auth	Android
POST	`/api/v1/shop/hours/reopen`	Owner session	YES
DELETE	`/api/v1/shop/hours/reopen`	Owner session	YES
GET	`/api/v1/shop/qr`	Owner session	Prefer `GET /shops/me/qr`
GET	`/api/v1/shop/devices`	Owner session	YES

GET /api/v1/shop/settings

Shop name/contact/brand colour/order acceptance.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none
Response JSON	`{ data: ShopSettings }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	shopRead.
Tables read	Shop, ShopSettings
Tables written	—
Android	YES

PUT /api/v1/shop/settings

Partial update (PUT with partial body): any subset of the fields, at least one.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ displayName?: string; address?: string | null; publicContact?: string | null; brandColor?: string | null; acceptsOrders?: boolean }` `brandColor` = `#RRGGBB`. Strict body.
Response JSON	`{ data: ShopSettings }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—

Idempotency	Idempotent (same body → same state).
Rate limit	shopMutation.
Tables read	Shop, ShopSettings
Tables written	Shop, ShopSettings, AuditLog
Android	YES

GET /api/v1/shop/hours

Operating hours, closures and the server-computed availability.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none
Response JSON	`{ data: { timezone: string; weeklyHours: any | null; specialClosures: { date: string; reason: string | null }[]; afterHoursPolicy: "ACCEPT" | "BLOCK"; temporaryClosure: any | null; reopenUntil: string | null; updatedAt: string(ISO-8601) | null; availability: any } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	shopRead.
Tables read	ShopSettings
Tables written	—
Android	YES
Notes	`weeklyHours` = `{ mon..sun: [{ start: "HH:MM", end: "HH:MM|24:00" }] }` (max 6 intervals/day, end < start crosses midnight). `availability.state` = open / closed / unknown.

PUT /api/v1/shop/hours

Update timezone / weekly hours / special closures / after-hours policy (partial).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ timezone?: string; weeklyHours?: any | null; specialClosures?: { date: string; reason?: string | null }[]; afterHoursPolicy?: "ACCEPT" | "BLOCK" }` `weeklyHours: null` removes the schedule; omitted = untouched. Max 366 closures.

Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation.
Tables read	ShopSettings
Tables written	Shop.updatedAt (row lock), ShopSettings, AuditLog
Android	YES

PUT /api/v1/shop/hours/temporary-closure

Start a temporary closure (also ends any emergency reopen).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ reason?: string | null; until?: string(ISO-8601) | null }` `until` must be in the future and within one year; null = until cleared.
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Idempotent (restarts `startedAt`).
Rate limit	shopMutation.
Tables read	ShopSettings
Tables written	ShopSettings, AuditLog
Android	YES

DELETE /api/v1/shop/hours/temporary-closure

Clear the temporary closure.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none

Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation.
Tables read	ShopSettings
Tables written	ShopSettings, AuditLog
Android	YES

POST /api/v1/shop/hours/reopen

Emergency reopen: counts as open for N minutes regardless of schedule/closures.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ minutes: number }` 5-1440.
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Not idempotent (resets the deadline).
Rate limit	shopMutation.
Tables read	ShopSettings
Tables written	ShopSettings.reopenUntil, AuditLog
Android	YES

DELETE /api/v1/shop/hours/reopen

Cancel an emergency reopen.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none

Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation.
Tables read	ShopSettings
Tables written	ShopSettings, AuditLog
Android	YES

GET /api/v1/shop/qr

Legacy QR payload (always returns a URL even if the shop is not live).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none
Response JSON	`{ data: { publicUrl: string; slug: string; shopName: string } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	shopRead.
Tables read	Shop
Tables written	—
Android	Prefer `GET /shops/me/qr` (readiness-aware).

GET /api/v1/shop/devices

Paired devices (Windows agents / Android devices) with presence and printers.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none

Response JSON	`{ data: { devices: Device[] } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	None (all devices).
Idempotency	Safe.
Rate limit	shopRead.
Tables read	ShopDevice, DevicePrinter
Tables written	—
Android	YES (read-only management view)

4. Shop pricing and quotes

Method	Path	Auth	Android
GET	`/api/v1/shop/pricing-rules`	Owner session	YES
POST	`/api/v1/shop/pricing-rules`	Owner session	YES
PUT	`/api/v1/shop/pricing-rules/:id`	Owner session	YES
DELETE	`/api/v1/shop/pricing-rules/:id`	Owner session	YES
POST	`/api/v1/public/shops/:slug/quotes`	None	Customer flow only

GET /api/v1/shop/pricing-rules

All pricing rules (active and inactive), ordered by colourMode, sides.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none
Response JSON	`{ data: PricingRule[] }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	None (max 4 rules: bw/colour × single/duplex).
Idempotency	Safe.
Rate limit	shopRead 600/min.
Tables read	PricingRule
Tables written	—
Android	YES
Notes	Unique per `(shopId, paperSize, colourMode, sides)`; `paperSize` is always `"A4"`.

POST /api/v1/shop/pricing-rules

Create a rule.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ paperSize?: "A4"; colourMode: "bw" | "colour"; sides: "single" | "duplex"; pricePerSheetPaise: number; active?: boolean }` `pricePerSheetPaise` 1-1,000,000 (integer paise). `active` default true. Strict.
Response JSON	`{ data: PricingRule }`
HTTP status	success 201
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 409 DUPLICATE_PRICING_RULE
Pagination	—
Idempotency	Not idempotent; a repeat → 409 DUPLICATE_PRICING_RULE (treat as "already exists", then PUT).
Rate limit	shopMutation 60/min.
Tables read	PricingRule
Tables written	PricingRule, AuditLog
Android	YES

PUT /api/v1/shop/pricing-rules/:id

Update a rule (partial: any of the fields, at least one).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id` (rule id, 8-64 chars)
Request JSON	`{ paperSize?: "A4"; colourMode?: "bw" | "colour"; sides?: "single" | "duplex"; pricePerSheetPaise?: number; active?: boolean }`
Response JSON	`{ data: PricingRule }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 NOT_FOUND, 409 DUPLICATE_PRICING_RULE
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation.
Tables read	PricingRule
Tables written	PricingRule, AuditLog

Android	YES
Notes	No `If-Match`/version field: last write wins.

DELETE /api/v1/shop/pricing-rules/:id

Delete a rule. Existing orders keep their immutable `priceSnapshot`.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request body	none
Response JSON	no body
HTTP status	success 204
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 NOT_FOUND
Pagination	—
Idempotency	Second call → 404.
Rate limit	shopMutation.
Tables read	PricingRule
Tables written	PricingRule (deleted), AuditLog
Android	YES

POST /api/v1/public/shops/:slug/quotes

Server-priced quote for an uploaded document and print options. Returns a signed `quoteId` (valid `QUOTE_TTL_SECONDS`, default 600).

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Parameters	path `slug` (`^[a-z0-9-]{3,80}$`)
Request JSON	`{ documentId: string; printOptions: PrintOptions }` `copies` 1-1000; `pageSelection` `{mode:"all"}` or ≤50 `{from,to}` ranges. Strict.
Response JSON	`{ data: { quoteId: string; printOptions: PrintOptions; selectedPageCount: number; sheetsPerCopy: number; totalSheets: number; unitPricePaise: number; totalPaise: number; currency: "INR"; expiresAt: string(ISO-8601) } }`
HTTP status	success 200
Application errors	404 NOT_FOUND, 409 SHOP_UNAVAILABLE, 409 SHOP_CLOSED, 422 DOCUMENT_UNAVAILABLE, 422 NO_PRICING_RULE, 422 INVALID_PAGE_RANGE, 400 VALIDATION_ERROR, 429 RATE_LIMITED
Pagination	—

Idempotency	Safe/stateless (nothing is written). Sheets = duplex ? ceil(pages/2) : pages, × copies.
Rate limit	`PUBLIC_RATE_LIMIT_MAX` 60/min per IP.
Tables read	Shop, ShopSettings, Subscription, Document, PricingRule
Tables written	—
Android	Customer flow only (not needed by the owner app)

5. Shop orders and live queue

Method	Path	Auth	Android
GET	`/api/v1/shop/orders`	Owner session	YES
GET	`/api/v1/shop/orders/counts`	Owner session	YES
GET	`/api/v1/shop/orders/history`	Owner session	YES
GET	`/api/v1/shop/orders/history/summary`	Owner session	YES
GET	`/api/v1/shop/orders/history/export.csv`	Owner session	Optional
GET	`/api/v1/shop/orders/:id`	Owner session	YES
POST	`/api/v1/shop/orders/:id/transitions`	Owner session	YES
POST	`/api/v1/shop/orders/:id/print-now`	Owner session	YES
POST	`/api/v1/shop/orders/:id/print-confirmation`	Owner session	NO
POST	`/api/v1/shop/orders/:id/document-access`	Owner session	YES
POST	`/api/v1/shop/orders/:id/document-download`	Owner session	YES

GET /api/v1/shop/orders

LIVE QUEUE: tenant-scoped, keyset-paginated order list with filters.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	query `status` (one or comma list of ≤8 statuses), `active` (1/true hides COLLECTED/CANCELLED/EXPIRED), `print` (pending | initiated | attention), `sort` (newest|oldest, default newest), `cursor`, `limit` 1-100 (default 50). Strict: unknown keys → 400.
Request body	none
Response JSON	`{ data: OrderPage }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	Opaque keyset cursor `nextCursor` (base64url of `[createdAt, id]`). Omitted when there is no next page.
Idempotency	Safe.
Rate limit	shopRead 600/min per session.
Tables read	Order, Document, DevicePrintJob (print=attention)

Tables written	—
Android	YES (primary list; combine with SSE / push / polling)
Notes	`print=pending` = Print not yet pressed; `initiated` = pressed; `attention` = latest print job FAILED or NEEDS_RECONCILIATION.

GET /api/v1/shop/orders/counts

Cheap live counters for badges.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request body	none
Response JSON	`{ data: { pending: number; attention: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	shopRead.
Tables read	Order, Document, DevicePrintJob
Tables written	—
Android	YES (badge / polling fallback)

GET /api/v1/shop/orders/history

Order history (metadata only; IST calendar-day range, search).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	query `from`/`to` (`YYYY-MM-DD`, Asia/Kolkata, inclusive; default today; max 93 days), `status`, `q` (order number or filename contains), `cursor`, `limit` 1-100 (default 25).
Request body	none

Response JSON	`{ data: { range: { from: string; to: string; timezone: string }; items: { id: string; orderNumber: string; status: string; createdAt: string(ISO-8601); originalFilename: string; pageCount: number | null; selectedPageCount: number | null; copies: number | null; colourMode: string | null; sides: string | null; paperSize: string | null; totalPaise: number; printInitiatedAt: string(ISO-8601) | null; documentStatus: string; fileDeletedAt: string(ISO-8601) | null; handledBy: string | null }[]; nextCursor?: string } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	Opaque keyset cursor `nextCursor` (base64url of `[createdAt, id]`). Omitted when there is no next page.
Idempotency	Safe.
Rate limit	shopRead.
Tables read	Order, Document, OrderStatusHistory, User, ShopDevice
Tables written	—
Android	YES

GET /api/v1/shop/orders/history/summary

Counts for the same filters.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	query `from`, `to`, `status`, `q` (no cursor/limit).
Request body	none
Response JSON	`{ data: { timezone: string; range: { from: string; to: string }; total: number; byStatus: Record<string, number>; printInitiated: number; filesDeleted: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	—
Idempotency	Safe.
Rate limit	shopRead.
Tables read	Order, Document

Tables written	—
Android	YES

GET /api/v1/shop/orders/history/export.csv

CSV of the history filters (max 5000 rows; `x-export-truncated` header).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	query as history summary.
Request body	none
Response JSON	text/csv; charset=utf-8, `content-disposition: attachment`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	—
Idempotency	Safe (audited).
Rate limit	`RATE_LIMIT_SHOP_EXPORT_MAX` = 6/min per session.
Tables read	Order, Document
Tables written	AuditLog
Android	Optional (share/download)

GET /api/v1/shop/orders/:id

Order detail with document state, price snapshot and status history. Never returns storage keys or URLs.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request body	none
Response JSON	`{ data: OrderDetail }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 404 ORDER_NOT_FOUND
Pagination	—
Idempotency	Safe.

Rate limit	shopRead.
Tables read	Order, Document, OrderStatusHistory
Tables written	—
Android	YES

POST /api/v1/shop/orders/:id/transitions

Cancel an order. The ONLY accepted `toStatus` is `CANCELLED` (accept/ready/collected are retired; Print drives NEW→ACCEPTED→PRINTING).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request JSON	`{ toStatus: "CANCELLED"; reason?: string; clientRequestId: string }` `clientRequestId` = UUID. `reason` ≤250.
Response JSON	`{ data: { order: OrderCore } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 ORDER_NOT_FOUND, 409 INVALID_STATUS_TRANSITION
Pagination	—
Idempotency	Already CANCELLED → 200 no-op. Conditional update guards races (409 on concurrent change).
Rate limit	`RATE_LIMIT_STATUS_MAX` 120/min.
Tables read	Order, Document
Tables written	Order.status, OrderStatusHistory, AuditLog
Android	YES

POST /api/v1/shop/orders/:id/print-now

"Print": the one shop action. First success atomically sets `printInitiatedAt = now`, `deleteAfter = now + PRINT_RETENTION_MINUTES (30)`, moves the order NEW→ACCEPTED→PRINTING and returns a short-lived document URL.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request JSON	`{ clientRequestId: string }`

Response JSON	`{ data: { order: { id: string; orderNumber: string; status: string }; transitioned: boolean; firstPrint: boolean; document: { status: string; printInitiatedAt: string(ISO-8601) | null; deleteAfter: string(ISO-8601) | null }; access: { url: string; expiresAt: string(ISO-8601); contentDisposition: "inline"; mimeType: string } } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 ORDER_NOT_FOUND, 409 INVALID_STATUS_TRANSITION, 409 DOCUMENT_UNAVAILABLE
Pagination	—
Idempotency	Conditional claim: only one request starts retention; repeats just re-issue access (`firstPrint: false`) and never move `printInitiatedAt` / `deleteAfter`.
Rate limit	`RATE_LIMIT_STATUS_MAX` 120/min.
Tables read	Order, Document
Tables written	Document (status → PRINTED_RETENTION, printInitiatedAt, deleteAfter), Order.status, OrderStatusHistory, AuditLog
Android	YES (open the URL in a PDF viewer / print service)
Notes	It does NOT prove paper was printed (`PRINTED` is never set by this API any more).

POST /api/v1/shop/orders/:id/print-confirmation

RETIRED. Always answers 410.

Status	Deprecated (retired, returns 410)
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request body	none
Response JSON	no body
HTTP status	success — (never succeeds)
Application errors	410 ENDPOINT_RETIRED, 401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED
Pagination	—
Idempotency	—
Rate limit	`RATE_LIMIT_PRINT_CONFIRM_MAX` 60/min.
Tables read	—
Tables written	—
Android	NO: do not call.

POST /api/v1/shop/orders/:id/document-access

Reprint / view: short-lived INLINE signed URL for the original document. Never extends retention.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request body	none
Response JSON	`{ data: DocumentAccess }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 404 ORDER_NOT_FOUND, 410 DOCUMENT_UNAVAILABLE, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Safe to repeat; URL TTL ≤ min(300 s, time to `deleteAfter` / `expiresAt`).
Rate limit	`RATE_LIMIT_DOCUMENT_ACCESS_MAX` 60/min.
Tables read	Order, Document
Tables written	AuditLog
Android	YES

POST /api/v1/shop/orders/:id/document-download

Explicit "Save file": short-lived ATTACHMENT URL. Never automatic, never extends retention.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request body	none
Response JSON	`{ data: DocumentAccess }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 404 ORDER_NOT_FOUND, 410 DOCUMENT_UNAVAILABLE, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Safe to repeat.
Rate limit	60/min.
Tables read	Order, Document
Tables written	AuditLog
Android	YES

6. Printing and devices

Method	Path	Auth	Android
POST	`/api/v1/shop/devices/pairing-codes`	Owner session	Optional
PATCH	`/api/v1/shop/devices/:id`	Owner session	Optional
POST	`/api/v1/shop/devices/:id/revoke`	Owner session	Optional
PATCH	`/api/v1/shop/devices/:id/printers/:printerId`	Owner session	Optional
POST	`/api/v1/shop/orders/:id/print-jobs`	Owner session	Optional
GET	`/api/v1/shop/print-jobs`	Owner session	Optional
POST	`/api/v1/shop/print-jobs/:id/cancel`	Owner session	Optional
POST	`/api/v1/shop/print-jobs/:id/retry`	Owner session	Optional
POST	`/api/v1/shop/print-jobs/:id/reconcile`	Owner session	Optional
POST	`/api/v1/device/pair`	None	Only for a dedicated Android print device
POST	`/api/v1/device/heartbeat`	Device credential	Device mode only
PUT	`/api/v1/device/push-token`	Device credential	Device mode only
GET	`/api/v1/device/me`	Device credential	Device mode only
GET	`/api/v1/device/orders`	Device credential	Device mode only
GET	`/api/v1/device/orders/:id`	Device credential	Device mode only
POST	`/api/v1/device/orders/:id/print`	Device credential	Device mode only
POST	`/api/v1/device/orders/:id/reprint`	Device credential	Device mode only
POST	`/api/v1/device/orders/:id/download`	Device credential	Device mode only
PUT	`/api/v1/device/printers`	Device credential	NO
POST	`/api/v1/device/print-jobs/claim`	Device credential	NO
GET	`/api/v1/device/print-jobs/active`	Device credential	NO
POST	`/api/v1/device/print-jobs/:id/download`	Device credential	NO
POST	`/api/v1/device/print-jobs/:id/progress`	Device credential	NO
POST	`/api/v1/device/credential/rotate`	Device credential	Device mode only

POST /api/v1/shop/devices/pairing-codes

Generate a one-time pairing code `PB-XXXX-XXXX` (10 min, single use, max 5 outstanding).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ }`
Response JSON	`{ data: { code: string; expiresAt: string(ISO-8601) } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 409 CONFLICT (too many active codes)
Pagination	—
Idempotency	Not idempotent: each call mints a new code.
Rate limit	shopMutation 60/min.

Tables read	DevicePairingCode
Tables written	DevicePairingCode, AuditLog
Android	Optional (owner shows code to a Windows agent)

PATCH /api/v1/shop/devices/:id

Rename a device.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request JSON	`{ name: string }`
Response JSON	`{ data: { device: Device } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 NOT_FOUND
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation.
Tables read	ShopDevice
Tables written	ShopDevice, AuditLog
Android	Optional

POST /api/v1/shop/devices/:id/revoke

Revoke a device (its credential stops working immediately).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request JSON	`{ }`
Response JSON	`{ data: { device: Device } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 NOT_FOUND
Pagination	—

Idempotency	Idempotent.
Rate limit	shopMutation.
Tables read	ShopDevice
Tables written	ShopDevice (status REVOKED, pushToken cleared), AuditLog
Android	Optional

PATCH /api/v1/shop/devices/:id/printers/:printerId

Authorize a printer for printing and/or make it the shop default (one default per shop).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id` (device), `printerId`
Request JSON	`{ authorized?: boolean; isDefault?: boolean }` At least one field. Default requires authorized and present.
Response JSON	`{ data: { printer: Printer } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 NOT_FOUND, 409 PRINTER_NOT_AUTHORIZED, 409 PRINTER_UNAVAILABLE
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation.
Tables read	DevicePrinter, ShopDevice
Tables written	DevicePrinter, AuditLog
Android	Optional

POST /api/v1/shop/orders/:id/print-jobs

Queue the order for a Windows printer: starts retention via the print engine, then inserts a DevicePrintJob under a per-order lock.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id` (order)
Request JSON	`{ clientRequestId: string; printerId?: string; reprint?: boolean; acknowledgeDuplicateRisk?: boolean }` `clientRequestId` UUID (idempotency key, unique per shop). Default printer used when `printerId` omitted.

Response JSON	`{ data: { job: PrintJob; created: boolean } }`
HTTP status	success 201 (created) / 200 (same `clientRequestId` replayed)
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 ORDER_NOT_FOUND, 404 NOT_FOUND, 409 IDEMPOTENCY_CONFLICT, 409 ACTIVE_PRINT_JOB_EXISTS, 409 DUPLICATE_RISK_NOT_ACKNOWLEDGED, 409 PRINTER_NOT_AUTHORIZED, 409 PRINTER_UNAVAILABLE, 409 PRINTER_UNSUPPORTED_OPTIONS, 409 DOCUMENT_UNAVAILABLE
Pagination	—
Idempotency	`(shopId, clientRequestId)` unique. A job already in flight → 409 ACTIVE_PRINT_JOB_EXISTS; after PRINT_INITIATED a repeat needs `reprint: true`.
Rate limit	shopMutation 60/min.
Tables read	Order, Document, DevicePrinter, ShopDevice, DevicePrintJob
Tables written	Document, Order.status, OrderStatusHistory, DevicePrintJob, AuditLog
Android	Optional (requires a Windows agent)

GET /api/v1/shop/print-jobs

Print jobs: in-flight plus last 24 h (max 100), or all for one order.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	query `orderId?`
Request body	none
Response JSON	`{ data: { jobs: PrintJob[] } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	None (max 100).
Idempotency	Safe (also applies lease-expiry recovery).
Rate limit	shopRead.
Tables read	DevicePrintJob, DevicePrinter
Tables written	DevicePrintJob (lease recovery), AuditLog
Android	Optional

POST /api/v1/shop/print-jobs/:id/cancel

Cancel a job (QUEUED → CANCELED; in flight → CANCEL_REQUESTED, the agent confirms).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request JSON	`{ }`
Response JSON	`{ data: { job: PrintJob } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 PRINT_JOB_NOT_FOUND, 409 INVALID_STATUS_TRANSITION
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation.
Tables read	DevicePrintJob
Tables written	DevicePrintJob, AuditLog
Android	Optional

POST /api/v1/shop/print-jobs/:id/retry

Retry a FAILED job as a NEW job (history kept).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request JSON	`{ clientRequestId: string; acknowledgeDuplicateRisk?: boolean }`
Response JSON	`{ data: { job: PrintJob; created: boolean } }`
HTTP status	success 201 / 200 (replay)
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 PRINT_JOB_NOT_FOUND, 409 INVALID_STATUS_TRANSITION, 409 DUPLICATE_RISK_NOT_ACKNOWLEDGED, 409 ACTIVE_PRINT_JOB_EXISTS
Pagination	—
Idempotency	Same `clientRequestId` returns the same new job.
Rate limit	shopMutation.
Tables read	DevicePrintJob, DevicePrinter
Tables written	DevicePrintJob, AuditLog

Android	Optional

POST /api/v1/shop/print-jobs/:id/reconcile

Human resolution of an ambiguous NEEDS_RECONCILIATION job.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request JSON	`{ resolution: "PRINTED" | "NOT_PRINTED_REQUEUE" | "NOT_PRINTED_GIVE_UP"; clientRequestId?: string }` `clientRequestId` is required when `resolution = NOT_PRINTED_REQUEUE`.
Response JSON	`{ data: { job: PrintJob; next: PrintJob | null } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 PRINT_JOB_NOT_FOUND, 409 INVALID_STATUS_TRANSITION
Pagination	—
Idempotency	Only valid from NEEDS_RECONCILIATION.
Rate limit	shopMutation.
Tables read	DevicePrintJob
Tables written	DevicePrintJob, AuditLog
Android	Optional

POST /api/v1/device/pair

Redeem a pairing code → device record + ONE-TIME device credential.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Request JSON	`{ code: string; deviceName: string; platform: "ANDROID" | "WINDOWS"; appVersion?: string; osVersion?: string }`
Response JSON	`{ data: { deviceId: string; deviceCredential: string; shop: { displayName: string }; heartbeatIntervalSeconds: number } }`
HTTP status	success 200
Application errors	400 PAIRING_CODE_INVALID, 400 VALIDATION_ERROR, 429 RATE_LIMITED
Pagination	—
Idempotency	Single-use code: a lost response means generating a new code.
Rate limit	10/min per IP (devicePair) + failure ceilings (10 per client / 10 min, 1000 global).
Tables read	DevicePairingCode, Shop, Subscription
Tables written	DevicePairingCode.usedAt, ShopDevice, AuditLog
Android	Only for a dedicated Android print device (see guide §13)

POST /api/v1/device/heartbeat

Presence ping (writes `lastSeenAt` at most every 30 s).

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device
Request JSON	`{ appVersion?: string; osVersion?: string }`
Response JSON	`{ data: { serverTime: string(ISO-8601); heartbeatIntervalSeconds: number; device: { id: string; name: string } } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED
Pagination	—
Idempotency	Idempotent.
Rate limit	deviceMutation 120/min per device.
Tables read	ShopDevice
Tables written	ShopDevice
Android	Device mode only

PUT /api/v1/device/push-token

Register the device's FCM token (device-credential push; separate from user FCM registration).

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device (ANDROID)
Request JSON	`{ token: string }`
Response JSON	`{ data: { ok: true } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 409 CONFLICT (not an ANDROID device)
Pagination	—
Idempotency	Idempotent (same token = no write).
Rate limit	deviceMutation.
Tables read	ShopDevice
Tables written	ShopDevice.pushToken, AuditLog
Android	Device mode only

GET /api/v1/device/me

Who am I (device id, name, platform, shop name).

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device

Request body	none
Response JSON	`{ data: { device: { id: string; name: string; platform: string; shopName: string } } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	deviceRead 300/min.
Tables read	ShopDevice, Shop
Tables written	—
Android	Device mode only

GET /api/v1/device/orders

Order list for the device's shop (same query/response as `GET /shop/orders`).

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device
Parameters	query as `GET /shop/orders`.
Request body	none
Response JSON	`{ data: OrderPage }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	Opaque keyset cursor `nextCursor` (base64url of `[createdAt, id]`). Omitted when there is no next page.
Idempotency	Safe.
Rate limit	deviceRead.
Tables read	Order, Document
Tables written	—
Android	Device mode only

GET /api/v1/device/orders/:id

Order detail for the device's shop.

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device
Parameters	path `id`
Request body	none
Response JSON	`{ data: OrderDetail }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 404 ORDER_NOT_FOUND
Pagination	—
Idempotency	Safe.

Rate limit	deviceRead.
Tables read	Order, Document, OrderStatusHistory
Tables written	—
Android	Device mode only

POST /api/v1/device/orders/:id/print

Same print engine as `print-now` (device actor).

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device
Parameters	path `id`
Request JSON	`{ clientRequestId: string }`
Response JSON	`{ data: { order: { id: string; orderNumber: string; status: string }; transitioned: boolean; firstPrint: boolean; document: any; access: DocumentAccess } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 404 ORDER_NOT_FOUND, 409 INVALID_STATUS_TRANSITION, 409 DOCUMENT_UNAVAILABLE
Pagination	—
Idempotency	Same conditional-claim semantics as print-now.
Rate limit	deviceMutation.
Tables read	Order, Document
Tables written	Document, Order.status, OrderStatusHistory, AuditLog
Android	Device mode only

POST /api/v1/device/orders/:id/reprint

Reprint: inline URL; requires that Print was already initiated.

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device
Parameters	path `id`
Request JSON	`{ }`
Response JSON	`{ data: DocumentAccess }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 404 ORDER_NOT_FOUND, 409 INVALID_STATUS_TRANSITION, 410 DOCUMENT_UNAVAILABLE
Pagination	—

Idempotency	Safe to repeat.
Rate limit	deviceMutation.
Tables read	Order, Document
Tables written	AuditLog
Android	Device mode only

POST /api/v1/device/orders/:id/download

Attachment URL for the order document.

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device
Parameters	path `id`
Request JSON	`{ }`
Response JSON	`{ data: DocumentAccess }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 404 ORDER_NOT_FOUND, 410 DOCUMENT_UNAVAILABLE
Pagination	—
Idempotency	Safe to repeat.
Rate limit	deviceMutation.
Tables read	Order, Document
Tables written	AuditLog
Android	Device mode only

PUT /api/v1/device/printers

Windows agent reports its installed printers (replaces the device snapshot; never touches `authorized`/`isDefault`).

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device (WINDOWS)
Request JSON	`{ printers: { name: string; driverName?: string; connectivity: "ONLINE" | "OFFLINE" | "ERROR" | "UNKNOWN"; statusDetail?: string; supportsColour: boolean; supportsDuplex: boolean; paperSizes?: string[] }[] }` ≤50 printers, unique names.
Response JSON	`{ data: { printers: Printer[] } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 409 CONFLICT (not a Windows agent), 400 VALIDATION_ERROR

Pagination	—
Idempotency	Idempotent (upsert by `(deviceId, name)`).
Rate limit	deviceMutation.
Tables read	DevicePrinter
Tables written	DevicePrinter
Android	NO (Windows agent)

POST /api/v1/device/print-jobs/claim

Atomically lease the next QUEUED job (one in-flight job per device; an already-held job is returned again with `resumed: true`).

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device (WINDOWS)
Request JSON	`{ }`
Response JSON	`{ data: { job: AgentJob | null; resumed: boolean } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 409 CONFLICT (not a Windows agent)
Pagination	—
Idempotency	Idempotent resume of the held lease (lease 120 s).
Rate limit	deviceMutation.
Tables read	DevicePrintJob, DevicePrinter
Tables written	DevicePrintJob, AuditLog
Android	NO (Windows agent)

GET /api/v1/device/print-jobs/active

Non-terminal jobs of this device (restart recovery).

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device (WINDOWS)
Request body	none
Response JSON	`{ data: { jobs: AgentJob[] } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 409 CONFLICT
Pagination	None (max 20).
Idempotency	Safe (applies lease-expiry recovery).
Rate limit	deviceRead.
Tables read	DevicePrintJob

Tables written	DevicePrintJob (lease recovery)
Android	NO (Windows agent)

POST /api/v1/device/print-jobs/:id/download

Short-lived (≤120 s) download URL for the leased job's document.

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device (WINDOWS)
Parameters	path `id`
Request JSON	`{ leaseId: string }`
Response JSON	`{ data: { url: string; expiresAt: string(ISO-8601); mimeType: string; byteSize: number | null; sha256: string | null } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 404 PRINT_JOB_NOT_FOUND, 409 LEASE_MISMATCH, 409 INVALID_STATUS_TRANSITION, 410 DOCUMENT_UNAVAILABLE
Pagination	—
Idempotency	Safe while CLAIMED/DOWNLOADING.
Rate limit	deviceMutation.
Tables read	DevicePrintJob, Order, Document
Tables written	AuditLog
Android	NO (Windows agent)

POST /api/v1/device/print-jobs/:id/progress

Lease holder reports progress along explicit state edges.

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device (WINDOWS)
Parameters	path `id`
Request JSON	`{ leaseId: string; status: "DOWNLOADING" | "SUBMITTING" | "SUBMITTED_TO_SPOOLER" | "PRINT_INITIATED" | "FAILED" | "CANCELED" | "NEEDS_RECONCILIATION"; spoolerJobId?: number; spoolerState?: string; errorCode?: string; errorMessage?: string }`
Response JSON	`{ data: { job: AgentJob } }`
HTTP status	success 200

Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED, 404 PRINT_JOB_NOT_FOUND, 409 LEASE_MISMATCH, 409 INVALID_STATUS_TRANSITION, 409 JOB_CANCEL_REQUESTED
Pagination	—
Idempotency	Repeating the current status is a no-op that renews the lease (replay-safe).
Rate limit	deviceMutation.
Tables read	DevicePrintJob
Tables written	DevicePrintJob, AuditLog
Android	NO (Windows agent)

POST /api/v1/device/credential/rotate

Rotate the device credential (old one stays valid for 600 s or until the new one is first used).

Status	Verified from code
Authentication	Device credential: `Authorization: Bearer pbd_<43 chars>` issued once by `POST /device/pair`. Distinct from user access tokens.
Role / scope	device
Request JSON	`{ }`
Response JSON	`{ data: { deviceCredential: string; credentialVersion: number; previousValidForSeconds: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 401 DEVICE_REVOKED, 403 SHOP_SUSPENDED, 403 SUBSCRIPTION_INACTIVE, 429 RATE_LIMITED
Pagination	—
Idempotency	Each call issues a new secret; persist it before using it.
Rate limit	deviceMutation.
Tables read	ShopDevice
Tables written	ShopDevice, AuditLog
Android	Device mode only

7. Notifications / FCM

Method	Path	Auth	Android
PUT	`/api/v1/users/me/fcm-token`	Owner session	YES
DELETE	`/api/v1/users/me/fcm-token`	Owner session	YES
GET	`/api/v1/shop/notifications`	Owner session	YES
POST	`/api/v1/shop/notifications/read-all`	Owner session	YES
POST	`/api/v1/shop/notifications/:id/read`	Owner session	YES
GET	`/api/v1/shop/events`	Owner session	Optional

PUT /api/v1/users/me/fcm-token

Register / refresh the FCM token of this app installation (upsert by `(user, installationId)`).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ installationId: string; token: string; platform: "android" | "ios" | "web" }`
Response JSON	`{ data: { registered: true; installationId: string; platform: string } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Idempotent upsert; the same token under another account revokes the older registration.
Rate limit	shopMutation.
Tables read	FcmRegistration
Tables written	FcmRegistration, AuditLog
Android	YES (call on every `onNewToken` and after login)
Notes	Token is stored AES-256-GCM encrypted and is never returned. `installationId` must equal the one used at sign-in.

DELETE /api/v1/users/me/fcm-token

Revoke a push registration.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ installationId?: string; token?: string }` Send EXACTLY ONE of `installationId` or `token` (a DELETE with a JSON body).
Response JSON	no body
HTTP status	success 204
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Idempotent (absent/foreign → 204, no probing).
Rate limit	shopMutation.
Tables read	FcmRegistration
Tables written	FcmRegistration (revoked), AuditLog
Android	YES (on sign-out / push opt-out)

GET /api/v1/shop/notifications

Durable in-app notification inbox (newest first). The recovery path for anything missed by SSE/push.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	query `limit` 1-50 (20), `cursor` (older than this `seq`), `after` (newer than this `seq`, catch-up), `unreadOnly` (true|false|1|0).
Request body	none
Response JSON	`{ data: { items: NotificationDto[]; nextCursor: string | null; hasMore: boolean; latestCursor: string | null; unreadCount: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	Numeric `seq` cursors (decimal strings): `cursor` pages back, `after` catches up. Retention 14 days.
Idempotency	Safe.
Rate limit	shopRead.
Tables read	ShopNotification
Tables written	—
Android	YES

POST /api/v1/shop/notifications/read-all

Mark all notifications of the shop read.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Request JSON	`{ }`
Response JSON	`{ data: { updated: number; unreadCount: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only)
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation.
Tables read	ShopNotification
Tables written	ShopNotification.readAt
Android	YES

POST /api/v1/shop/notifications/:id/read

Mark one notification read.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	path `id`
Request JSON	`{ }`
Response JSON	`{ data: { notification: NotificationDto; unreadCount: number } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 CSRF_INVALID (cookie transport only), 404 NOT_FOUND
Pagination	—
Idempotency	Idempotent.
Rate limit	shopMutation.
Tables read	ShopNotification
Tables written	ShopNotification.readAt
Android	YES

GET /api/v1/shop/events

Server-Sent Events stream: `order.created`, `order.statusChanged`, `order.updated`, `printJob.updated`, `document.deletionScheduled`, `notification.created`, plus `: ping` heartbeats every ≤25 s.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	header `Last-Event-ID` (optional replay from a per-shop in-memory ring buffer)
Request body	none
Response JSON	`text/event-stream` (hijacked response; no JSON envelope)
HTTP status	success 200 (stream)
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 429 RATE_LIMITED (max 10 streams per shop, 5 per session, 30 connects/min)
Pagination	—
Idempotency	Reconnect with `Last-Event-ID`; then reconcile with `GET /shop/notifications?after=` and `GET /shop/orders`.
Rate limit	sseConnect 30/min per session.
Tables read	Session, ShopNotification
Tables written	—
Android	Optional (foreground only; OkHttp SSE with Bearer header). Push (FCM) is the background path.
Notes	The event bus is in memory on a single API instance; events from the worker process never arrive. Never rely on SSE alone.

8. Analytics

Method	Path	Auth	Android
GET	`/api/v1/shop/analytics`	Owner session	YES
GET	`/api/v1/shop/analytics/revenue`	Owner session	Optional
GET	`/api/v1/shop/analytics/revenue/export.csv`	Owner session	Optional

GET /api/v1/shop/analytics

Dashboard figures for a range (default today IST; max 93 days).

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	query `from`, `to` (ISO date-times).
Request body	none
Response JSON	`{ data: { timezone: string; range: { from: string(ISO-8601); to: string(ISO-8601) }; ordersToday: number; orderCount: number; pagesToday: number; estimatedOrderValuePaise: number; bwCount: number; colourCount: number; printsInitiated: number; newPrintRequests: number; documentsAutoDeleted: number; ordersByStatus: Record<string, number>; recentActivity: { orderId: string; orderNumber: string; fromStatus: string | null; toStatus: string; at: string(ISO-8601) }[] } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	—
Idempotency	Safe.
Rate limit	shopRead.
Tables read	Order, OrderStatusHistory, Document
Tables written	—
Android	YES

Notes	Money is an ESTIMATE of quoted order value; cash is settled outside the app.

GET /api/v1/shop/analytics/revenue

Estimated-order-value report (totals, daily series, breakdowns, previous-range comparison).

Status	Verified from code (inner `totals` / `daily` / `breakdown` shapes: see domain/revenue.ts; Requires confirmation before hard-typing)
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	query `from`, `to` (`YYYY-MM-DD`, IST, ≤93 days).
Request body	none
Response JSON	`{ data: { timezone: string; range: { from: string; to: string; days: number }; basis: "ESTIMATED_ORDER_VALUE"; collected: { available: false; reason: string }; totals: any; daily: any[]; breakdown: any; comparison: any } }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR
Pagination	—
Idempotency	Safe.
Rate limit	shopRead.
Tables read	Order (raw grouped SQL), Document
Tables written	—
Android	Optional

GET /api/v1/shop/analytics/revenue/export.csv

CSV of the revenue report rows.

Status	Verified from code
Authentication	Owner session: Shop owner (`SHOP_OWNER` with a shop). Android: `Authorization: Bearer <accessToken>` (no CSRF). Web: `printout_session` cookie + `X-CSRF-Token` on POST/PUT/PATCH/DELETE.
Role / scope	SHOP_OWNER
Parameters	query as revenue.
Request body	none
Response JSON	text/csv; charset=utf-8
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 SHOP_SUSPENDED, 429 RATE_LIMITED, 400 VALIDATION_ERROR

Pagination	—
Idempotency	Safe (audited).
Rate limit	6/min per session.
Tables read	Order, Document
Tables written	AuditLog
Android	Optional

9. Customer public APIs

Method	Path	Auth	Android
GET	`/api/v1/public/shops/:slug`	None	Customer flow / deep link only
POST	`/api/v1/public/shops/:slug/uploads/initiate`	None	Customer flow only
PUT	`/api/v1/public/uploads/:uploadId/content`	Upload token	Customer flow only
POST	`/api/v1/public/shops/:slug/uploads/:uploadId/complete`	None	Customer flow only
POST	`/api/v1/public/shops/:slug/orders`	None	Customer flow only
GET	`/api/v1/public/orders/:token`	Tracking token (path)	YES for the customer flow

GET /api/v1/public/shops/:slug

Public shop page data: name, address, contact, availability, branding, print capabilities, retention window. Non-ACTIVE shops return a minimal "unavailable" object.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Parameters	path `slug`
Request body	none
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	404 NOT_FOUND, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	60/min per IP.
Tables read	Shop, ShopSettings, Subscription
Tables written	—
Android	Customer flow / deep link only

Notes	Key fields: `status, slug, displayName, address, publicContact, acceptsOrders, availability{state,orderingBlocked,serverTime,...}, retentionMinutes, branding{brandColor}, printCapabilities`.

POST /api/v1/public/shops/:slug/uploads/initiate

Start an upload: creates an UPLOADING Document and returns a signed one-shot content URL.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Parameters	path `slug`
Request JSON	`{ fileName: string; byteSize: number; declaredMimeType: "application/pdf" | "image/jpeg" | "image/png" }`
Response JSON	`{ data: { uploadId: string; uploadUrl: string; requiredHeaders: { content-type: string }; expiresAt: string(ISO-8601); limits: { acceptedMimeTypes: string[]; maxBytes: number; maxPdfPages: number; imagePageCount: number } } }`
HTTP status	success 200
Application errors	404 SHOP_UNAVAILABLE, 409 SHOP_CLOSED, 422 EMPTY_FILE, 413 FILE_TOO_LARGE, 400 VALIDATION_ERROR, 429 RATE_LIMITED
Pagination	—
Idempotency	Not idempotent: each call creates a Document (abandoned ones are swept after 60 min).
Rate limit	20/min per IP (`PUBLIC_RATE_LIMIT_MAX/3`).
Tables read	Shop, ShopSettings, Subscription
Tables written	Document
Android	Customer flow only

PUT /api/v1/public/uploads/:uploadId/content

Stream the raw file bytes (single use; token in query).

Status	Verified from code
Authentication	Upload token: `token` query parameter from `uploads/initiate` (HMAC, 15 min, single use).
Role / scope	—
Parameters	path `uploadId`; query `token`; header `Content-Type` = declared MIME (or octet-stream)
Request body	Raw file bytes (not JSON). Max `UPLOAD_MAX_BYTES` (50 MiB default).
Response JSON	`{ data: { uploadId: string; byteSize: number } }`
HTTP status	success 200
Application errors	403 FORBIDDEN (token invalid/expired), 404 NOT_FOUND, 409 CONFLICT (already used), 413 FILE_TOO_LARGE, 415 INVALID_FILE_TYPE, 422 EMPTY_FILE, 429 RATE_LIMITED
Pagination	—
Idempotency	Single use: a second PUT → 409.

Rate limit	30/min per IP.
Tables read	Document
Tables written	Object storage (private bucket), Document.checksum / byteSize
Android	Customer flow only

POST /api/v1/public/shops/:slug/uploads/:uploadId/complete

Finish upload: inspects the stored file (magic bytes, PDF parse, page count) and marks the Document AVAILABLE (expires after `UNPRINTED_RETENTION_HOURS` = 24).

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Parameters	path `slug`, `uploadId`
Request JSON	`{ }`
Response JSON	`{ data: { documentId: string; detectedMimeType: string | null; byteSize: number; pageCount: number | null; documentStatus: string; uploadedAt: string(ISO-8601) | null; expiresAt: string(ISO-8601) | null } }`
HTTP status	success 200
Application errors	404 NOT_FOUND, 409 CONFLICT (bytes not received), 409 DOCUMENT_UNAVAILABLE, 413 FILE_TOO_LARGE, 422 INVALID_FILE_TYPE, 422 INVALID_PDF, 422 PASSWORD_PROTECTED_PDF, 422 PDF_TOO_MANY_PAGES, 422 EMPTY_FILE, 429 RATE_LIMITED
Pagination	—
Idempotency	Idempotent once AVAILABLE (returns the same payload).
Rate limit	60/min per IP.
Tables read	Document, Shop, Object storage
Tables written	Document
Android	Customer flow only

POST /api/v1/public/shops/:slug/orders

Create the order from a `quoteId`. The price is RE-computed from the shop's current active rules; nothing price-like is trusted from the token.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Parameters	path `slug`
Request JSON	`{ quoteId: string; customerDisplayNameOrReference?: string; clientRequestId: string }` `clientRequestId` UUID. `customerDisplayNameOrReference` ≤100.

Response JSON	`{ data: { orderNumber: string; trackingToken: string; status: string; totalPaise: number; currency: string } }`
HTTP status	success 200
Application errors	404 NOT_FOUND, 409 SHOP_UNAVAILABLE, 409 SHOP_CLOSED, 409 DOCUMENT_ALREADY_USED, 409 IDEMPOTENCY_CONFLICT, 422 INVALID_QUOTE, 422 QUOTE_EXPIRED, 422 DOCUMENT_UNAVAILABLE, 422 NO_PRICING_RULE, 400 VALIDATION_ERROR, 429 RATE_LIMITED
Pagination	—
Idempotency	`(shopId, clientRequestId)` unique: a retry returns the ORIGINAL order (HTTP 200, not 201). One order per Document.
Rate limit	60/min per IP.
Tables read	Shop, ShopSettings, Subscription, Document, PricingRule, Order
Tables written	Shop.orderCounter, Order, OrderStatusHistory, ShopNotification, FCM push (best effort)
Android	Customer flow only
Notes	Same transaction writes the order, its first history row and the notification (transactional outbox). Order numbers look like `CENTRA-0001`.

GET /api/v1/public/orders/:token

Customer order tracking by the unguessable `trackingToken`.

Status	Verified from code
Authentication	Tracking token (path): The unguessable `trackingToken` in the URL is the capability.
Role / scope	—
Parameters	path `token`
Request body	none
Response JSON	`{ data: { serverTime: string(ISO-8601); retentionMinutes: number; shopSlug: string; orderNumber: string; shopName: string; pickup: { address: string | null; contact: string | null }; printIssue: boolean; status: string; totalPaise: number; currency: string; createdAt: string(ISO-8601); updatedAt: string(ISO-8601); printOptions: PrintOptions; selectedPageCount: number | null; document: { fileName: string | null; pageCount: number | null; status: string; deleteAfter: string(ISO-8601) | null; deletedAt: string(ISO-8601) | null }; documentDeleteAfter: string(ISO-8601) | null; timeline: { status: string; at: string(ISO-8601) }[] } }`
HTTP status	success 200

Application errors	404 NOT_FOUND (unknown and malformed tokens are indistinguishable), 429 RATE_LIMITED
Pagination	—
Idempotency	Safe. Poll gently (≤ once per few seconds); no push exists for customers.
Rate limit	60/min per IP.
Tables read	Order, Shop, ShopSettings, Document, OrderStatusHistory, DevicePrintJob
Tables written	—
Android	YES for the customer flow
Notes	`printIssue` = latest print job FAILED/NEEDS_RECONCILIATION. `document.fileName` is null once the file is deleted. The token is a bearer secret: never log it.

10. Admin-only APIs

Method	Path	Auth	Android
GET	`/api/v1/admin/dashboard`	Admin session	NO
GET	`/api/v1/admin/orders`	Admin session	NO
GET	`/api/v1/admin/shops`	Admin session	NO
POST	`/api/v1/admin/shops`	Admin session	NO
GET	`/api/v1/admin/shops/:id`	Admin session	NO
PUT	`/api/v1/admin/shops/:id`	Admin session	NO
GET	`/api/v1/admin/plans`	Admin session	NO
POST	`/api/v1/admin/plans`	Admin session	NO
PUT	`/api/v1/admin/plans/:id`	Admin session	NO
GET	`/api/v1/admin/subscriptions`	Admin session	NO
PUT	`/api/v1/admin/subscriptions/:id`	Admin session	NO
GET	`/api/v1/admin/audit-logs`	Admin session	NO
GET	`/api/v1/admin/account`	Admin session	NO
POST	`/api/v1/admin/account/email`	Admin session	NO
POST	`/api/v1/admin/account/password`	Admin session	NO
GET	`/api/v1/admin/account/2fa`	Admin session	NO
POST	`/api/v1/admin/account/2fa/enroll`	Admin session	NO
POST	`/api/v1/admin/account/2fa/enroll/verify`	Admin session	NO
POST	`/api/v1/admin/account/2fa/recovery-codes/acknowledge`	Admin session	NO
POST	`/api/v1/admin/account/2fa/recovery-codes`	Admin session	NO
POST	`/api/v1/admin/account/2fa/disable`	Admin session	NO

GET /api/v1/admin/dashboard

Platform totals, status breakdowns, recent shops/orders.

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request body	none
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	adminRead 600/min
Tables read	Shop, Subscription, Order, Document, Plan
Tables written	—
Android	NO (admin web only)

GET /api/v1/admin/orders

Cross-shop order list (operational metadata only; no filenames, tokens or URLs).

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Parameters	query `shopId`, `status`, `cursor`, `limit` 1-100 (50)
Request body	none
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED
Pagination	Opaque keyset cursor `nextCursor` (base64url of `[createdAt, id]`). Omitted when there is no next page.
Idempotency	Safe.
Rate limit	adminRead 600/min
Tables read	Order, Document, Shop
Tables written	—
Android	NO (admin web only)

GET /api/v1/admin/shops

Shop list with owner, subscription, order count.

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).

Role / scope	PLATFORM_ADMIN
Parameters	query `status`, `q`, `cursor`, `limit`
Request body	none
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED
Pagination	Opaque keyset cursor `nextCursor` (base64url of `[createdAt, id]`). Omitted when there is no next page.
Idempotency	Safe.
Rate limit	adminRead 600/min
Tables read	Shop, Subscription, Plan, User, Order
Tables written	—
Android	NO (admin web only)

POST /api/v1/admin/shops

Create a shop + owner (password supplied once, Argon2id-hashed).

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request JSON	`{ slug: string; displayName: string; address?: string | null; planId?: string; owner: { email: string; displayName: string; password: string } }`
Response JSON	`{ data: any }`
HTTP status	success 201
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 409 CONFLICT
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	Plan
Tables written	Shop, ShopSettings, User, Subscription, AuditLog
Android	NO (admin web only)

GET /api/v1/admin/shops/:id

Shop detail with usage and pricing rules.

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Parameters	path `id`
Request body	none

Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 404 NOT_FOUND
Pagination	—
Idempotency	Safe.
Rate limit	adminRead 600/min
Tables read	Shop, User, Subscription, Order, PricingRule
Tables written	—
Android	NO (admin web only)

PUT /api/v1/admin/shops/:id

Update shop (status SUSPENDED revokes all shop sessions and refresh tokens immediately).

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Parameters	path `id`
Request JSON	`{ displayName?: string; address?: string | null; acceptsOrders?: boolean; status?: "ACTIVE" | "SUSPENDED" | "PILOT" | "PAST_DUE" | "CANCELLED" }`
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 404 NOT_FOUND, 409 CONFLICT
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	Shop
Tables written	Shop, Session, RefreshToken, AuditLog
Android	NO (admin web only)

GET /api/v1/admin/plans

Legacy billing: plans.

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request body	none
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED

Pagination	—
Idempotency	Safe.
Rate limit	adminRead 600/min
Tables read	Plan
Tables written	—
Android	NO (admin web only)

POST /api/v1/admin/plans

Legacy billing: create plan.

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request JSON	`{ name: string; pricePaise: number; active: boolean }`
Response JSON	`{ data: any }`
HTTP status	success 201
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 409 CONFLICT
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	—
Tables written	Plan, AuditLog
Android	NO (admin web only)

PUT /api/v1/admin/plans/:id

Legacy billing: update plan.

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Parameters	path `id`
Request JSON	`{ name?: string; pricePaise?: number; active?: boolean }`
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 404 NOT_FOUND, 409 CONFLICT
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	Plan
Tables written	Plan, AuditLog
Android	NO (admin web only)

GET /api/v1/admin/subscriptions

Legacy billing: subscriptions.

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Parameters	query `status`, `cursor`, `limit`
Request body	none
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED
Pagination	Opaque keyset cursor `nextCursor` (base64url of `[createdAt, id]`). Omitted when there is no next page.
Idempotency	Safe.
Rate limit	adminRead 600/min
Tables read	Subscription, Plan, Shop
Tables written	—
Android	NO (admin web only)

PUT /api/v1/admin/subscriptions/:id

Legacy billing: create/update the subscription of shop `:id` (`:id` is the SHOP id).

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Parameters	path `id` (shop id)
Request JSON	`{ status?: "ACTIVE" | "SUSPENDED" | "CANCELLED"; planId?: string; renewsAt?: string(ISO-8601) | null }`
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 404 NOT_FOUND, 409 CONFLICT
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	Shop, Plan, Subscription
Tables written	Subscription, AuditLog
Android	NO (admin web only)

GET /api/v1/admin/audit-logs

Audit log (metadata only).

Status	Verified from code

Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Parameters	query `shopId`, `action`, `cursor`, `limit`
Request body	none
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED
Pagination	Opaque keyset cursor `nextCursor` (base64url of `[createdAt, id]`). Omitted when there is no next page.
Idempotency	Safe.
Rate limit	adminRead 600/min
Tables read	AuditLog
Tables written	—
Android	NO (admin web only)

GET /api/v1/admin/account

Admin's own account and session info.

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request body	none
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	adminRead 600/min
Tables read	User, Session
Tables written	—
Android	NO (admin web only)

POST /api/v1/admin/account/email

Change own email (current password; signs out everywhere).

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request JSON	`{ currentPassword: string; newEmail: string }`

Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 INVALID_CURRENT_PASSWORD, 409 CONFLICT / EMAIL_UNAVAILABLE / TWO_FACTOR_ALREADY_ENABLED / NO_PENDING_ENROLLMENT / TWO_FACTOR_NOT_ENABLED
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	User
Tables written	User, Session, RefreshToken, AdminLoginChallenge, AuditLog
Android	NO (admin web only)

POST /api/v1/admin/account/password

Change own password (current password; signs out everywhere).

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request JSON	`{ currentPassword: string; newPassword: string }`
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 INVALID_CURRENT_PASSWORD, 409 CONFLICT / EMAIL_UNAVAILABLE / TWO_FACTOR_ALREADY_ENABLED / NO_PENDING_ENROLLMENT / TWO_FACTOR_NOT_ENABLED
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	User
Tables written	User, Session, RefreshToken, AdminLoginChallenge, AuditLog
Android	NO (admin web only)

GET /api/v1/admin/account/2fa

2FA status.

Status	Verified from code

Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request body	none
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED
Pagination	—
Idempotency	Safe.
Rate limit	adminRead 600/min
Tables read	AdminTwoFactor, AdminRecoveryCode
Tables written	—
Android	NO (admin web only)

POST /api/v1/admin/account/2fa/enroll

Start TOTP enrollment (returns otpauth URI once).

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request JSON	`{ currentPassword: string }`
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 INVALID_CURRENT_PASSWORD, 409 CONFLICT / EMAIL_UNAVAILABLE / TWO_FACTOR_ALREADY_ENABLED / NO_PENDING_ENROLLMENT / TWO_FACTOR_NOT_ENABLED
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	User, AdminTwoFactor
Tables written	AdminTwoFactor (pending), AuditLog
Android	NO (admin web only)

POST /api/v1/admin/account/2fa/enroll/verify

Confirm enrollment with a code; returns recovery codes once.

Status	Verified from code

Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request JSON	`{ code: string }`
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 INVALID_CURRENT_PASSWORD, 409 CONFLICT / EMAIL_UNAVAILABLE / TWO_FACTOR_ALREADY_ENABLED / NO_PENDING_ENROLLMENT / TWO_FACTOR_NOT_ENABLED
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	AdminTwoFactor
Tables written	AdminTwoFactor, AdminRecoveryCode, User.passwordHash (rollback guard), Session, AuditLog
Android	NO (admin web only)

POST /api/v1/admin/account/2fa/recovery-codes/acknowledge

Acknowledge stored recovery codes (204).

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request body	none
Response JSON	no body
HTTP status	success 204
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 INVALID_CURRENT_PASSWORD, 409 CONFLICT / EMAIL_UNAVAILABLE / TWO_FACTOR_ALREADY_ENABLED / NO_PENDING_ENROLLMENT / TWO_FACTOR_NOT_ENABLED
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	AdminTwoFactor
Tables written	AdminTwoFactor.recoveryAckAt
Android	NO (admin web only)

POST /api/v1/admin/account/2fa/recovery-codes

Regenerate recovery codes (password + TOTP).

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request JSON	`{ currentPassword: string; code: string }`
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 INVALID_CURRENT_PASSWORD, 409 CONFLICT / EMAIL_UNAVAILABLE / TWO_FACTOR_ALREADY_ENABLED / NO_PENDING_ENROLLMENT / TWO_FACTOR_NOT_ENABLED
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min
Tables read	AdminTwoFactor, AdminRecoveryCode
Tables written	AdminRecoveryCode, AuditLog
Android	NO (admin web only)

POST /api/v1/admin/account/2fa/disable

Disable 2FA (password + TOTP/recovery).

Status	Verified from code
Authentication	Admin session: `PLATFORM_ADMIN` session. Cookie + `X-CSRF-Token` on non-GET (Bearer is technically accepted by the guard but admin sign-in is web-only).
Role / scope	PLATFORM_ADMIN
Request JSON	`{ currentPassword: string; method?: "totp" | "recovery"; code: string }`
Response JSON	`{ data: any }`
HTTP status	success 200
Application errors	401 UNAUTHORIZED, 403 FORBIDDEN, 403 CSRF_INVALID, 429 RATE_LIMITED, 400 VALIDATION_ERROR, 403 INVALID_CURRENT_PASSWORD, 409 CONFLICT / EMAIL_UNAVAILABLE / TWO_FACTOR_ALREADY_ENABLED / NO_PENDING_ENROLLMENT / TWO_FACTOR_NOT_ENABLED
Pagination	—
Idempotency	See code (conditional updates).
Rate limit	adminMutation 60/min

Tables read	AdminTwoFactor
Tables written	AdminTwoFactor (deleted), AdminRecoveryCode (deleted), AdminLoginChallenge, Session, AuditLog
Android	NO (admin web only)

11. Operations (not part of the product API)

Method	Path	Auth	Android
GET	`/health`	None	Optional
GET	`/ready`	None	NO
GET	`/api/v1/internal/documents/:key`	Signed URL	Indirect

GET /health

Liveness (never touches the database). NOT under `/api/v1`.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Request body	none
Response JSON	`{ data: { status: "ok" } }`
HTTP status	success 200
Application errors	—
Pagination	—
Idempotency	Safe.
Rate limit	—
Tables read	—
Tables written	—
Android	Optional (connectivity probe)

GET /ready

Readiness (`SELECT 1` within 2 s). NOT under `/api/v1`.

Status	Verified from code
Authentication	None: Unauthenticated. Per-IP rate limit.
Role / scope	—
Request body	none
Response JSON	`{ data: { status: "ready" } }`
HTTP status	success 200
Application errors	503 INTERNAL_ERROR (database unavailable)
Pagination	—
Idempotency	Safe.
Rate limit	—
Tables read	PostgreSQL (SELECT 1)
Tables written	—
Android	NO

GET /api/v1/internal/documents/:key

Local-storage driver only: serves a stored document for a signed URL (`exp`, `sig`, `dl`). With the S3 driver, URLs point at the object store instead.

Status	Verified from code (active only when STORAGE_DRIVER=local; production driver Requires confirmation)
Authentication	Signed URL: HMAC `exp` + `sig` query parameters issued by the API (local storage driver only).
Role / scope	—
Parameters	path `key` (object key); query `exp`, `sig`, `dl` (1 = attachment)
Request body	none
Response JSON	File bytes (PDF/JPEG/PNG), supports `Range`
HTTP status	success 200 / 206
Application errors	403 FORBIDDEN, 404 NOT_FOUND, 410 DOCUMENT_UNAVAILABLE, 416
Pagination	—
Idempotency	Safe; DB row re-checked each request.
Rate limit	—
Tables read	Document, Local object storage
Tables written	—
Android	Indirect (the app opens URLs returned by print-now/document-access)

Not implemented (do not call)

These appear in the target-architecture spec (`docs/API_TARGET_SPEC.md`) but have **no route in the code**. They are listed so nobody integrates against them.

Item	Status
Customer accounts / customer login / persistent customer role	**Not implemented**: customers are anonymous; the tracking token is the only handle.
`PrintAgent` / `Printer` / `PrintJob` / `DeletionReceipt` / `CustomerSession` APIs	**Not implemented**: the tables exist (additive schema) but no route reads or writes them. The live print path uses `ShopDevice` / `DevicePrinter` / `DevicePrintJob`.
Payment gateway, `Invoice` / `Payment` APIs, customer payments	**Not implemented**: platform is free; no payment processing.
`GET /users/:id`, multi-shop switching, staff (`Membership`) endpoints	**Not implemented** (single owner, single shop).
Customer push notifications / customer FCM	**Not implemented**: customers poll `GET /public/orders/:token`.
Phase B smart-print / printer-management additions	**Not on** `main` (the branch `feature/daksh-phase-b-smart-print-printers` has no commits beyond `main` at the time of writing).

Retired / deprecated endpoints

Endpoint	Behaviour
`POST /api/v1/shop/orders/:id/print-confirmation`	Always `410 ENDPOINT_RETIRED`. Retention now starts at Print (`print-now`).
Order transitions Accept / Start printing / Ready / Collected via `POST /shop/orders/:id/transitions`	Rejected by validation: `toStatus` must be `CANCELLED`. Legacy `PRINTED/READY/COLLECTED` rows remain readable.

API ↔ database mapping

For each registered route: how the caller authenticates, which tables it **reads**, which it **writes**, and whether the Android app should use it. Table names are Prisma models (see full-er-diagram.md). `AuditLog` rows are appended by nearly every mutation; they are listed where the code writes one.

Android usage: **Yes** = part of the owner app; **Customer** = only for an Android customer flow; **Device mode** = only for a paired `pbd_` device; **No** = web/admin/Windows only.

1. Authentication and onboarding

Method	Endpoint	Authentication	Reads	Writes	Android usage
POST	`/api/v1/auth`	None	User, Shop, UserGoogleIdentity, UserPhoneIdentity, ShopSettings, Subscription, Plan, PricingRule, AuditLog	Session, RefreshToken, PhoneOnboardingGrant (new identity), FcmRegistration (optional), UserGoogleIdentity/UserPhoneIdentity (lastAuthTime), User.lastLoginAt, AuditLog	YES: the only Android sign-in endpoint.
POST	`/api/v1/auth/refresh`	None	RefreshToken, User, Shop, UserPhoneIdentity, UserGoogleIdentity	RefreshToken (rotatedAt / new row), Session (old invalidated, new created), AuditLog	YES
POST	`/api/v1/auth/logout`	Session, or refresh token in body	Session, RefreshToken, PhoneOnboardingGrant	Session.invalidatedAt, RefreshToken.revokedAt, FcmRegistration (revoked), PhoneOnboardingGrant (deleted), AuditLog	YES
POST	`/api/v1/auth/logout-all`	Any session	Session, RefreshToken	Session, RefreshToken, FcmRegistration, AuditLog	YES
GET	`/api/v1/auth/session`	Any session	Session, User, Shop	—	NO: use `GET /users/me` (richer, same auth).
POST	`/api/v1/auth/login`	None	User, Shop, AdminTwoFactor	Session, AdminLoginChallenge, User.lastLoginAt, AuditLog	NO: sets a cookie and returns a CSRF token. Use `POST /auth` with `loginMethod: "email"`.
POST	`/api/v1/auth/login/2fa`	Pending 2FA challenge cookie	AdminLoginChallenge, AdminTwoFactor, AdminRecoveryCode, User	AdminLoginChallenge, AdminTwoFactor.lastUsedStep, AdminRecoveryCode.usedAt, Session, AuditLog	NO (admin web only)
POST	`/api/v1/auth/login/2fa/cancel`	None	AdminLoginChallenge	AdminLoginChallenge.consumedAt	NO (admin web only)
POST	`/api/v1/auth/google/session`	None	UserGoogleIdentity, User, Shop	Session, PhoneOnboardingGrant, UserGoogleIdentity.lastAuthTime, AuditLog	NO: use `POST /auth` (`loginMethod: "google"`).
POST	`/api/v1/auth/phone/session`	None	UserPhoneIdentity, User, Shop	Session, UserPhoneIdentity.lastAuthTime, AuditLog	NO: use `POST /auth` (`loginMethod: "otp"`).
GET	`/api/v1/auth/phone/config`	None	—	—	Optional (feature detection)

Method	Endpoint	Authentication	Reads	Writes	Android usage
POST	`/api/v1/auth/firebase/exchange`	None	UserPhoneIdentity, User, Shop	Session, RefreshToken, PhoneOnboardingGrant, AuditLog	NO: `POST /auth` is the documented mobile entry (same logic).
POST	`/api/v1/shops/onboarding`	Restricted onboarding grant	PhoneOnboardingGrant, UserGoogleIdentity, UserPhoneIdentity, User	Shop, ShopSettings, User, UserGoogleIdentity or UserPhoneIdentity, Session, RefreshToken, AuditLog	YES (first-time Google users)
GET	`/api/v1/users/me/identities`	Owner session	User, UserPhoneIdentity, UserGoogleIdentity	—	YES (Settings)
POST	`/api/v1/users/me/identities/google`	Owner session	User, UserGoogleIdentity, UserPhoneIdentity	UserGoogleIdentity, Session (others ended), RefreshToken (others revoked), AuditLog	YES (Settings)
POST	`/api/v1/users/me/identities/google/unlink`	Owner session	User, UserGoogleIdentity, UserPhoneIdentity	UserGoogleIdentity (deleted), Session, RefreshToken, AuditLog	YES (Settings)
GET	`/api/v1/shop/account/phone`	Owner session	UserPhoneIdentity	—	Optional
POST	`/api/v1/shop/account/phone`	Owner session	User, UserPhoneIdentity	UserPhoneIdentity, Session, RefreshToken, AuditLog	Optional
POST	`/api/v1/shop/account/phone/unlink`	Owner session	User, UserPhoneIdentity, UserGoogleIdentity	UserPhoneIdentity (deleted), Session, RefreshToken, AuditLog	Optional

2. User profile

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/api/v1/users/me`	Session or restricted grant	User, UserPhoneIdentity, UserGoogleIdentity, Shop, ShopSettings, Subscription, Plan, PricingRule, AuditLog, PhoneOnboardingGrant	—	YES: call at app start / after refresh to drive navigation by `nextStep`.
PATCH	`/api/v1/users/me`	Any session	User	User, Session (others ended on email change), RefreshToken, AuditLog	YES
GET	`/api/v1/shops/me`	Owner session	User, Shop, ShopSettings, Subscription, Plan, PricingRule, AuditLog	—	YES (includes `rateInfo` = active pricing rules)
PATCH	`/api/v1/shops/me`	Owner session	Shop	Shop, AuditLog	YES
GET	`/api/v1/shops/me/qr`	Owner session	Shop	—	YES (render QR in-app)

3. Shop settings

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/api/v1/shop/settings`	Owner session	Shop, ShopSettings	—	YES
PUT	`/api/v1/shop/settings`	Owner session	Shop, ShopSettings	Shop, ShopSettings, AuditLog	YES
GET	`/api/v1/shop/hours`	Owner session	ShopSettings	—	YES
PUT	`/api/v1/shop/hours`	Owner session	ShopSettings	Shop.updatedAt (row lock), ShopSettings, AuditLog	YES
PUT	`/api/v1/shop/hours/temporary-closure`	Owner session	ShopSettings	ShopSettings, AuditLog	YES
DELETE	`/api/v1/shop/hours/temporary-closure`	Owner session	ShopSettings	ShopSettings, AuditLog	YES
POST	`/api/v1/shop/hours/reopen`	Owner session	ShopSettings	ShopSettings.reopenUntil, AuditLog	YES
DELETE	`/api/v1/shop/hours/reopen`	Owner session	ShopSettings	ShopSettings, AuditLog	YES
GET	`/api/v1/shop/qr`	Owner session	Shop	—	Prefer `GET /shops/me/qr` (readiness-aware).
GET	`/api/v1/shop/devices`	Owner session	ShopDevice, DevicePrinter	—	YES (read-only management view)

4. Shop pricing and quotes

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/api/v1/shop/pricing-rules`	Owner session	PricingRule	—	YES
POST	`/api/v1/shop/pricing-rules`	Owner session	PricingRule	PricingRule, AuditLog	YES
PUT	`/api/v1/shop/pricing-rules/:id`	Owner session	PricingRule	PricingRule, AuditLog	YES
DELETE	`/api/v1/shop/pricing-rules/:id`	Owner session	PricingRule	PricingRule (deleted), AuditLog	YES
POST	`/api/v1/public/shops/:slug/quotes`	None	Shop, ShopSettings, Subscription, Document, PricingRule	—	Customer flow only (not needed by the owner app)

5. Shop orders and live queue

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/api/v1/shop/orders`	Owner session	Order, Document, DevicePrintJob (print=attention)	—	YES (primary list; combine with SSE / push / polling)

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/api/v1/shop/orders/counts`	Owner session	Order, Document, DevicePrintJob	—	YES (badge / polling fallback)
GET	`/api/v1/shop/orders/history`	Owner session	Order, Document, OrderStatusHistory, User, ShopDevice	—	YES
GET	`/api/v1/shop/orders/history/summary`	Owner session	Order, Document	—	YES
GET	`/api/v1/shop/orders/history/export.csv`	Owner session	Order, Document	AuditLog	Optional (share/download)
GET	`/api/v1/shop/orders/:id`	Owner session	Order, Document, OrderStatusHistory	—	YES
POST	`/api/v1/shop/orders/:id/transitions`	Owner session	Order, Document	Order.status, OrderStatusHistory, AuditLog	YES
POST	`/api/v1/shop/orders/:id/print-now`	Owner session	Order, Document	Document (status → PRINTED_RETENTION, printInitiatedAt, deleteAfter), Order.status, OrderStatusHistory, AuditLog	YES (open the URL in a PDF viewer / print service)
POST	`/api/v1/shop/orders/:id/print-confirmation`	Owner session	—	—	NO: do not call.
POST	`/api/v1/shop/orders/:id/document-access`	Owner session	Order, Document	AuditLog	YES
POST	`/api/v1/shop/orders/:id/document-download`	Owner session	Order, Document	AuditLog	YES

6. Printing and devices

Method	Endpoint	Authentication	Reads	Writes	Android usage
POST	`/api/v1/shop/devices/pairing-codes`	Owner session	DevicePairingCode	DevicePairingCode, AuditLog	Optional (owner shows code to a Windows agent)
PATCH	`/api/v1/shop/devices/:id`	Owner session	ShopDevice	ShopDevice, AuditLog	Optional
POST	`/api/v1/shop/devices/:id/revoke`	Owner session	ShopDevice	ShopDevice (status REVOKED, pushToken cleared), AuditLog	Optional
PATCH	`/api/v1/shop/devices/:id/printers/:printerId`	Owner session	DevicePrinter, ShopDevice	DevicePrinter, AuditLog	Optional
POST	`/api/v1/shop/orders/:id/print-jobs`	Owner session	Order, Document, DevicePrinter, ShopDevice, DevicePrintJob	Document, Order.status, OrderStatusHistory, DevicePrintJob, AuditLog	Optional (requires a Windows agent)

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/api/v1/shop/print-jobs`	Owner session	DevicePrintJob, DevicePrinter	DevicePrintJob (lease recovery), AuditLog	Optional
POST	`/api/v1/shop/print-jobs/:id/cancel`	Owner session	DevicePrintJob	DevicePrintJob, AuditLog	Optional
POST	`/api/v1/shop/print-jobs/:id/retry`	Owner session	DevicePrintJob, DevicePrinter	DevicePrintJob, AuditLog	Optional
POST	`/api/v1/shop/print-jobs/:id/reconcile`	Owner session	DevicePrintJob	DevicePrintJob, AuditLog	Optional
POST	`/api/v1/device/pair`	None	DevicePairingCode, Shop, Subscription	DevicePairingCode.usedAt, ShopDevice, AuditLog	Only for a dedicated Android print device (see guide §13)
POST	`/api/v1/device/heartbeat`	Device credential	ShopDevice	ShopDevice	Device mode only
PUT	`/api/v1/device/push-token`	Device credential	ShopDevice	ShopDevice.pushToken, AuditLog	Device mode only
GET	`/api/v1/device/me`	Device credential	ShopDevice, Shop	—	Device mode only
GET	`/api/v1/device/orders`	Device credential	Order, Document	—	Device mode only
GET	`/api/v1/device/orders/:id`	Device credential	Order, Document, OrderStatusHistory	—	Device mode only
POST	`/api/v1/device/orders/:id/print`	Device credential	Order, Document	Document, Order.status, OrderStatusHistory, AuditLog	Device mode only
POST	`/api/v1/device/orders/:id/reprint`	Device credential	Order, Document	AuditLog	Device mode only
POST	`/api/v1/device/orders/:id/download`	Device credential	Order, Document	AuditLog	Device mode only
PUT	`/api/v1/device/printers`	Device credential	DevicePrinter	DevicePrinter	NO (Windows agent)
POST	`/api/v1/device/print-jobs/claim`	Device credential	DevicePrintJob, DevicePrinter	DevicePrintJob, AuditLog	NO (Windows agent)
GET	`/api/v1/device/print-jobs/active`	Device credential	DevicePrintJob	DevicePrintJob (lease recovery)	NO (Windows agent)
POST	`/api/v1/device/print-jobs/:id/download`	Device credential	DevicePrintJob, Order, Document	AuditLog	NO (Windows agent)

Method	Endpoint	Authentication	Reads	Writes	Android usage
POST	`/api/v1/device/print-jobs/:id/progress`	Device credential	DevicePrintJob	DevicePrintJob, AuditLog	NO (Windows agent)
POST	`/api/v1/device/credential/rotate`	Device credential	ShopDevice	ShopDevice, AuditLog	Device mode only

7. Notifications / FCM

Method	Endpoint	Authentication	Reads	Writes	Android usage
PUT	`/api/v1/users/me/fcm-token`	Owner session	FcmRegistration	FcmRegistration, AuditLog	YES (call on every `onNewToken` and after login)
DELETE	`/api/v1/users/me/fcm-token`	Owner session	FcmRegistration	FcmRegistration (revoked), AuditLog	YES (on sign-out / push opt-out)
GET	`/api/v1/shop/notifications`	Owner session	ShopNotification	—	YES
POST	`/api/v1/shop/notifications/read-all`	Owner session	ShopNotification	ShopNotification.readAt	YES
POST	`/api/v1/shop/notifications/:id/read`	Owner session	ShopNotification	ShopNotification.readAt	YES
GET	`/api/v1/shop/events`	Owner session	Session, ShopNotification	—	Optional (foreground only; OkHttp SSE with Bearer header). Push (FCM) is the background path.

8. Analytics

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/api/v1/shop/analytics`	Owner session	Order, OrderStatusHistory, Document	—	YES
GET	`/api/v1/shop/analytics/revenue`	Owner session	Order (raw grouped SQL), Document	—	Optional
GET	`/api/v1/shop/analytics/revenue/export.csv`	Owner session	Order, Document	AuditLog	Optional

9. Customer public APIs

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/api/v1/public/shops/:slug`	None	Shop, ShopSettings, Subscription	—	Customer flow / deep link only
POST	`/api/v1/public/shops/:slug/uploads/initiate`	None	Shop, ShopSettings, Subscription	Document	Customer flow only
PUT	`/api/v1/public/uploads/:uploadId/content`	Upload token	Document	Object storage (private bucket), Document.checksum / byteSize	Customer flow only
POST	`/api/v1/public/shops/:slug/uploads/:uploadId/complete`	None	Document, Shop, Object storage	Document	Customer flow only
POST	`/api/v1/public/shops/:slug/orders`	None	Shop, ShopSettings, Subscription, Document, PricingRule, Order	Shop.orderCounter, Order, OrderStatusHistory, ShopNotification, FCM push (best effort)	Customer flow only
GET	`/api/v1/public/orders/:token`	Tracking token (path)	Order, Shop, ShopSettings, Document, OrderStatusHistory, DevicePrintJob	—	YES for the customer flow

10. Admin-only APIs

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/api/v1/admin/dashboard`	Admin session	Shop, Subscription, Order, Document, Plan	—	NO (admin web only)
GET	`/api/v1/admin/orders`	Admin session	Order, Document, Shop	—	NO (admin web only)
GET	`/api/v1/admin/shops`	Admin session	Shop, Subscription, Plan, User, Order	—	NO (admin web only)
POST	`/api/v1/admin/shops`	Admin session	Plan	Shop, ShopSettings, User, Subscription, AuditLog	NO (admin web only)
GET	`/api/v1/admin/shops/:id`	Admin session	Shop, User, Subscription, Order, PricingRule	—	NO (admin web only)
PUT	`/api/v1/admin/shops/:id`	Admin session	Shop	Shop, Session, RefreshToken, AuditLog	NO (admin web only)
GET	`/api/v1/admin/plans`	Admin session	Plan	—	NO (admin web only)

Method	Endpoint	Authentication	Reads	Writes	Android usage
POST	`/api/v1/admin/plans`	Admin session	—	Plan, AuditLog	NO (admin web only)
PUT	`/api/v1/admin/plans/:id`	Admin session	Plan	Plan, AuditLog	NO (admin web only)
GET	`/api/v1/admin/subscriptions`	Admin session	Subscription, Plan, Shop	—	NO (admin web only)
PUT	`/api/v1/admin/subscriptions/:id`	Admin session	Shop, Plan, Subscription	Subscription, AuditLog	NO (admin web only)
GET	`/api/v1/admin/audit-logs`	Admin session	AuditLog	—	NO (admin web only)
GET	`/api/v1/admin/account`	Admin session	User, Session	—	NO (admin web only)
POST	`/api/v1/admin/account/email`	Admin session	User	User, Session, RefreshToken, AdminLoginChallenge, AuditLog	NO (admin web only)
POST	`/api/v1/admin/account/password`	Admin session	User	User, Session, RefreshToken, AdminLoginChallenge, AuditLog	NO (admin web only)
GET	`/api/v1/admin/account/2fa`	Admin session	AdminTwoFactor, AdminRecoveryCode	—	NO (admin web only)
POST	`/api/v1/admin/account/2fa/enroll`	Admin session	User, AdminTwoFactor	AdminTwoFactor (pending), AuditLog	NO (admin web only)
POST	`/api/v1/admin/account/2fa/enroll/verify`	Admin session	AdminTwoFactor	AdminTwoFactor, AdminRecoveryCode, User.passwordHash (rollback guard), Session, AuditLog	NO (admin web only)
POST	`/api/v1/admin/account/2fa/recovery-codes/acknowledge`	Admin session	AdminTwoFactor	AdminTwoFactor.recoveryAckAt	NO (admin web only)
POST	`/api/v1/admin/account/2fa/recovery-codes`	Admin session	AdminTwoFactor, AdminRecoveryCode	AdminRecoveryCode, AuditLog	NO (admin web only)
POST	`/api/v1/admin/account/2fa/disable`	Admin session	AdminTwoFactor	AdminTwoFactor (deleted), AdminRecoveryCode (deleted), AdminLoginChallenge, Session, AuditLog	NO (admin web only)

11. Operations (not part of the product API)

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/health`	None	—	—	Optional (connectivity probe)
GET	`/ready`	None	PostgreSQL (SELECT 1)	—	NO

Method	Endpoint	Authentication	Reads	Writes	Android usage
GET	`/api/v1/internal/documents/:key`	Signed URL	Document, Local object storage	—	Indirect (the app opens URLs returned by print-now/document-access)